

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第一篇

機器人的身體與系統

第 3 章 機器人的組成與系統架構

第 4 章 機構、連桿與自由度

第 5 章 致動器、馬達與傳動系統

第 6 章 感測器與機器人的感官

第 7 章 機器人電子系統與嵌入式控制

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 8 月

第一篇 篇章導讀

導讀篇建立了視野與方法，從這一篇開始，我們正式動手拆解機器人。這五章的任務只有一個：讓你能夠拿到一份需求規格，就畫得出系統方塊圖、算得出自由度、選得出馬達與感測器、設計得出電源與通訊架構，並列出完整的零件清單與成本。

編排的順序刻意由整體到局部。第 3 章先建立系統觀，把機器人拆解為七大子系統並理解它們之間的關係——這一步不能省略，因為先學零件再學系統的人，往往認得很多元件卻不知道怎麼組成一台會動的機器。有了系統的地圖，接下來四章才逐一深入：第 4 章分析機構與自由度，決定機器人能做什麼的物理上限；第 5 章選定致動器，賦予它力量；第 6 章配置感測器，賦予它感知；第 7 章則用電子系統把這一切連接起來，讓訊號與能量正確流動。

這一篇有一個貫穿全程的暗線：**故障**。第 3 章開場，一台醫院送藥機器人因為一條磨破的電線而報廢；第 5 章開場，一顆馬達被自己發的熱燒死；第 7 章開場，一台循跡車在比賽中反覆重開機，程式卻一行錯都沒有。這三則故事不是為了嚇唬讀者，而是要傳達一件教科書很少明說的事——機器人工程的成敗，往往不取決於你懂多少高深的演算法，而取決於你有沒有把最基本的物理現實照顧好。

這一篇要回答的六個問題

- 一台機器人由哪些子系統組成？它們如何分工與協作？
- 為什麼上位機與下位機必須分家？什麼任務算「即時」？
- 自由度是什麼？怎麼算？為什麼它不等於馬達數量？
- 馬達該選多大？為什麼幾乎每個關節都需要減速機？
- 各種感測器的物理原理決定了什麼樣的極限與失效模式？
- 為什麼超過一半的「程式問題」其實是電的問題？

讀完這一篇之後

你將擁有一位機器人系統工程師的基本功：能用工程師的語言描述一台機器人的完整構成、能做扭力與慣量的選型計算、能判讀感測器規格書中沒有明說的限制、能設計安全的電源與急停架構，並能在機器人故障時用系統性的方法快速定位問題。

但是，能組出一台會動的機器人，與能讓它精確地走到指定位置，中間還隔著一整片數學的

海洋。第二篇開始，我們將學習描述空間的語言——座標系、旋轉矩陣、齊次變換與運動學，把本篇建立的物理直覺，轉化為可以計算、可以最佳化、可以交給電腦執行的數學模型。

換句話說：第一篇教你把機器人組起來，第二篇開始教你把它算清楚。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 3 章	機器人的組成與系統架構	七大子系統、上下位機、即時性、感知—規劃—控制架構	「機器人有骨骼、肌肉、神經和大腦嗎？」——解剖一台報廢的送藥機器人
第 4 章	機構、連桿與自由度	關節類型、Grübler 公式、串聯與並聯、冗餘、工作空間	「為什麼人手很靈活，機器手臂卻常卡住？」——手掌貼牆卻能抬起手肘
第 5 章	致動器、馬達與傳動系統	扭力轉速曲線、減速機、慣量匹配、選型七步驟	「同樣是馬達，為什麼玩具車馬達不能做機器手臂？」——被自己發的熱燒死
第 6 章	感測器與機器人的感官	編碼器、IMU 互補濾波、測距原理、規格指標、校正	「閉著眼睛也能摸到鼻子，機器人做得到嗎？」——沒有感測器會說它壞了
第 7 章	機器人電子系統與嵌入式控制	GPIO、ADC、PWM、通訊、電源壓降、EMI、中斷、急停	「明明接好了電線，為什麼一動就重新開機？」——電源在說謊

第一篇 機器人的身體與系統

第 3 章

機器人的組成與系統架構

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一台被完全拆解的服務型機器人「解剖照」：鋁擠型骨架、兩顆帶減速機的馬達、主控板與驅動板、光達與相機、鋰電池、線束與紅色急停按鈕，分門別類地攤放在工作檯上，旁邊各以標籤註明其在系統中的角色——骨骼、肌肉、大腦、感官、心臟、神經、保險。

叫獸開講 「機器人有骨骼、肌肉、神經和大腦嗎？」

實驗室的解剖檯上，躺著一台壽終正寢的服務型機器人。它曾經在某家醫院的走廊送藥，服役三年，最後因為主機板燒毀而退役，被廠商捐給了叫獸的實驗室。今天下午的課，主題只有一個：把它拆開。

學生們圍成一圈，帶著手套與螺絲起子。第一層外殼卸下，露出一根鋁擠型的骨架，上面鑽滿了螺絲孔。「這是它的骨骼，」叫獸說，「決定它有多重、多硬、能承載多少東西。」接著是兩顆帶減速機的直流馬達，齒輪箱上還沾著三年份的灰塵。「這是肌肉。注意看，馬達本身轉得很快但沒有力氣，靠減速機把轉速換成扭力——就像人的肌肉配上槓桿一樣的骨頭。」

再往裡，一片主機板與四張小電路板現身。「這裡是大腦與脊髓。」叫獸指著中間那片：「這是主控電腦，跑地圖與路徑規劃，一秒鐘想十次。旁邊這四片是馬達驅動器，一秒鐘想一千次——它們負責把大腦說的『往前走』翻譯成每個輪子每毫秒該轉多快。」

有位學生舉手：「為什麼要分兩層？一台電腦全部做完不就好了？」叫獸拿起一條被燒黑的排線：「因為它們的時間尺度差一百倍。大腦可以偶爾卡頓零點一秒，你不會發現；但如果馬達控制迴路卡頓零點一秒，機器人就會撞牆。這叫『上位機與下位機』的分工，是所有機器人系統的共同架構。」

拆到最後，只剩一顆鋰電池、幾捆線與一個紅色的蘑菇頭按鈕。學生把零件在桌上排成一排：骨架、馬達、感測器、控制板、電池、線束、急停開關——七堆。叫獸環顧一圈：「現在請告訴我，這台機器人是怎麼死的？」

學生們翻找燒毀的主機板，卻在電池旁邊發現了答案：一條電源線的絕緣皮被機構的邊角磨破了，長期摩擦後短路，燒穿了電源模組，連帶燒毀主機板。「三年的研發、幾十萬的成本，」叫獸把那條線舉起來，「死於一條沒有加保護套管的電線。這就是為什麼我們要用一整章來談『系統』——因為機器人不是零件的加總，而是零件之間關係的總和。任何一個環節的疏忽，都能讓其他所有的精妙設計歸零。」

本章為什麼重要

第 1 章談了機器人是什麼，第 2 章談了怎麼學，從本章開始進入實質內容。本章的任務是建立「系統觀」——在深入每一個子系統（第 4~7 章的機構、馬達、感測器、電子）之前，先看

清楚它們如何組合成一台完整的機器人。這個順序很重要：如果先學馬達再學系統，你會知道很多零件卻不知道怎麼組；先建立系統觀，每學一個零件你都知道它在整體中的位置與責任。本章同時建立三個貫穿全書的重要概念：感知—規劃—控制的軟體分層、上位機與下位機的硬體分工、以及即時與非即時運算的時間尺度差異。這三個概念會在第 29 章的 ROS 2、第 34 章的產線整合中反覆出現。

學習目標

- 能列舉機器人系統的七大子系統，並說明各自的功能與相互關係。
- 能繪製一台機器人的系統方塊圖，標示訊號流與功率流。
- 能區分控制器、嵌入式電腦與微控制器的角色差異與選型考量。
- 能說明感測器系統的訊號鏈路：物理量、轉換、調理、取樣到數值。
- 能計算機器人的功率需求與電池續航時間。
- 能比較常見通訊介面的頻寬、距離與適用場合。
- 能說明感知—規劃—控制三層架構的資料流與各層時間尺度。
- 能解釋即時系統的意義，並判斷哪些任務必須即時、哪些不必。
- 能說明上位機與下位機的分工原則，並舉出實際的硬體組合。
- 能比較集中式與分散式機器人架構的優缺點與適用時機。

先備知識

第 1 章的感知—思考—行動循環概念。本章涉及少量電學計算（歐姆定律、功率與電池容量），若不熟悉可先閱讀附錄 D.1 的基本電學速成，或在閱讀 3.5 節時再回頭補充。本章不需要任何微積分或矩陣運算。

核心名詞中英對照

表 3-1 本章核心名詞

中文	英文	說明
子系統	Subsystem	構成完整系統的功能單元
機械本體	Mechanical Structure	機器人的骨架、連桿與外殼

中文	英文	說明
控制器	Controller	執行控制演算法、輸出指令的運算單元
微控制器	Microcontroller Unit, MCU	整合處理器與周邊介面的單晶片
單板電腦	Single Board Computer, SBC	具作業系統的小型電腦，如 Raspberry Pi
驅動器	Driver	將控制訊號放大為馬達所需功率的電路
訊號調理	Signal Conditioning	放大、濾波、隔離感測器訊號的處理
類比數位轉換器	Analog-to-Digital Converter, ADC	將連續電壓轉為數值的元件
取樣頻率	Sampling Rate	每秒讀取訊號的次數
即時系統	Real-Time System	必須在規定時限內完成回應的系統
控制週期	Control Cycle	控制迴路重複執行一次的時間間隔
上位機	Host Computer / High-Level Controller	負責規劃與決策的高階運算單元
下位機	Low-Level Controller	負責即時控制與硬體介面的運算單元
現場匯流排	Fieldbus	工業設備間的即時通訊網路，如 CAN、EtherCAT
系統方塊圖	System Block Diagram	以方塊與箭頭表示子系統與訊號流的圖

3.1 機器人的機械本體

3.1.1 骨架的三個任務

機械本體是機器人的骨骼與軀殼，承擔三個任務。第一是支撐：承受自身重量、負載重量與運動時的慣性力。第二是定位：保證各個關節與感測器的相對位置精確且穩定——如果相機的安裝支架會晃動，再好的視覺演算法也算不準物體位置。第三是保護：外殼隔絕粉塵、水氣與碰撞，同時避免人員接觸到運動件與高壓電。

這三個任務常常互相衝突。要支撐得穩就要粗壯，但粗壯就重；要保護周全就要密封，但密封就散熱困難。機器人的機構設計，本質上是在剛性、重量、成本與可維護性之間找平衡點——第 4 章與第 27 章會深入這個主題。

3.1.2 常見的結構形式與材料

實務上常見的結構形式有：鋁擠型骨架（模組化、易改裝，教學與研發首選）、鈹金外殼

(量產成本低、剛性佳)、鑄件(大型工業手臂的關節座,剛性最高但開模成本高)、碳纖維(輕量高剛性,用於無人機與競賽機器人)、以及 3D 列印件(快速原型與異形零件)。

材料的選擇有一個容易被忽略的關鍵指標：比剛度(剛度除以密度)。對移動機器人與機械手臂而言,增加的每一克重量都要消耗能量去搬運,並降低可用負載——這就是為什麼賽車與機器人都愛用鋁與碳纖維,而不是更便宜的鋼。

3.1.3 一個新手常犯的錯：忽略累積誤差

學生自製機器手臂時,常常每一節都做得「差不多」,結果末端偏差達數公分。原因是誤差會沿著運動鏈累積：第一關節的安裝偏差零點五度,經過三十公分的連桿放大後,末端就偏了將近三公釐；三個關節加起來,誤差可能達到一公分以上。這個現象在第 9 章的正向運動學中會有數學上的解釋——連桿越長、關節越多,對安裝精度的要求越高。

◎ 拆解練習的價值

如果你的實驗室或社團有報廢的印表機、掃地機器人或影印機,請務必拆開它。你會看到：工程師如何用最便宜的方式達成定位(塑膠齒輪加光遮斷器)、如何佈線避免磨損(套管與束帶)、如何做防呆(接頭形狀不同就插不進去)。這些細節在教科書裡佔不到半頁,卻是量產工程的精華。

3.2 控制器與嵌入式電腦

3.2.1 三個層級的運算單元

機器人的「大腦」通常不是一顆晶片,而是一組分工的運算單元。依運算能力與即時性由低到高排列,可分為三個層級。

微控制器(MCU)：如 Arduino 使用的 ATmega、STM32、ESP32。特點是內建類比數位轉換器、PWM 輸出、UART / I²C / SPI 介面,開機即執行程式,沒有作業系統的延遲,時序極為穩定。適合做馬達控制迴路、感測器讀取、安全監控。運算能力有限(通常數十至數百 MHz、數十至數百 KB 記憶體),無法跑影像辨識。

單板電腦(SBC)：如 Raspberry Pi、樹莓派同級產品。具備完整的 Linux 作業系統,可跑 Python、OpenCV 與 ROS 2,能處理影像、地圖與網路。缺點是作業系統會造成不可預測的延遲(排程、記憶體管理),不適合直接做毫秒級的控制迴路。

邊緣 AI 運算平台：如 NVIDIA Jetson 系列,在單板電腦的基礎上加入 GPU 或專用 AI 加速

器，能即時執行深度學習模型（物件偵測、語意分割、VLA 模型）。功耗與成本較高，通常只在需要視覺 AI 的機器人上。

表 3-2 三個層級運算單元的比較

層級	代表產品	運算能力	即時性	典型任務	本書章節
微控制器 MCU	Arduino、 STM32、ESP32	低（MHz 級）	極佳（無 OS 延遲）	馬達控制、感測器讀 取、急停監控	第 7、16 章
單板電腦 SBC	Raspberry Pi 5	中（GHz 級 CPU）	普通（OS 排 程延遲）	路徑規劃、SLAM、 ROS 2 節點	第 25、29 章
邊緣 AI 平 台	NVIDIA Jetson 系 列	高（含 GPU / NPU）	普通	影像辨識、深度學習 推論、VLA	第 22、31 ~ 33 章

3.2.2 選型的三個問題

面對琳瑯滿目的開發板，選型只需回答三個問題。第一，最嚴苛的即時要求是多少？如果需要一毫秒的控制迴路，就必須有 MCU。第二，最重的運算任務是什麼？如果要跑 YOLO 物件偵測，就需要 GPU 或 NPU。第三，開發時間有多少？MCU 的開發較繁瑣但穩定，SBC 上用 Python 開發極快但需處理作業系統的複雜性。

多數實用的機器人採用組合方案：SBC 或 Jetson 當上位機負責規劃與感知，MCU 當下位機負責即時控制，兩者用 UART 或 CAN 連接。這正是 3.8 節要展開的主題，也是開頭故事中那台送藥機器人的架構。

3.3 感測器系統

3.3.1 訊號鏈路：從物理世界到程式變數

感測器不是插上去就有數字。一個完整的感測器系統包含五個環節，任何一環出問題，程式讀到的都是垃圾。

第一是換能：把物理量轉成電訊號。應變片把力轉成電阻變化、光電二極體把光轉成電流、編碼器把角度轉成脈波。第二是訊號調理：微弱的訊號需要放大，高頻雜訊需要濾除，高壓或不同接地的訊號需要隔離。第三是類比數位轉換（ADC）：把連續的電壓轉成數值，解析度（幾位元）決定了能分辨多細的差異。第四是取樣：以固定頻率重複讀取，取樣頻率必須至少是訊號最高頻率的兩倍（Nyquist 定理），否則會產生混疊失真。第五是數值處理：單位換算、校正、濾

波，最後才成為程式裡那個叫做 distance 或 angle 的變數。

圖 3-1 感測器訊號鏈路

建議配圖：由左至右五個方塊——物理量（力／光／角度）→ 換能器 → 訊號調理（放大、濾波、隔離）→ ADC 與取樣 → 數值處理（校正、濾波）→ 程式變數。下方標註各環節常見的問題：靈敏度不足、雜訊、解析度不夠、混疊、單位錯誤。

3.3.2 內部感測與外部感測

如第 1 章所述，感測器分為兩大類。內部感測告訴機器人「我自己怎麼了」：關節角度（編碼器）、姿態與角速度（IMU）、馬達電流（間接反映負載與碰撞）、電池電壓。外部感測告訴機器人「外面有什麼」：距離（超音波、紅外線、光達）、影像（相機、深度相機）、力與接觸（力覺感測器、觸覺開關）、位置（GNSS、UWB 室內定位）。

一個關鍵的工程原則是：內部感測是控制的基礎，外部感測是自主的基礎。沒有編碼器，馬達連轉了幾度都不知道，連 PID 都做不了；沒有外部感測，機器人只能重播固定動作，無法應對環境變化。這也解釋了為什麼工業手臂可以只有編碼器（環境固定），而移動機器人必須有豐富的外部感測（環境未知）。

3.3.3 感測器融合的必要性

沒有任何單一感測器是完美的。編碼器精確但會累積誤差（輪子打滑時它一無所知）；IMU 反應快但會漂移；GNSS 不漂移但在室內失效且更新慢；相機資訊豐富但怕光線變化。實務上的解法是融合：用多個互補的感測器彼此校正。輪速計提供短期精確的位移、IMU 提供快速的姿態變化、光達提供絕對的環境參照，三者融合後的定位精度遠優於任何單一來源。這正是第 24 章 Kalman 濾波器的核心價值。

3.4 致動器與驅動器

3.4.1 致動器：能量轉換的終點

致動器把電能（或液壓、氣壓的能量）轉換成機械運動。機器人上最常見的是電動致動器：直流馬達（便宜、控制簡單）、無刷直流馬達（效率高、壽命長、功率密度大）、步進馬達（開迴路即可定位、低速扭力大）、伺服馬達（內建位置回授，直接下角度指令）。液壓致動器功率密度極高，用於大型工程機械與早期的人形機器人；氣壓致動器則柔順、便宜，常用於夾爪。第 5 章會完整比較。

3.4.2 驅動器：被忽略的關鍵環節

控制器輸出的是幾毫安培的訊號，馬達需要的是幾安培甚至數十安培的電流——中間必須有驅動器 (Driver) 做功率放大。這是初學者最常忽略、卻最常燒毀的環節。

驅動器的核心是功率電晶體組成的 H 橋電路，透過 PWM 訊號控制導通比例，決定馬達的等效電壓與轉速；改變橋臂的導通組合，則決定轉向。選用驅動器時必須注意三個規格：持續電流（能長時間承受的電流）、峰值電流（啟動或堵轉瞬間的電流，通常是持續電流的數倍）、以及散熱能力。學生常見的失敗是：只看馬達的額定電流選驅動器，忽略了堵轉電流可能是額定值的五到十倍，結果第一次撞牆就燒掉驅動器。

3.4.3 致動器與機構的匹配

馬達不是越大越好。選型時要同時考慮：所需扭力（含加速時的慣性扭力，不只是靜態負載）、所需轉速、減速比（減速機把高轉速低扭力換成低轉速高扭力）、以及慣量匹配（馬達轉子慣量與負載慣量相差太大時，控制會不穩定或反應遲鈍）。這幾項的計算會在第 5 章展開，本章只需建立一個觀念：致動器、減速機與機構是一個整體，不能分開選。

3.5 電源、電池與電源管理

3.5.1 為什麼電源是機器人的第一殺手

在教學現場，機器人「動不了」的原因排行榜上，電源問題長年高居第一。原因是電源同時牽涉三個容易出錯的面向：電壓不對（燒毀元件或帶不動）、電流不足（一啟動就重開機）、以及接地不當（訊號亂跳、通訊中斷）。而且電源問題的症狀往往偽裝成別的問題——程式看起來當機、感測器讀值突然亂跳、通訊莫名斷線，追查半天才發現是電池電壓在馬達啟動時瞬間掉到臨界值以下。

3.5.2 常見電池與關鍵規格

機器人常用的電池有三類。鋰聚合物電池 (LiPo) 放電電流大、能量密度高，是無人機與競賽機器人的主流，但過充過放會膨脹起火，必須使用專用充電器與平衡充。鋰離子電池 (Li-ion ，如 18650) 能量密度高、循環壽命長，適合續航需求高的移動機器人。鉛酸電池笨重但便宜且耐用，多見於大型 AGV 與工業設備。

選電池時要看四個規格：標稱電壓（決定能不能直接驅動馬達）、容量（以 mAh 或 Ah 表

示，決定續航）、放電倍率（C 數，決定能瞬間輸出多大電流）、以及重量。一個常被忽略的陷阱是放電倍率：一顆 2000 mAh、10C 的電池最大可放 20 A，但同樣容量 1C 的電池只能放 2 A——後者接上大電流馬達，電壓會瞬間崩塌。

3.5.3 電源架構：分離與穩壓

良好的機器人電源架構有兩個原則。第一是分離：功率電源（馬達、加熱器）與訊號電源（微控制器、感測器）盡量分開供電，或至少在電源入口分開走線，最後單點共地。這能避免馬達的電流突波拉低控制器電壓。第二是穩壓：電池電壓會隨放電下降（鋰電池從 4.2 V 降到 3.0 V），必須經過降壓（Buck）或升壓（Boost）模組穩壓後，才供給需要穩定電壓的元件。

此外還需要保護電路：過流保護（保險絲）、過放保護（低電壓切斷）、逆接保護（防止電池接反），以及反電動勢保護（馬達停止瞬間會產生反向電壓，需以二極體或電容吸收）。這些元件加起來成本不到兩百元，卻能保住價值數萬元的主控板。

圖 3-2 機器人電源架構示意圖

建議配圖：電池 → 主開關與保險絲 → 分為兩路：上路經降壓模組供給控制器與感測器（訊號電源，5 V / 3.3 V），下路直接供給馬達驅動器（功率電源）；兩路在電池負極單點共地。急停開關串接於功率電源路徑上。

3.5.4 續航估算

續航時間的粗略估算方式為：續航（小時）= 電池容量（Ah）× 可用比例 ÷ 平均消耗電流（A）。可用比例通常取 0.8（保留 20% 避免過放）。平均消耗電流需分項估算：主控電腦、感測器、馬達（依工作週期加權）。例題 3-1 會完整示範這個計算。

實務上有兩個經驗法則。第一，馬達的平均消耗遠低於峰值，但電源系統必須按峰值設計。第二，實測續航通常比計算值短 20% 至 30%，因為電池老化、低溫、與轉換效率損失。設計時務必留有餘裕。

3.6 通訊系統

3.6.1 機器人內部的三層通訊

一台機器人內部的通訊可分為三層。晶片級：微控制器與感測器晶片之間，使用 I²C（兩線、可掛多個裝置、速度較慢）或 SPI（四線、速度快、適合高速感測器如 IMU）。板級：控制板之間，使用 UART（簡單的點對點序列通訊）或 CAN Bus（多節點、抗雜訊強、有優先權機制，汽

車與工業標準)。系統級：上位機與外部世界之間，使用乙太網路、WiFi 或 5G。

工業機器人與高階系統則使用現場匯流排：EtherCAT (基於乙太網的即時協定，可達微秒級同步，適合多軸協調)、PROFINET、EtherNet/IP。這些協定的共同特徵是「確定性」——不只是快，而是每次傳輸的延遲都可預測，這對多軸同步至關重要。

表 3-3 常見通訊介面比較

介面	線數	速度	距離	多節點	典型用途
I ² C	2	100k ~ 1M bps	數十公分	可 (位址定址)	感測器、EEPROM
SPI	4+	數 M ~ 數十 Mbps	數十公分	可 (片選)	高速 IMU、SD 卡、顯示器
UART	2	9.6k ~ 1M bps	數公尺	否 (點對點)	模組通訊、除錯訊息
CAN Bus	2	125k ~ 1 Mbps	數十至數百公尺	可 (廣播 + 優先權)	多馬達驅動器、車輛
乙太網路	4 ~ 8	100M ~ 1 Gbps	百公尺	可 (交換機)	相機、光達、上位機
EtherCAT	4 ~ 8	100 Mbps (微秒級同步)	百公尺	可 (串接)	工業多軸即時控制
WiFi / 藍牙	無線	數 M ~ 數百 Mbps	數十公尺	可	遙控、監控、資料回傳

3.6.2 選擇通訊介面的三個考量

選介面時考慮三件事。第一是頻寬需求：一顆 IMU 每秒送幾百筆小資料，用 I²C 綽綽有餘；一台深度相機每秒送數十 MB，就必須用 USB 3 或乙太網路。第二是即時性需求：控制指令必須準時抵達，適合 CAN 或 EtherCAT；影像串流可以容忍偶爾延遲，用 WiFi 即可。第三是抗雜訊能力：機器人內部有大電流馬達，長距離的訊號線容易受干擾，CAN 的差動訊號比 UART 的單端訊號可靠得多。

◎ 無線通訊的陷阱

許多學生把 WiFi 用在控制迴路上——遙控時感覺良好，比賽時卻因為現場數百支手機的干擾而失控。原則是：安全相關與即時控制絕不依賴無線；無線只用於監控、遙測與非即時的指令下達。急停功能更必須是硬體線路，不能透過任何通訊協定。

3.7 軟體與演算法

3.7.1 機器人軟體的三層堆疊

機器人的軟體同樣分層。最底層是韌體與驅動層：直接操作硬體暫存器，讀取編碼器、輸出 PWM、處理中斷。中間層是控制與狀態估測層：PID 迴路、運動學計算、感測器融合、里程計。最上層是規劃與決策層：路徑規劃、任務排程、行為決策、人機介面。

這種分層有兩個好處。第一是可替換性：換一顆不同的馬達，只需改韌體層，上層演算法不動。第二是可測試性：每一層可以獨立驗證——先確認編碼器讀值正確，再測試 PID 是否收斂，最後才測整體路徑追蹤。跳過分層測試而直接跑全系統，是初學者最常見的除錯災難。

3.7.2 中介框架：為什麼需要 ROS 2

當機器人的軟體模組超過五個，就會面臨一個問題：模組之間如何交換資料？自己寫通訊機制不僅費時，還很難支援跨語言（C++ 與 Python 混用）與跨機器（上位機與遠端電腦）。ROS 2 正是為此而生的中介框架：它提供節點（獨立的程式模組）、話題（發布訂閱式的資料流）、服務（請求回應式的呼叫）、動作（有進度回報的長時間任務）等機制，讓模組化開發變得標準化。

此外，ROS 2 的生態系提供了大量現成套件：SLAM、路徑規劃、感測器驅動、視覺化工具，可以直接站在巨人的肩膀上。第 29 章會完整介紹，本章只需理解它在系統中的定位——它是軟體模組之間的「神經系統」。

3.7.3 狀態機：機器人行為的骨架

除了演算法，機器人軟體還需要一個行為的組織方式，最常見的是有限狀態機（Finite State Machine）。以送藥機器人為例，狀態可能包括：待命、前往取藥點、等待裝載、前往病房、等待取藥、返回充電。每個狀態有明確的進入條件、執行內容與離開條件。狀態機的價值在於行為變得可預測、可測試、可除錯——當機器人卡住時，第一個問題永遠是「它現在在哪個狀態？」

3.8 上位機與下位機

3.8.1 為什麼要分工

開頭故事中學生的疑問——「為什麼不用一台電腦全部做完」——答案是時間尺度。控制迴路需要每一毫秒執行一次且不能延遲，這是即時任務；路徑規劃可能需要幾十毫秒計算一次，偶爾慢一點無妨，這是非即時任務。把兩者放在同一個作業系統上，作業系統的排程隨時可能為了處理網路封包而打斷控制迴路，導致馬達失控。

因此標準做法是硬體上分家：上位機（SBC 或工控機，跑 Linux 與 ROS 2）負責感知、地圖、規劃與人機介面；下位機（MCU，裸機或即時作業系統）負責馬達控制迴路、感測器高頻取樣、安全監控。兩者以 UART、CAN 或乙太網路連接，上位機下達「速度指令」，下位機回傳「編碼器與狀態」。

圖 3-3 上位機與下位機的分工架構

建議配圖：上半部為上位機（Jetson / Raspberry Pi）方塊，內含 SLAM、路徑規劃、視覺辨識、ROS 2 節點；下半部為下位機（STM32 / Arduino）方塊，內含 PID 控制迴路、編碼器讀取、急停監控。中間以雙向箭頭連接，上行標註「速度指令（10~50 Hz）」，下行標註「里程計與狀態回報（100~1000 Hz）」，下位機再連接馬達驅動器與感測器。

3.8.2 介面設計的原則

上下位機之間的介面設計，有三個實務原則。第一是抽象層級要一致：上位機應該說「以每秒零點五公尺前進、每秒零點二弧度左轉」，而不是「左輪 PWM 給 180、右輪給 150」——後者把機構細節洩漏到了上層。第二是要有超時保護：如果上位機當機或連線中斷，下位機必須在數百毫秒內自動停止馬達，而不是維持最後一個指令繼續衝出去。第三是狀態要雙向：下位機不只回傳感測值，也要回傳自身狀態（是否過流、是否急停、電池電壓），讓上位機能做出正確決策。

3.8.3 典型的硬體組合

依規模不同，常見的組合有：入門級——ESP32 單機兼任上下位（適合循跡小車，因為運算需求低）；中階——Raspberry Pi 5 加 STM32 或 Arduino，以 UART 連接（適合教學型 AMR）；高階——NVIDIA Jetson 加多顆馬達驅動器，以 CAN 或 EtherCAT 連接（適合研究型移動機器人與機械手臂）；工業級——工控機加專用運動控制卡，以 EtherCAT 連接伺服驅動器（適合產線設備）。

3.9 即時控制與非即時運算

3.9.1 什麼是即時

「即時」(Real-Time) 不等於「快」，而是「準時」。一個即時系統的定義是：不僅要算出正確答案，還必須在規定的時限 (deadline) 之前算出來，否則答案就失去價值。一個平均延遲一毫秒但偶爾延遲一百毫秒的系統，不是即時系統；一個穩定延遲五毫秒的系統，反而是。

即時性又分兩級。硬即時 (Hard Real-Time)：錯過時限會造成災難，例如手臂的力矩控制迴路、無人機的姿態控制、急停響應。軟即時 (Soft Real-Time)：錯過時限只降低品質，例如影像顯示掉幀、地圖更新慢一拍。分清楚哪些任務是硬即時，是系統設計的核心決策。

3.9.2 典型的時間尺度

機器人系統中，不同層級的控制週期有數量級的差異，理解這個「時間金字塔」對系統設計極為重要。電流迴路 (馬達驅動器內部)：約 10 ~ 100 微秒。速度與位置迴路 (下位機)：約 0.1 ~ 1 毫秒。運動學與軌跡插補：約 1 ~ 10 毫秒。狀態估測與定位：約 10 ~ 100 毫秒。路徑規劃：約 100 毫秒至數秒。任務決策與 AI 推論：數百毫秒至數秒。

這個金字塔解釋了許多設計選擇。為什麼馬達驅動器要有自己的處理器？因為 10 微秒的迴路無法交給上層。為什麼視覺不能直接進控制迴路？因為相機每秒三十幀 (33 毫秒)，比控制迴路慢了兩個數量級——這正是視覺伺服控制 (第 22 章) 的核心難題。

表 3-4 機器人系統的時間尺度金字塔

層級	典型週期	執行位置	即時性要求	範例任務
電流迴路	10 ~ 100 μ s	馬達驅動器	硬即時	PWM 換相、電流控制
速度 / 位置迴路	0.1 ~ 1 ms	下位機 MCU	硬即時	PID 控制、編碼器讀取
軌跡插補	1 ~ 10 ms	下位機或上位機	硬即時	運動學計算、軌跡生成
狀態估測	10 ~ 100 ms	上位機	軟即時	感測器融合、里程計、定位
路徑規劃	0.1 ~ 數秒	上位機	非即時	A*、RRT、避障重規劃
任務決策 / AI	數百 ms ~ 數秒	上位機 / 邊緣 AI	非即時	物件辨識、任務排程、LLM

3.9.3 如何達成即時性

達成即時性有三個層次的做法。最簡單的是裸機程式 (bare-metal)：MCU 上沒有作業系統，用計時器中斷觸發控制迴路，時序最精確，適合單一控制任務。其次是即時作業系統 (RTOS，如 FreeRTOS)：提供多任務與優先權排程，高優先權任務可搶佔低優先權任務，適

合任務較多的下位機。最複雜的是即時 Linux (PREEMPT_RT 補丁)：在完整的 Linux 上取得毫秒級的可預測性，適合需要同時跑 ROS 2 與即時控制的工控機。

此外還有一個常被忽略的實務要點：即時任務中絕對不能做「執行時間不可預測」的事——不要動態配置記憶體、不要印出除錯訊息、不要等待網路回應。這三件事是初學者破壞即時性的三大元凶。

3.10 感知—規劃—控制架構

3.10.1 三層架構的資料流

把前面各節整合起來，機器人的軟體系統呈現一個經典的三層架構：感知層、規劃層、控制層。這是第 1 章「感知—思考—行動」在工程實作上的具體化。

感知層負責回答「世界現在是什麼樣子」。它接收原始感測資料（影像、點雲、編碼器脈波、IMU 讀值），經過濾波、融合與特徵擷取，輸出一個結構化的世界模型：我在地圖上的哪個座標、周圍有哪些障礙物、目標物在哪裡。這一層的產出是「狀態」與「地圖」。

規劃層負責回答「我該做什麼」。它接收感知層的世界模型與任務目標，輸出一條可執行的軌跡或一串動作序列。這一層包含全域路徑規劃（從 A 到 B 怎麼走）、區域避障（前面突然出現行人怎麼繞）、以及任務層決策（先送藥還是先充電）。

控制層負責回答「怎麼讓身體照做」。它接收規劃層的軌跡，計算每個關節或輪子該輸出多少力矩或速度，並透過回授不斷修正誤差。這一層直接與硬體對話，也是即時性要求最高的一層。

圖 3-4 感知—規劃—控制三層架構

建議配圖：三個橫向色帶由上而下——感知層（相機、光達、IMU、編碼器 → 融合 → 狀態與地圖）、規劃層（任務目標 + 地圖 → 全域路徑 → 區域避障 → 軌跡）、控制層（軌跡 → 運動學 → PID → 驅動器 → 馬達）。右側標註各層的更新頻率；左側以一條回饋箭頭由馬達經環境回到感測器，形成閉環。

3.10.2 各層的時間尺度與耦合

三層的更新頻率遞增：規劃層最慢（0.1 ~ 數秒）、感知層居中（10 ~ 100 毫秒）、控制層最快（0.1 ~ 1 毫秒）。這個梯度是自然的——世界的變化沒有那麼快，但馬達的物理響應非常快。

但三層並非完全獨立。當感知層發現前方突然出現障礙物時，必須能快速觸發規劃層重新規

劃，甚至直接觸發控制層緊急煞車——這條「反射弧」通常繞過規劃層，直接由感知連到控制，就像人類碰到燙的東西會先縮手再思考。這種設計在機器人學中稱為反應式行為（reactive behavior），與深思熟慮的規劃式行為（deliberative behavior）互補。

3.10.3 架構的演變：從三層到端到端

值得注意的是，這個經典三層架構近年正受到挑戰。深度學習與 VLA 模型的興起，帶來了「端到端」（end-to-end）的可能性：直接把相機影像輸入神經網路，輸出馬達指令，中間不再有明確的地圖與規劃層。兩種路線各有優劣：三層架構可解釋、可除錯、可驗證安全性；端到端架構能處理難以建模的複雜情境，但像個黑盒子。目前的主流看法是混合——用學習處理感知與技能，用傳統架構保證安全與可靠。第 33 章會深入這個辯論。

3.11 集中式與分散式機器人架構

3.11.1 集中式：一個大腦說了算

集中式架構把所有運算集中在一台主控電腦上：它讀取所有感測器、執行所有演算法、輸出所有指令，周邊只有簡單的訊號轉換電路。優點是設計簡單、資料同步容易、除錯集中。缺點是：布線複雜（所有感測器的線都要拉到主控）、可靠性差（主控當機則全機癱瘓）、擴充困難（增加一顆馬達可能就得換主控板）。小型教學機器人多採此架構。

3.11.2 分散式：各司其職的節點網路

分散式架構把運算分散到多個節點：每顆馬達有自己的智慧驅動器（內含電流與速度迴路）、每組感測器有自己的處理模組，彼此以 CAN 或 EtherCAT 連接，主控只負責高層協調。優點是布線簡潔（一條匯流排串接所有節點）、可靠性高（單一節點故障可隔離）、易於擴充（新增節點只需接上匯流排並設定位址）。缺點是成本較高、需要處理通訊協定與時間同步。工業機器人、汽車與大型移動平台幾乎都採此架構。

表 3-5 集中式與分散式架構的比較

面向	集中式	分散式
運算位置	單一主控	多個智慧節點
布線	星狀，線束複雜	匯流排，串接簡潔
可靠性	低（單點故障即全癱）	高（可隔離故障節點）
擴充性	差（受主控 I/O 限制）	佳（掛上匯流排即可）

面向	集中式	分散式
成本	低	較高
除錯難度	低（狀態集中）	較高（需觀察匯流排）
典型應用	教學機器人、小型專題	工業機器人、AGV、汽車

3.11.3 混合式與實務選擇

實務上多數機器人採混合式：關鍵的即時控制分散到智慧驅動器，感知與規劃集中在一台上位機。選擇的原則是——依「軸數」與「距離」決定。軸數少（三軸以下）、距離短（同一塊板子上）採集中式即可；軸數多（六軸以上）、距離長（分佈在整台車上）則必須分散式。第 34 章討論產線整合時，會看到這個原則如何延伸到整個工廠的層級。

3.12 機器人的系統方塊圖

3.12.1 方塊圖的價值

系統方塊圖是機器人工程師的共同語言。它用方塊代表子系統、用箭頭代表訊號或功率的流向，讓整台機器人的架構在一頁紙上一目了然。它的價值有三：溝通（讓機構、電子與軟體工程師對齊認知）、設計（在動手前發現介面缺口與衝突）、除錯（故障時能快速定位到某個方塊或某條連線）。

3.12.2 畫方塊圖的五個步驟

步驟一，先畫出四個核心方塊：感測器、控制器、驅動器、致動器，並以箭頭連成閉環（別忘了從致動器經「環境」回到感測器的那條回饋線）。步驟二，加入電源系統，用不同顏色或粗線標示功率流，與訊號流區分開來。步驟三，若採上下位機架構，將控制器方塊拆成上位機與下位機，標註兩者的通訊介面與方向。步驟四，在每條連線上標註介面規格（I²C、CAN、PWM）與更新頻率。步驟五，補上安全相關元件：急停開關、保險絲、過流保護，並確認急停能直接切斷功率而不經過軟體。

圖 3-5 教學型自主移動機器人系統方塊圖範例

建議配圖：完整方塊圖。左側感測器群（光達、深度相機、IMU、編碼器）；中央上位機（Raspberry Pi 5：SLAM、路徑規劃、ROS 2）以 UART 連下位機（STM32：PID、里程計、急停監控）；右側馬達驅動器與兩顆直流馬達。下方電池經開關、保險絲、急停後分為功率路（粗線至驅動器）與訊號路（經降壓模組至上下位機與感測器），單點共地。各連線標註介面與頻率。

3.12.3 由方塊圖到規格表

方塊圖畫完後，下一步是把每個方塊與連線轉成規格表：每個元件的型號、電壓、電流、通訊介面、更新頻率、成本。這份表格就是採購清單與設計文件的基礎，也是第 36 章系統工程流程的起點。學生專題最常見的失敗，就是跳過這一步直接買零件——結果買回來才發現感測器是 5 V 訊號、主控是 3.3 V，或是驅動器的電流撐不住馬達的堵轉。

完整計算例題

例題 3-1 電池續航估算

題目：一台教學型 AMR 使用 12 V、10 Ah 的鋰電池。系統耗電如下：主控電腦（Raspberry Pi 5 含光達）平均 1.5 A；下位機與感測器 0.3 A；兩顆驅動馬達在行進時各 2.0 A，但工作週期（行進時間佔比）為 60%。取電池可用比例 80%。求理論續航時間。若實測續航為 1.8 小時，求折損率並分析可能原因。

解：先計算平均消耗電流。馬達平均電流 = $2 \text{ 顆} \times 2.0 \text{ A} \times 0.6 = 2.4 \text{ (A)}$ 。總平均電流 = $1.5 + 0.3 + 2.4 = 4.2 \text{ (A)}$ 。可用電量 = $10 \text{ Ah} \times 0.8 = 8 \text{ (Ah)}$ 。理論續航 = $8 \div 4.2 \approx 1.90 \text{ (小時)}$ ，約 1 小時 54 分。

實測 1.8 小時，折損率 = $(1.90 - 1.80) \div 1.90 \approx 5.3\%$ ，屬於合理範圍。可能原因包括：降壓模組的轉換效率（通常 85~92%，會多耗約 10% 的電）、電池老化導致實際容量低於標稱、地毯或斜坡使馬達電流高於估算值、以及低溫下鋰電池內阻上升。若折損率超過 20%，則應懷疑有異常耗電（如驅動器待機電流過高）或電池已嚴重老化。

例題 3-2 控制週期與取樣頻率設計

題目：某差速輪機器人的輪子編碼器每圈輸出 1000 個脈波，輪子直徑 10 公分，最高速度 1.0 m/s。（a）求最高速度時的脈波頻率。（b）若下位機的控制週期為 1 ms，每個週期能計數幾個脈波？此解析度對應的速度誤差為何？（c）若改用 10 ms 控制週期，速度解析度如何變化？

解：（a）輪子周長 = $\pi \times 0.1 \approx 0.314 \text{ (m)}$ 。最高速度時每秒轉數 = $1.0 \div 0.314 \approx 3.18 \text{ (轉/秒)}$ 。脈波頻率 = $3.18 \times 1000 \approx 3183 \text{ (Hz)}$ 。

（b）1 ms 內的脈波數 = $3183 \times 0.001 \approx 3.18 \text{ (個)}$ 。由於計數必須是整數，實際只會讀到 3 個或 4 個，量化誤差達 ± 1 個脈波。一個脈波對應的距離 = $0.314 \div 1000 = 0.314 \text{ (mm)}$ ，

在 1 ms 內對應的速度解析度 = $0.000314 \div 0.001 = 0.314$ (m/s) ——這高達最高速度的 31% ，完全無法用於精確的速度控制。

(c) 改用 10 ms 週期，脈波數 ≈ 31.8 個，量化誤差仍為 ± 1 個脈波，但速度解析度變為 $0.000314 \div 0.01 = 0.0314$ (m/s)，僅為最高速度的 3.1%，改善了十倍。

結論與工程啟示：控制週期越短，控制反應越快，但速度解析度越差——這是一組必須權衡的矛盾。實務解法有三：一是提高編碼器線數（如改用 4000 線）；二是採用邊緣計時法（測量脈波之間的時間間隔而非計數）；三是採用串級控制，內層電流迴路用高頻、外層速度迴路用較低頻。這個「快與準」的權衡，會在第 16 章的數位控制中再次出現。

程式範例：系統健康監控

以下 Python 程式模擬一個機器人的系統監控模組：定期檢查各子系統的狀態，並在異常時分級告警。這是每一台實用機器人都應具備的基本功能——開頭故事中那台送藥機器人，若有電源異常監控，或許就不會因為一條磨破的電線而報廢。

```
import time
import random

# 各子系統的安全門檻
LIMITS = {
    "battery_v": (11.0, 12.6), # 電池電壓 (V)：低於 11.0 需警告
    "motor_a": (0.0, 4.0), # 馬達電流 (A)：超過 4.0 疑似堵轉
    "cpu_temp": (0.0, 75.0), # 主控溫度 (°C)
    "comm_delay": (0.0, 50.0), # 上下位機通訊延遲 (ms)
}

def read_sensors():
    """模擬讀取各子系統狀態（實機上改為真實感測器讀值）"""
    return {
        "battery_v": random.uniform(10.5, 12.6),
        "motor_a": random.uniform(0.5, 5.0),
        "cpu_temp": random.uniform(45.0, 80.0),
        "comm_delay": random.uniform(5.0, 60.0),
    }

def check_health(readings):
    """檢查狀態，回傳（等級，訊息）清單"""
    alerts = []
    for key, value in readings.items():
        lo, hi = LIMITS[key]
        if value < lo:
```

```

        alerts.append(("嚴重", f"{key} 過低：{value:.1f} ( 下限 {lo} ) "))
    elif value > hi:
        level = "嚴重" if value > hi * 1.1 else "警告"
        alerts.append((level, f"{key} 過高：{value:.1f} ( 上限 {hi} ) "))
    return alerts

def main_loop(cycles=5):
    for i in range(cycles):
        readings = read_sensors()
        alerts = check_health(readings)
        print(f"--- 第 {i+1} 次檢查 ---")
        if not alerts:
            print("所有子系統正常")
        for level, msg in alerts:
            print(f"[{level}] {msg}")
            if level == "嚴重":
                print(" → 動作：停止馬達、進入安全狀態、回報上位機")
        time.sleep(0.2)

main_loop()

```

請試著修改：加入一個「連續三次警告才升級為嚴重」的邏輯，避免單次雜訊造成誤停機。這個「去彈跳」的概念，在真實的安全監控系統中極為重要。

ROS 2 與模擬實作：畫出你的系統方塊圖

本章的實作不寫程式，而是設計。請依以下步驟，為你打算製作的機器人（或附錄中的任一）繪製完整的系統方塊圖與規格表：

1. 在紙上或繪圖軟體（建議 draw.io 或 Fritzing）畫出四個核心方塊：感測器、控制器、驅動器、致動器，並連成閉環。
2. 加入電源系統：電池、開關、保險絲、降壓模組、急停開關。以粗線標示功率流、細線標示訊號流，並標出單點共地位置。
3. 決定是否採用上下位機架構。若是，將控制器拆為兩個方塊並標註通訊介面（UART / CAN）與方向。
4. 在每條連線上標註介面規格與更新頻率，例如「I²C, 100 Hz」「PWM, 20 kHz」「UART 115200, 50 Hz」。
5. 將方塊圖轉為規格表：元件名稱、型號、工作電壓、額定電流、通訊介面、預估單價，並加總成本。

6. 檢查三件事：（一）所有元件的電壓是否相容？（二）電源的總電流是否足夠（含峰值）？（三）急停是否直接切斷功率、不經過軟體？

（進階）若已安裝 ROS 2，可執行 `rqt_graph` 觀察一個現成範例的節點圖——你會發現它就是本章方塊圖的軟體版本。第 29 章會教你如何為自己的系統建立這張圖。

國中小也看得懂：機器人的身體和我們一樣

機器人就像我們的身體，也有好幾個「系統」在一起工作。骨頭是機器人的鋁架，撐住身體不讓它垮掉。肌肉是馬達，負責出力讓輪子轉、手臂動。眼睛耳朵是感測器，看得到前面有沒有東西、知道自己轉了幾圈。大腦是控制板，負責想「該往哪走」。血管是電線，把電池的電送到每個地方。神經是訊號線，把眼睛看到的東西告訴大腦，再把大腦的命令傳給肌肉。

最重要的一件事：這些系統要「合作」才有用。如果眼睛很好但大腦沒接到線，機器人還是會撞牆；如果大腦很聰明但電池沒電，它就一動也不動。所以做機器人時，不能只顧一個部分做得很棒，要讓每個部分都健康、而且好好連在一起。這就叫做「系統」。下次你的小車不會動，記得照順序檢查：有電嗎？線接好了嗎？眼睛有看到嗎？大腦有想嗎？肌肉有動嗎？

叫獸提醒與常見錯誤

- 誤區一：先買零件再想架構。應該先畫方塊圖、列規格表，確認電壓與介面相容後再採購——順序反了會浪費大量金錢與時間。
- 誤區二：只看馬達額定電流選驅動器。堵轉電流可能是額定的五到十倍，驅動器必須按峰值選，並加上過流保護。
- 誤區三：訊號線與功率線網在一起走。馬達的電流突波會透過電磁耦合干擾訊號線，導致感測器讀值亂跳。兩者應分開走線或加隔離。
- 誤區四：把急停做成軟體功能。急停必須是硬體迴路，直接切斷功率電源。任何經過軟體或通訊協定的「急停」，在系統當機時都會失效。
- 誤區五：把即時控制放在跑作業系統的電腦上。Linux 的排程延遲不可預測，毫秒級控制迴路請交給 MCU 或即時作業系統。
- 誤區六：忽略通訊超時保護。上位機當機時，下位機若沒有超時停止機制，機器人會維持最後指令一路衝出去。

- 誤區七：不做分層測試。請依序驗證：電源正常 → 感測器讀值正確 → 單軸控制收斂 → 多軸協調 → 全系統。跳級測試會讓除錯時間暴增。
- 誤區八：布線不做保護。開頭故事的教訓——線束經過機構邊角處必須加套管、固定與圓角處理，這是最便宜的可靠度投資。

課堂討論

1. 開頭故事中的送藥機器人死於一條磨破的電線。若你是設計者，會用哪三項具體措施防止此事？請分別從機構、電子與軟體監控三方面提出。
2. 為什麼掃地機器人可以用集中式架構，而六軸工業手臂幾乎都用分散式？請從軸數、距離與可靠度三方面分析。
3. 端到端的 AI 控制（影像直接輸出馬達指令）省略了明確的規劃層。你認為在什麼場合可以接受？什麼場合絕對不行？
4. 如果你只能為機器人增加一項「監控」功能來提升可靠度，你會選什麼？請說明理由與實作方式。
5. 比較人體與機器人的系統架構：人類的「下位機」在哪裡（提示：脊髓反射）？這個對照對機器人設計有什麼啟發？

章末摘要

- 機器人由七大子系統組成：機械本體、控制器、感測器、致動器與驅動器、電源、通訊、軟體；系統的價值不在零件加總，而在關係的總和。
- 運算單元分三層：MCU（即時性佳、運算低）、SBC（可跑 OS 與 ROS 2）、邊緣 AI 平台（可跑深度學習）；實用機器人多採組合方案。
- 感測器訊號鏈路有五環：換能、訊號調理、ADC、取樣、數值處理；任一環出錯，程式讀到的都是垃圾。
- 驅動器必須按峰值電流（堵轉可達額定五至十倍）選型，並具備過流與散熱設計。
- 電源是機器人故障排行榜第一名；架構原則為功率與訊號分離、單點共地、加裝穩壓與保護電路；續航估算須保留 20 ~ 30% 餘裕。
- 通訊介面依頻寬、即時性與抗雜訊選擇；安全相關與即時控制絕不依賴無線，急停必須是硬

體迴路。

- 軟體分三層：韌體驅動、控制與狀態估測、規劃與決策；ROS 2 是模組間的神經系統，有限狀態機是行為的骨架。
- 上下位機分工的理由是時間尺度差異；介面設計三原則為抽象層級一致、超時保護、狀態雙向回報。
- 即時不等於快，而是準時；系統存在時間尺度金字塔，由電流迴路（微秒）到 AI 決策（秒），跨越六個數量級。
- 感知—規劃—控制三層架構是第 1 章循環的工程實作；集中式適合小型系統，分散式適合多軸長距離；系統方塊圖是工程師的共同語言。

觀念題與計算題

觀念題

1. 列舉機器人的七份子系統，並各用一句話說明其功能。
2. 比較 MCU、SBC 與邊緣 AI 平台的差異，並各舉一個適合的任務。
3. 畫出感測器的五環訊號鏈路，並在每一環標出一個常見的失效模式。
4. 為什麼說「內部感測是控制的基礎，外部感測是自主的基礎」？請以工業手臂與移動機器人各舉一例說明。
5. 說明驅動器在系統中的角色，以及選型時必須注意的三個規格。
6. 電源架構的兩大原則是什麼？為什麼馬達啟動會導致微控制器重開機？
7. 比較 I²C、CAN Bus 與 EtherCAT 三種介面的特性與適用場合。
8. 解釋「即時不等於快」的意義，並區分硬即時與軟即時，各舉兩個機器人上的例子。
9. 說明上位機與下位機分工的理由，以及介面設計的三個原則。
10. 比較集中式與分散式架構的優缺點，並說明選擇的判準。
11. 說明感知—規劃—控制三層架構的資料流，以及「反射弧」為何要繞過規劃層。
12. 畫系統方塊圖的五個步驟為何？為什麼急停必須畫在功率路徑上而非訊號路徑上？

計算題

1. 某巡檢機器人使用 24 V、20 Ah 電池。耗電為：主控 2.0 A、光達 0.5 A、四顆馬達各 1.5 A

(工作週期 70%)、雲台 0.3 A (工作週期 20%)。電池可用比例取 80%，降壓模組效率 90%。求理論續航時間 (提示：訊號側耗電需除以轉換效率)。

2. 某機械手臂關節使用 2500 線編碼器 (每圈 2500 脈波)，減速比 50:1，馬達最高轉速 3000 rpm。(a) 求關節端的最高轉速 (度 / 秒)。(b) 求馬達端最高脈波頻率。(c) 若控制週期為 0.5 ms，計算關節角度的量化解析度 (度)。(d) 若要求角度解析度優於 0.01 度，編碼器線數至少需多少？
3. 某移動機器人採 CAN Bus 連接 6 個節點，每個節點每 10 ms 送出一則 8 位元組的訊息 (連同協定開銷共約 130 位元)。(a) 求匯流排的平均使用率 (以 500 kbps 計)。(b) 若要新增 4 個相同節點，使用率變為多少？是否仍在建議的 40% 安全上限內？

動手做與專題延伸

本章實作：機器人解剖與系統還原

找一台報廢或閒置的電子設備 (掃地機器人、噴墨印表機、光碟機、玩具遙控車皆可)，在安全前提下拆解，並完成下表。拆解前請先斷電並取出電池；若含高壓元件 (如印表機的電源模組、CRT) 請勿拆解。

表 3-6 解剖記錄表 (範例列供參考)

子系統	找到的零件	規格 / 型號	在系統中的角色	設計巧思或缺陷
機械本體	塑膠底盤與金屬支架	ABS 塑膠、厚約 2 mm	支撐與保護內部元件	以卡榫取代螺絲，組裝快但難維修
致動器	直流馬達加塑膠齒輪箱	6 V、減速比約 1:48	驅動輪子前進與轉向	齒輪為塑膠，成本低但易磨損
(自行填寫其餘五個子系統)				

完成後，請依 3.12 節的五步驟，把你拆到的零件還原成一張系統方塊圖，並回答：這台設備採用集中式還是分散式架構？它有沒有上下位機的分工？

專題延伸

- 為附錄 F 的循跡小車繪製完整系統方塊圖與規格表，並計算其電池續航時間。實測後比較理論值與實測值的差異，分析原因。

- 訪查一家自動化設備廠商或參觀工廠，記錄一台工業機器人的系統架構：主控是什麼？用什麼匯流排？急停如何設計？安全圍籬與感測如何配置？寫成一千字的參訪報告。
- 進階：為一台假想的「校園導覽機器人」做完整的系統設計——列出需求（續航四小時、能上下斜坡、能辨識人臉、能語音互動），據此選定各子系統的元件、繪製方塊圖、估算成本與續航，並標出三個最大的技術風險與因應方案。

參考文獻與延伸閱讀

- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (第一部分：機器人系統與元件) 。
- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer.
- Niku, S. B., Introduction to Robotics: Analysis, Control, Applications, Wiley (機器人子系統與致動器章節) 。
- Horowitz, P. and Hill, W., The Art of Electronics, 3rd ed., Cambridge University Press (電源、訊號調理與雜訊處理) 。
- Kopetz, H., Real-Time Systems: Design Principles for Distributed Embedded Applications, Springer (即時系統理論) 。
- ROS 2 官方文件 (docs.ros.org)：節點、話題與系統架構設計指南。
- CAN in Automation (CiA) 與 EtherCAT Technology Group 官方技術文件。
- IEC 60204-1 與 ISO 13850：機械電氣安全與緊急停止裝置相關標準。
- 各開發板官方文件：Raspberry Pi、NVIDIA Jetson、STM32、ESP32 資料手冊與應用筆記。

第 4 章

機構、連桿與自由度

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】三聯圖：左為一隻手掌平貼白板、手肘卻高高抬起的手臂 (冗餘自由度) ；中為一台六軸工業手臂的第三節卡在治具上 (自由度不足的困境) ；右為一台 *Delta* 機器人三條支鏈張開如飛翔的三角形。三張圖共同提問：幾個關節，才叫剛剛好？

叫獸開講 「為什麼人手看起來很靈活，機器手臂卻常常卡住？」

那天下午，叫獸在課堂上做了一個小實驗。他請一位學生上台，把右手手掌平貼在白板上，五指張開、手掌不能離開白板，然後說：「請你把手肘抬高。」

學生照做了——手掌沒動，手肘卻緩緩往上抬起，整條手臂像一扇被推開的門那樣繞著某條看不見的軸旋轉。台下一片驚呼：手的位置完全沒變，手臂的形狀卻變了。

「這個動作，」叫獸說，「是機器人學裡最重要的現象之一，叫做冗餘。你的手臂從肩膀到手掌，總共有七個自由度，但『把手掌固定在空間中的某個位置與姿態』只需要六個。多出來的那一個，就讓你可以在保持手掌不動的情況下，讓手肘上上下下。這一度自由，讓你可以在鍵盤前繞開桌角、能伸手到冰箱深處拿飲料而不撞到隔板。」

接著他打開投影片，播放一段工業現場的影片：一台六軸機械手臂正要把一支零件插進狹縫，但手臂的第三節撞上了旁邊的治具。操作員在示教器上調了半天，最後只能把整個治具挪開重來。

「這台手臂只有六個自由度，」叫獸說，「它能讓末端到達你要的位置與姿態，但『用什麼姿勢到達』是被綁死的。它沒有多餘的那一度自由，所以撞到就是撞到，沒有第二種走法。」

有學生問：「那為什麼工業手臂不做七軸就好了？」叫獸笑了：「好問題。多一軸就多一顆馬達、多一組減速機、多一段連桿——成本上升、剛性下降、控制變複雜、而且逆運動學會有無限多組解，你得再花力氣去挑一組。工程從來不是『越多越好』，而是『剛剛好』。」

他在白板上畫了三個東西：一台會爬樓梯的四足機器狗、一台在無塵室高速取放晶片的 SCARA、一台飛行模擬器底下那個六支油壓缸撐起的平台。「這三台機器，自由度從十二、到四、到六，形態完全不同——但它們共用同一套數學。今天這堂課，我們要學會兩件事：第一，怎麼數出一台機器有幾個自由度；第二，這個數字為什麼決定了它能做什麼、不能做什麼。」

下課前他留了一個問題：「回家後把手掌貼在桌上，重複剛才那個動作。然後想想看——如果你的手臂只有六個自由度，你的生活會有哪裡不方便？」

本章為什麼重要

第3章建立了系統觀，從本章開始我們逐一拆解子系統，第一站是機構——機器人的骨骼。機構決定了一台機器人「能做什麼」的物理上限：工作範圍多大、能不能繞過障礙、能不能舉起重物、精度能到多細。這些都在畫第一張設計圖時就被決定了，後面再強的控制演算法與 AI 都

無法突破。

本章的核心概念是自由度 (DoF)。它是機器人學中最基礎、也最常被誤用的名詞——很多人以為「自由度等於馬達數」，但這只在特定條件下成立。學會正確計算自由度，你才能判斷一個機構設計是否可行、是否過度約束、是否有冗餘。這個概念會在第 9 章 (正向運動學)、第 10 章 (逆向運動學的多解與無解)、第 12 章 (奇異點與冗餘) 中反覆出現，是整個第二篇的地基。

學習目標

- 能定義機構、連桿、關節與運動鏈，並辨識實際機器人中的對應部位。
- 能列舉常見關節類型及其自由度，並說明旋轉關節與移動關節的差異。
- 能區分開放鏈與閉合鏈，並說明兩者在剛性、工作空間與分析難度上的差異。
- 能說明自由度的物理意義，並解釋為何自由度不必然等於馬達數。
- 能運用 Grübler 公式計算平面與空間機構的自由度。
- 能比較串聯式與並聯式機器人的優缺點與適用場合。
- 能說明 SCARA、Delta 與 Stewart 平台的構型特徵與典型應用。
- 能定義冗餘自由度，並舉例說明其優點與代價。
- 能區分工作空間、可達空間與靈巧空間，並描述常見構型的工作空間形狀。
- 能依任務需求，為一個實際應用選定合適的機器人構型並說明理由。

先備知識

第 3 章的系統架構觀念。本章需要基本的空間想像能力與簡單的加減乘法，不需要矩陣或微積分。若能先閱讀附錄 D.1 對三維座標的說明會更順暢，但非必要——本章刻意把所有概念用圖與實物說明，數學留待第 8、9 章展開。

核心名詞中英對照

表 4-1 本章核心名詞

中文	英文	說明
機構	Mechanism	由連桿與關節組成、能傳遞運動與力的系統

中文	英文	說明
連桿	Link	機構中的剛體構件
關節	Joint	連接兩連桿並限制其相對運動的元件
運動鏈	Kinematic Chain	連桿與關節依序連接形成的鏈狀結構
自由度	Degree of Freedom, DoF	描述系統位形所需的獨立參數數目
旋轉關節	Revolute Joint	允許繞單一軸旋轉的關節，1 自由度
移動關節	Prismatic Joint	允許沿單一軸平移的關節，1 自由度
球關節	Spherical Joint	允許三軸旋轉的關節，3 自由度
開放鏈	Open Chain / Serial Chain	一端固定、一端自由的鏈狀結構
閉合鏈	Closed Chain	形成迴路的機構結構
串聯式機器人	Serial Robot	關節依序串接的機器人，如六軸手臂
並聯式機器人	Parallel Robot	多條支鏈共同支撐動平台的機器人
冗餘	Redundancy	自由度多於任務所需的情形
工作空間	Workspace	末端執行器可到達的空間範圍
靈巧工作空間	Dexterous Workspace	末端可以任意姿態到達的空間範圍
過度約束	Overconstrained	約束數超過理論所需，卻仍能運動的機構

4.1 機構與機器的基本概念

4.1.1 從機器到機構

在機械工程的傳統定義中，機器 (machine) 是能傳遞或轉換能量、完成有用功的裝置；機構 (mechanism) 則是機器中負責傳遞運動與力的部分，關注的是「怎麼動」而非「用多少能量」。一台機械手臂的馬達與減速機屬於驅動系統，而連桿與關節組成的那副骨架，就是機構。

機構學研究三件事：位置分析 (給定關節位置，末端在哪裡)、速度分析 (關節轉多快，末端跑多快)、與力分析 (末端受力多大，關節要出多少力矩)。這三件事在機器人學中分別對應第 9 章的正向運動學、第 11 章的雅可比矩陣，以及第 13 章的靜力學——所以本章其實是整個第二篇與第三篇的入口。

4.1.2 剛體假設：一個有用的謊言

機構學的一切分析都建立在「剛體假設」上：假設每根連桿都是完全不變形的剛體。這當然

不是真的——鋁桿會彎、齒輪會有背隙、軸承會有間隙。但這個假設極其有用，因為它讓運動分析變成純粹的幾何問題，可以用矩陣精確描述。

什麼時候這個謊言會失效？當手臂很長很細、當運動速度很高、當負載很重時，彈性變形就不能忽略——這就是為什麼工業手臂的連桿又粗又重（提高剛性），也是為什麼高速運動時末端會殘留振動（第 15 章的軌跡平滑度、第 27 章的結構剛性都與此有關）。初學階段先接受這個假設，但要記得它的邊界。

◎ 三個名詞的區分

機構 (mechanism)：連桿與關節的組合，管運動的傳遞。機器 (machine)：機構加上動力源與控制，能做有用功。機器人 (robot)：機器加上感知與自主決策（第 1 章的三判準）。一台自行車是機構加機器，但不是機器人；一台六軸手臂三者皆是。

4.2 連桿、關節與運動鏈

4.2.1 連桿：機構的骨頭

連桿 (link) 是機構中的剛體構件。在機器人手臂上，它就是關節與關節之間的那一段結構。連桿的編號有一個慣例：固定不動的基座編為連桿 0，往末端依序編為連桿 1、2、3.....這個編號方式在第 9 章的 DH 參數法中會嚴格使用，現在先建立習慣。

連桿的設計要同時滿足三個要求：足夠的剛性（受力時變形要小）、盡可能輕（減少慣性與能耗）、以及提供精確的關節安裝面（安裝偏差會沿著鏈條累積放大，如第 3 章所述）。工程上常用的解法是中空結構——把材料集中在外圍，因為抗彎剛性主要來自截面的邊緣材料，中心的材料貢獻很小卻很重。這就是為什麼機器人連桿、腳踏車車架與飛機大樑都是中空的。

4.2.2 關節：決定運動的自由

關節 (joint) 連接兩根連桿，並限制它們之間的相對運動。關節的種類決定了「留下」多少自由度。兩個在空間中完全自由的剛體，彼此有六個相對自由度（三個平移、三個旋轉）；加上一個關節之後，其中一部分被約束掉，剩下的就是該關節的自由度。

最常見的兩種關節，也是機器人手臂上幾乎唯一使用的兩種：旋轉關節 (revolute joint，符號 R) 允許繞單一軸相對旋轉，1 個自由度，實作上就是一組軸承加馬達；移動關節 (prismatic joint，符號 P) 允許沿單一軸相對平移，1 個自由度，實作上是線性滑軌加螺桿或氣

壓缸。

表 4-2 常見關節類型與自由度

關節類型	英文與符號	自由度	允許的運動	機器人上的實例
旋轉關節	Revolute (R)	1	繞一軸旋轉	六軸手臂的每一軸、伺服馬達關節
移動關節	Prismatic (P)	1	沿一軸平移	SCARA 的 Z 軸、3D 印表機軸、氣壓夾爪
螺旋關節	Helical / Screw (H)	1	旋轉與平移耦合	滾珠螺桿傳動
圓柱關節	Cylindrical (C)	2	繞軸旋轉 + 沿軸平移	旋轉伸縮式取放機構
萬向關節	Universal (U)	2	兩軸旋轉	傳動軸、Delta 機器人支鏈
球關節	Spherical (S)	3	三軸旋轉	Stewart 平台支腳、人形機器人髖關節
平面關節	Planar (E)	3	平面內兩平移 + 一旋轉	平面滑動接觸

4.2.3 運動鏈與運動樹

連桿與關節依序連接就形成運動鏈 (kinematic chain)。若每根連桿最多連接兩個關節、且不形成迴路，就是簡單的鏈；若某根連桿連接三個以上的關節，運動鏈就會分岔，形成運動樹 (kinematic tree) ——人形機器人就是典型的樹狀結構，軀幹連桿同時分岔出兩條手臂、兩條腿與一個頭。

樹狀結構的分析並不比鏈狀複雜多少：每一條分支都可以獨立套用鏈狀的公式，只要記得它們共享同一個基座即可。第 9 章的正向運動學會用「從基座往末端連乘變換矩陣」的方式處理，樹狀結構只是有多條路徑要各算一次。

4.3 旋轉關節與移動關節

4.3.1 兩者的工程特性比較

雖然旋轉關節與移動關節在數學上都只是「1 個自由度」，但工程特性差異極大，選用時必須權衡。

旋轉關節的優點是：行程無限（可以連續轉動）、結構緊湊、密封容易、成本低（軸承便宜）。缺點是：末端位置與關節角度是非線性關係（三角函數，第 9 章）、力矩會隨姿態變化、且離軸越遠的位置誤差被放大越多。

移動關節的優點是：運動與位置是線性關係（好算、好控）、剛性高、在整個行程上出力一致。缺點是：行程有限（滑軌多長就只能走多遠）、佔用空間大（伸出去時機構會變長）、防塵防水困難（滑軌暴露在外）、成本較高。

表 4-3 旋轉關節與移動關節的比較

面向	旋轉關節 R	移動關節 P
行程	可無限旋轉	受滑軌長度限制
位置關係	非線性（三角函數）	線性
工作空間形狀	球狀或環狀	方正、直角座標式
空間佔用	緊湊	伸展時佔空間大
剛性	受懸臂長度影響	通常較高
防護	容易密封	滑軌暴露、需防塵罩
成本	較低	較高
典型應用	多關節手臂、人形、四足	龍門機構、3D 印表機、SCARA Z 軸

4.3.2 為什麼工業手臂幾乎都用旋轉關節

觀察市面上的六軸工業手臂，會發現六個軸全是旋轉關節。原因有三：第一，工作空間效率——用同樣的機構體積，旋轉關節能掃出大得多的工作範圍；一台佔地半平方公尺的六軸手臂，工作半徑可達一點五公尺，這是移動關節做不到的。第二，成本與可靠度——軸承與減速機的技術成熟、壽命長、可完全密封，適合工廠的粉塵油污環境。第三，仿生——人類的手臂全部由旋轉關節組成，而工廠的作業流程原本就是為人類設計的。

反過來說，需要大範圍平面運動且要求高精度時（如 PCB 檢測、雷射切割、3D 列印），龍門式的直角座標機構（全用移動關節）反而更合適：定位精度高、程式直觀（XYZ 座標直接對應）、剛性好。這說明構型選擇沒有絕對優劣，只有適不適合。

4.4 開放鏈與閉合鏈

4.4.1 開放鏈：一端固定、一端自由

開放鏈 (open chain , 又稱串聯鏈) 是最常見的機器人結構：基座固定，連桿一節接一節往外延伸，最末端是自由的末端執行器。六軸工業手臂、人的手臂、機器狗的每一條腿 (在騰空時) ，都是開放鏈。

開放鏈的優點是工作空間大、結構簡單、分析容易——正向運動學只是把變換矩陣依序相乘 (第 9 章) ，不需解聯立方程。缺點是剛性差 (誤差與變形逐節累積，就像一根長長的懸臂) 、每個關節都要承受下游所有連桿與負載的重量 (所以靠近基座的馬達必須很大顆) 、且高速運動時慣性大。

4.4.2 閉合鏈：形成迴路的結構

閉合鏈 (closed chain) 中的連桿形成了迴路：從某根連桿出發，沿著關節走一圈可以回到原點。四連桿機構、Delta 機器人的三條支鏈、Stewart 平台的六支腿，以及機器狗站立時 (雙腳同時著地、透過地面形成迴路) ，都是閉合鏈。

閉合鏈的優點正是開放鏈的缺點：剛性高 (多條支鏈共同承力，像三腳架比獨腳架穩) 、精度好 (誤差不累積而是被平均) 、馬達可以固定在基座 (大幅降低運動部分的慣量，因此能高速) 。缺點則是工作空間小、關節活動範圍受限、且運動學分析複雜——閉合鏈的正向運動學往往需要解非線性聯立方程，反而是逆運動學比較容易 (與開放鏈恰好相反，這是一個有趣的對偶現象) 。

圖 4-1 開放鏈與閉合鏈的對照

建議配圖：左側為六軸串聯手臂 (開放鏈) ，以虛線標出從基座到末端的單一路徑，旁註「誤差累積、工作空間大、剛性低」；右側為 Delta 機器人 (閉合鏈) ，三條支鏈由基座延伸並共同連接到動平台，形成迴路，旁註「誤差平均、剛性高、馬達在基座、工作空間小」。

4.4.3 混合結構

實務上許多機器人是混合的。例如常見的六軸手臂，其第二、三軸之間常設計成平行四連桿機構 (closed loop) 來傳遞第三軸的動力，使第三軸的馬達可以往後移到基座附近，降低手臂前端的重量——這是在串聯手臂中局部嵌入閉合鏈的經典設計。人形機器人的腿部也常採此策略，把膝關節的驅動器移到大腿根部。

判斷一個機構是開放還是閉合，最簡單的方法是問：從任一根連桿出發，沿著關節能不能走回原點？能，就有閉合迴路。這個判斷會直接影響下一節自由度公式的用法。

4.5 自由度的物理意義

4.5.1 定義：描述位形所需的獨立參數數目

自由度 (Degree of Freedom, DoF) 是指完整描述一個系統的位形 (configuration , 即所有零件的位置與姿態) 所需要的獨立參數個數。這個定義聽起來抽象，但用具體例子立刻清楚。

一個點在平面上有 2 個自由度 (x, y) ; 在空間中有 3 個 (x, y, z) 。一個剛體在平面上有 3 個自由度 (x, y 加上一個旋轉角 θ) ; 在空間中有 6 個 (三個平移 x, y, z 加上三個旋轉，即滾轉、俯仰、偏航) 。這個「空間剛體六自由度」是機器人學最重要的數字之一——它解釋了為什麼工業手臂通常是六軸：六個自由度恰好足以把末端執行器擺到空間中任意位置與任意姿態。

平面剛體：DoF = 3	空間剛體：DoF = 6
--------------	--------------

這兩個數字是所有自由度計算的基礎，請務必記住。

4.5.2 自由度不等於馬達數

初學者最常見的誤解是「自由度等於馬達數量」。這在多數串聯手臂上恰巧成立，但並非通則，有三種例外。

第一種：驅動不足 (underactuated) 。馬達數少於自由度。四足機器狗在騰空瞬間，身體的六個自由度中沒有任何一個被直接驅動——它只能靠腿部的甩動與落地的接觸力間接控制姿態。人類走路也是如此：從地面到重心的這條「虛擬鏈」沒有馬達，所以走路本質上是一連串「受控的跌倒」(第 28 章) 。

第二種：冗餘驅動 (overactuated) 。馬達數多於自由度。有些並聯機器人與多指機械手會刻意讓馬達數超過自由度，藉此消除背隙、提高剛性、或分散負載——代價是各馬達之間必須協調，否則會互相「打架」而燒毀。

第三種：被動關節。有些關節完全沒有馬達，只是提供運動自由，例如 Delta 機器人支鏈上的球關節、或是機器人腳踝的被動彈簧。它們貢獻自由度，但不貢獻驅動。

◎ 三個容易混淆的自由度

機構自由度：機構整體能獨立運動的參數數目 (本節主題) 。關節自由度：單一關節允許的相對運動數 (表 4-2) 。任務自由度：完成任務所需的自由度數，例如「畫平面圖形」只需 3，「任意姿態抓取」需要 6。三者的比較決定了機器人是否足夠、是否冗餘——這正是 4.10 節的主題。

4.6 平面與空間機構的自由度：Grübler 公式

4.6.1 公式的直覺推導

計算自由度的方法很直觀：先假設所有連桿都自由漂浮，數出總自由度；再把關節加上去，每個關節扣掉它所約束的自由度；剩下的就是機構的自由度。

設一個機構有 N 根連桿（含固定基座）、 J 個關節，每個關節 i 允許 f_i 個自由度。在空間中，每根剛體有 6 個自由度，但基座是固定的，所以自由漂浮的連桿數是 $N - 1$ ，總自由度為 $6(N - 1)$ 。每個關節在空間中會約束掉 $(6 - f_i)$ 個自由度。因此：

$$\text{空間機構：DoF} = 6(N - 1) - \sum_i (6 - f_i) = 6(N - 1 - J) + \sum_i f_i$$

此即 Grübler-Kutzbach 公式的空間形式。 N 為連桿數（含基座）， J 為關節數， f_i 為第 i 個關節的自由度。

平面機構的推導完全相同，只是把 6 換成 3：

$$\text{平面機構：DoF} = 3(N - 1 - J) + \sum_i f_i$$

平面情形下，每根剛體 3 個自由度，每個關節約束 $(3 - f_i)$ 個。

4.6.2 串聯機構的特例

對於開放鏈（串聯機構），公式會退化成一個極簡的結果。開放鏈中，連桿數與關節數的關係恆為 $N = J + 1$ （每加一個關節就多一根連桿）。代入公式， $6(N - 1 - J)$ 這一項變成 $6(J + 1 - 1 - J) = 0$ ，因此：

$$\text{開放鏈：DoF} = \sum_i f_i \quad (\text{所有關節自由度的總和})$$

這就是為什麼六軸手臂（六個 1 自由度的旋轉關節）的自由度恰好是 6。

這解釋了初學者的印象為何常常正確——因為他們接觸的多半是串聯手臂。但一旦遇到閉合鏈，就必須回到完整的 Grübler 公式。

4.6.3 使用公式的三個注意事項

第一，基座要算進 N 。忘記把固定基座算進連桿數，是最常見的計算錯誤，會讓答案剛好差 6（或平面的 3）。

第二，公式算的是「理論自由度」，不保證機構真的能動到那麼多。實際機構可能因為關節行程限制、或干涉碰撞，而無法走完所有位形。

第三，也是最重要的——Grübler 公式對某些特殊機構會算錯。這類機構稱為過度約束機構

(overconstrained mechanism)，公式算出來的自由度是 0 甚至負數，但它們實際上能動。原因是公式假設所有約束彼此獨立，而這些機構因為特殊的幾何排列（例如所有軸互相平行或共點），使得部分約束重複了。經典例子是 Bennett 連桿與 Sarrus 機構。這提醒我們：公式是工具不是真理，最後仍要靠幾何直覺與實體驗證。

◎ 計算自由度的四步驟

一、數連桿數 N （記得含基座）。二、數關節數 J ，並列出每個關節的自由度 f_i 。三、判斷是平面（用 3）還是空間（用 6）機構。四、代入公式計算，並用物理直覺檢驗答案是否合理（例如：一台明顯能自由移動的機器人算出 0 自由度，一定是哪裡數錯了或遇到過度約束）。

4.7 串聯式與並聯式機器人

4.7.1 串聯式：大範圍、高彈性

串聯式機器人就是開放鏈的機器人，關節一個接一個串起來。它是工業上最主流的形態，六軸手臂、SCARA、協作機器人皆屬此類。

優點：工作空間相對於機構體積而言非常大；運動學直觀（正向運動學只需連乘）；每個關節可獨立設計與更換；容易加長臂展。缺點：剛性隨臂展下降；靠近基座的關節要承受整條手臂的重量與慣性，馬達與減速機必須很大；誤差逐節累積，末端精度受限於最差的那一節；高速時因慣性大而振動明顯。

4.7.2 並聯式：高剛性、高速度

並聯式機器人以多條支鏈同時連接基座與動平台，形成閉合鏈。經典代表是 Delta（三支鏈，取放專用）與 Stewart 平台（六支鏈，飛行模擬器與精密定位）。

優點：剛性高（力量由多條支鏈分擔）；精度好（誤差被平均而非累積）；馬達可全部固定在基座上，運動部分極輕，因此加速度可達數十個 g ，是高速取放的王者；承載自重比極佳。缺點：工作空間小（多條支鏈互相牽制）；姿態範圍受限；容易在工作空間內遇到奇異點；正向運動學需解非線性方程，計算較複雜。

表 4-4 串聯式與並聯式機器人的比較

面向	串聯式（開放鏈）	並聯式（閉合鏈）
工作空間	大	小

面向	串聯式（開放鏈）	並聯式（閉合鏈）
剛性	較低（懸臂效應）	高
精度	誤差累積	誤差平均
速度 / 加速度	中等（運動慣量大）	極高（馬達在基座）
承載自重比	較低	高
正向運動學	簡單（連乘）	複雜（解非線性方程）
逆向運動學	較複雜（多解）	相對簡單
典型代表	六軸手臂、SCARA、協作機器人	Delta、Stewart 平台
典型應用	焊接、組裝、搬運、上下料	高速取放、精密定位、模擬平台

◎ 一個記憶法

串聯像人的手臂——伸得遠、動作靈活，但舉重物時會抖。並聯像三腳架或起重機的支撐——範圍小，但穩如泰山。當任務需要「伸進去」時用串聯；需要「又快又穩」時用並聯。

4.8 SCARA、Delta 與 Stewart 平台

4.8.1 SCARA：電子業的速度之王

SCARA 全名為 Selective Compliance Assembly Robot Arm（選擇性順應裝配機器手臂），由日本學者牧野洋於 1978 年提出。它的構型是 RRPR 或 RRP：兩個平行的垂直軸旋轉關節提供水平面的運動，再加上一個垂直的移動關節與一個末端旋轉軸，總共四個自由度。

名稱中的「選擇性順應」是理解它的關鍵：這個機構在水平面方向較為順應（有一點柔性），但在垂直方向極為剛硬。這正是垂直插裝作業所需要的特性——插銷入孔時，水平方向的微小柔性可以自動修正對位誤差，而垂直方向的高剛性則保證能壓得下去。這是機構設計「順應性剛好用在對的方向」的經典案例，與第 20 章的柔順控制遙相呼應。

SCARA 的四個自由度恰好對應平面裝配任務所需：平面位置（ x, y ）、高度（ z ）與繞垂直軸的旋轉（ θ ）。它不能傾斜末端——但電子組裝幾乎所有零件都是垂直插入的，因此不需要。少兩個自由度換來的是更高的速度、更低的成本與更好的剛性，這是「剛剛好就好」的最佳範例。

4.8.2 Delta：飛翔的三角形

Delta 機器人由瑞士學者 Reymond Clavel 於 1980 年代發明，原始構想來自巧克力包裝生

產線的高速取放需求。它的結構是：三個安裝在基座上的旋轉馬達，各自驅動一根主動臂；每根主動臂再透過平行四邊形連桿（由球關節或萬向關節連接）連到中央的動平台。

這個平行四邊形設計有一個關鍵作用：它保證動平台永遠保持與基座平行，不會傾斜。因此 Delta 提供三個平移自由度（ x, y, z ），姿態被固定。若需要旋轉，通常在中央加一根伸縮傳動軸提供第四軸。

Delta 的性能極為驚人：因為三顆馬達全部固定在基座、運動部分只有輕巧的碳纖維連桿，加速度可達 10 g 以上，每分鐘可完成兩百次以上的取放。今天你在超商買到的每一盒包裝食品、每一片包裝好的藥錠，很可能都經過 Delta 機器人之手。

4.8.3 Stewart 平台：六支腳撐起的世界

Stewart 平台（又稱 Gough-Stewart 平台或 Hexapod）由六根可伸縮的支腿連接基座與動平台，每根支腿兩端各有一個球關節或萬向關節。六根腿的長度變化組合，恰好提供動平台完整的六個自由度——三平移加三旋轉。

它的特點是剛性極高、承載能力極強、且定位精度可達微米級。典型應用包括：飛行模擬器（撐起整個駕駛艙並模擬六自由度的運動）、精密機械加工的定位平台、望遠鏡的鏡面調整、以及部分手術機器人。缺點是工作空間非常小（尤其是旋轉範圍），以及正向運動學必須用數值方法求解。

圖 4-2 三種經典構型的對照

建議配圖：三格並排。左為 SCARA（俯視加側視，標出兩個垂直旋轉軸、Z 軸移動與末端旋轉，共 4 DoF）；中為 Delta（三支鏈由基座向下延伸至動平台，標註平行四邊形連桿與 3 DoF 平移）；右為 Stewart 平台（六支可伸縮腿呈交錯排列，標註 6 DoF）。下方各標註自由度、速度、剛性與典型應用。

表 4-5 三種經典構型規格對照

構型	自由度	關節配置	強項	限制	代表應用
SCARA	4	RRPR	水平高速、垂直剛硬、成本低	末端無法傾斜	電子組裝、鎖螺絲、點膠
Delta	3（可加至 4）	3 條 R + 平行四邊形支鏈	加速度極高、輕量	工作空間小、姿態固定	食品藥品高速取放、分揀
Stewart 平台	6	6 條 UPS 支腿	剛性與精度極高、承載大	工作空間與旋轉範圍極小	飛行模擬器、精密定位、手術

4.9 冗餘自由度

4.9.1 定義與人類手臂的例子

當機器人的自由度多於完成任務所需的自由度時，就稱為冗餘（redundancy）。開頭故事中人類手臂的七個自由度，對「把手掌放到指定位置與姿態」這個六自由度任務而言，就多出一個——這一個多餘的自由度，讓手肘可以在手掌固定的情況下擺動。

在數學上，冗餘意味著逆運動學有無限多組解：達成同一個末端位姿，關節角度的組合形成一個連續的集合（稱為自運動流形，self-motion manifold）。第 12 章會說明如何在這個無限多的解中，用零空間運動去選出最好的一組。

4.9.2 冗餘的四大好處

第一，避障：在末端保持軌跡不變的前提下，改變手肘或關節的姿勢以繞開障礙物——這正是開頭故事中六軸手臂做不到、而七軸手臂做得到的事。

第二，避開奇異點：奇異點是機器人失去某個方向運動能力的位形（第 12 章）。有冗餘時，可以用多出來的自由度繞開這些位形。

第三，避開關節極限：當某個關節接近行程末端時，可以重新分配各關節的角度，讓它退回安全範圍。

第四，最佳化次要目標：在完成主要任務的同時，還可以順便最小化能耗、最大化操作靈巧度、或維持一個對人類而言看起來自然的姿勢——這對協作機器人與人形機器人特別重要。

4.9.3 冗餘的代價

冗餘不是免費的。每多一個自由度，就多一顆馬達、一組減速機、一段連桿與一組線路：成本上升、重量增加、剛性下降、故障點變多。控制上也更複雜——逆運動學不再有唯一解，必須額外定義「要選哪一組」的準則（第 12 章的分層逆運動學）。

這解釋了產業現況：多數工業手臂是六軸（剛好夠用、成本最低），七軸手臂則多見於需要在狹窄空間作業或與人協作的場合。台灣廠商的協作機器人常見六軸與七軸並存的產品線，正是針對這兩種不同的需求。

4.10 工作空間與可達空間

4.10.1 三種空間的定義

工作空間 (workspace) 是末端執行器所能到達的所有位置的集合，它是機器人選型時最直接的規格。但工程上必須進一步區分三個層次。

可達工作空間 (reachable workspace)：末端至少能以某一種姿態到達的所有點。這是最寬鬆的定義，也是型錄上「工作半徑」通常指的範圍。

靈巧工作空間 (dexterous workspace)：末端能以任意姿態到達的所有點。這個範圍恆小於可達工作空間，有時甚至小得多。對需要從各種角度接近工件的任務（如焊接、噴塗），靈巧工作空間才是真正有意義的規格。

有效工作空間 (usable workspace)：扣除掉自我碰撞、關節極限、線纜干涉、以及靠近奇異點而不可用的區域後，實際能穩定作業的範圍。這通常又比靈巧工作空間再小一圈，卻是實務規劃時真正該用的數字。

圖 4-3 三種工作空間的層次關係

建議配圖：以六軸手臂為例的剖面圖，畫出三層同心的區域：最外層淺色為可達工作空間（含手臂完全伸直的邊界球面）、中層為靈巧工作空間、最內層深色為有效工作空間；並標出三個被扣除的區域：基座附近的自我碰撞區、完全伸直的邊界奇異區、以及關節極限造成的死角。

4.10.2 常見構型的工作空間形狀

直角座標式 (PPP，三個移動關節)：長方體。優點是形狀直觀、程式好寫、精度均勻；缺點是機構體積遠大於工作空間。3D 印表機與龍門式取放機屬此類。

圓柱座標式 (RPP)：中空圓柱體。一個旋轉軸加兩個移動軸，適合繞著中心排列的取放作業。

球座標式 (RRP)：中空球殼的一部分。早期的 Unimate 即為此類。

關節式 (RRR，六軸手臂)：近似球體但形狀不規則，中間有基座造成的空洞、外緣有伸直極限。這是工作空間對機構體積比最高的構型，也是主流。

SCARA：圓柱狀但較扁平，因為 Z 軸行程通常遠小於水平臂展。

Delta：一個上寬下窄的碗狀或圓錐狀區域。

4.10.3 邊界的兩個陷阱

第一個陷阱是工作空間邊界處的性能急遽下降。當手臂完全伸直時，它剛好到達可達空間的邊界，但在這個位形上，末端沿著徑向已經無法再前進——這正是第 12 章的邊界奇異點。此時即使關節仍能轉動，末端在某個方向就是動不了，且逆運動學的解會變得極不穩定。因此實務上，規劃作業位置時務必與邊界保持餘裕，通常建議保留百分之十以上。

第二個陷阱是工作空間的空洞。多數六軸手臂在基座正上方與正下方附近有無法到達的區域（因為機構會自我碰撞，或需要無限大的關節速度通過）。學生在配置工作站時，若把待取工件放在基座正前方太近的位置，常常會發現「型錄上明明說工作半徑一點五公尺，為什麼零點三公尺處反而拿不到」——答案就在這裡。

4.11 機器人結構選型

4.11.1 選型的六個問題

為一個任務挑選機器人構型，可以依序回答六個問題。第一，任務需要幾個自由度？平面搬運只需 3、垂直插裝需 4（SCARA 即可）、任意姿態作業需 6、需要避開障礙或與人協作則考慮 7。

第二，工作空間的形狀與大小為何？是長條狀的產線（考慮龍門或加裝第七軸行走軸）、還是圍繞一點的環狀配置（六軸手臂）、或是扁平的平面（SCARA）？

第三，速度與節拍要求多高？每分鐘超過六十次的取放，幾乎只有 Delta 與高速 SCARA 能勝任。

第四，負載多重？含末端夾具的總重，並記得預留百分之二十以上的餘裕，且要考慮加速時的動態負載，而非只看靜態重量。

第五，精度要求多少？請區分重複精度（回到同一點的一致性，通常是型錄上較漂亮的那個數字）與絕對精度（到達指定座標的準確度，通常差一個數量級）。多數產線任務只需重複精度，離線編程與視覺導引則需要絕對精度。

第六，環境與安全條件為何？無塵室、油污、高溫、需與人共處——這些決定了防護等級與是否必須採用協作機器人。

表 4-6 依任務選擇構型的速查表

任務需求	建議構型	理由
電子零件垂直插裝、鎖螺絲	SCARA	4 DoF 剛好夠用、水平快、垂直剛硬、成本低
食品藥品高速分揀取放	Delta	加速度極高、運動慣量小、易清潔
焊接、噴塗、任意角度作業	六軸串聯	6 DoF 可任意姿態、工作空間大
狹窄空間作業、與人協作	七軸協作機器人	冗餘可避障、力量受限較安全
大面積平面加工、3D 列印	龍門直角座標	精度均勻、程式直觀、行程可延長
飛行模擬、精密微定位	Stewart 平台	6 DoF、剛性與精度極高
跨區域搬運物料	AMR + 機械手臂	移動平台擴展工作空間 (第 23 章)

4.11.2 一個常見的選型錯誤

學生與初次導入的企業最常犯的錯誤，是只看「負載」與「工作半徑」兩個數字就下單。實際上，機器人型錄上的額定負載通常是在末端法蘭處、以標準姿態、在低速下量測的。當你裝上一支長 300 公釐的夾具、抓取一個重心偏移的工件、並要求它高速運動時，等效負載與力矩可能是額定值的兩三倍，導致手臂震動、精度下降、甚至觸發過載保護。

正確的做法是計算「負載慣量矩」(load inertia moment)：不只算重量，還要算重量乘上重心到法蘭的距離。各大廠商都提供負載能力曲線圖與線上選型工具，務必使用。這個概念會在第 5 章的馬達選型與第 14 章的動力學中再次出現。

◎ 台灣產業案例

中部某工具機廠評估工具機上下料自動化。工件重 8 公斤、夾具重 3 公斤、需在兩台機台之間搬運 (相距 2.5 公尺)、要求節拍 25 秒。初步構想是買一台大型六軸手臂放在中間，但工作半徑需 1.5 公尺以上，價格昂貴且佔地大。最後採用的方案是：中型六軸手臂 (負載 20 公斤、半徑 1.4 公尺) 加裝一支 3 公尺的第七軸行走軸 (移動關節)。這個「串聯手臂 + 線性軸」的組合，用較低的成本換到了長條狀的工作空間——正是 4.11.1 節第二個問題的實際體現。

完整計算例題

例題 4-1 平面四連桿機構的自由度

題目：一個平面四連桿機構 (如汽車雨刷或縫紉機的連桿)，由四根連桿 (含固定基座) 以四個旋轉關節首尾相連成一個封閉迴路。求其自由度。

解：這是平面機構，使用平面 Grübler 公式。連桿數 $N = 4$ (含基座)，關節數 $J = 4$ ，每個

旋轉關節 $f_i = 1$ ，故 $\sum f_i = 4$ 。

$$\text{DoF} = 3(N - 1 - J) + \sum_i f_i = 3(4 - 1 - 4) + 4 = 3(-1) + 4 = 1$$

平面四連桿機構的自由度為 1。

結果為 1，符合物理直覺：只要驅動其中任一根連桿（例如馬達帶動曲柄），整個機構的運動就完全確定。這正是四連桿機構在工程上如此普及的原因——一顆馬達就能產生複雜且可重複的運動軌跡。若讀者算出 4（誤用了開放鏈的公式），請回頭確認是否忽略了「封閉迴路」這個關鍵條件。

例題 4-2 六軸工業手臂與 Delta 機器人的自由度

題目：(a) 求六軸串聯工業手臂的自由度。(b) 求標準 Delta 機器人的自由度。Delta 的結構為：基座（1 根）+ 三根主動臂（3 根）+ 三組平行四邊形連桿（每組視為 2 根連桿）+ 動平台（1 根）；關節配置為：3 個旋轉關節（主動，連接基座與主動臂）+ 每條支鏈上下各 2 個球關節（共 $3 \times 4 = 12$ 個球關節）。

解 (a)：六軸手臂是開放鏈，可直接用簡化式。 $N = 7$ （基座 + 6 根連桿）， $J = 6$ ，全為旋轉關節。

$$\text{DoF} = 6(N - 1 - J) + \sum_i f_i = 6(7 - 1 - 6) + 6 = 0 + 6 = 6$$

六軸手臂的自由度為 6，恰好足以達成空間中任意位置與姿態。

解 (b)：Delta 為空間閉合鏈。連桿數 $N = 1$ （基座）+ 3（主動臂）+ 6（三組平行四邊形，每組 2 根）+ 1（動平台）= 11。關節數 $J = 3$ （旋轉）+ 12（球關節）= 15。自由度總和 $\sum f_i = 3 \times 1 + 12 \times 3 = 3 + 36 = 39$ 。

$$\text{DoF} = 6(11 - 1 - 15) + 39 = 6(-5) + 39 = -30 + 39 = 9$$

公式算出 9，但實際 Delta 只有 3 個自由度。

為什麼差了 6？因為每組平行四邊形連桿中的兩根連桿，可以各自繞著自身的軸線自由旋轉——這是「被動的局部自由度」（local mobility），對動平台的運動沒有任何貢獻。三組共有 6 個這樣的無效自由度。扣除後： $9 - 6 = 3$ ，正是 Delta 的三個平移自由度。

工程啟示：Grübler 公式算的是機構中所有的獨立運動參數，包含那些「空轉但不影響輸出」的局部自由度。分析並聯機構時，務必逐條檢查是否存在此類無效自由度——這也是 4.6.3 節所說「公式是工具不是真理」的具體體現。

例題 4-3 工作空間與冗餘判定

題目：某任務需要在一個 60 公分 × 40 公分的平面上，將零件垂直插入孔位，插入時零件必須保持垂直、但可繞垂直軸調整角度。(a) 此任務需要幾個自由度？(b) 若使用六軸手臂，冗餘度為何？(c) 若改用 SCARA，是否足夠？

解 (a)：末端需要控制的量為：平面位置 x 、 y (2)、插入深度 z (1)、繞垂直軸的旋轉角 θ (1)。至於傾斜的兩個旋轉自由度，任務要求「保持垂直」，即這兩個量被固定為常數，不需要主動控制。故任務自由度 = 4。

解 (b)：六軸手臂有 6 個自由度，任務只需 4 個，冗餘度 = $6 - 4 = 2$ 。這兩個多出來的自由度可用於避開治具干涉、或選擇對手臂更省力的姿勢。

解 (c)：SCARA 的四個自由度 (x, y, z, θ) 與任務需求完全一致，冗餘度 = 0。它足夠且剛好，加上垂直方向剛性高、水平方向具選擇性順應（有利於插入時的自動對正）、速度快、成本低——因此 SCARA 是此任務的最佳選擇。這是「剛剛好就好」的典型範例。

程式範例：自由度計算器

以下 Python 程式實作 Grübler 公式，可計算平面或空間機構的自由度，並支援扣除局部自由度。程式最後以例題中的三個機構驗證。

```
JOINT_DOF = {
    "R": 1,    # 旋轉關節 Revolute
    "P": 1,    # 移動關節 Prismatic
    "H": 1,    # 螺旋關節 Helical
    "C": 2,    # 圓柱關節 Cylindrical
    "U": 2,    # 萬向關節 Universal
    "S": 3,    # 球關節 Spherical
    "E": 3,    # 平面關節 Planar
}

def grubler(n_links, joints, space=True, local_dof=0):
    """
    n_links : 連桿總數 (含固定基座)
    joints  : 關節型別清單，如 ["R", "R", "S"]
    space   : True 為空間機構(6)，False 為平面機構(3)
    local_dof : 需扣除的被動局部自由度數
    """
    m = 6 if space else 3
    j = len(joints)
```

```

sum_f = sum(JOINT_DOF[t] for t in joints)
dof = m * (n_links - 1 - j) + sum_f - local_dof
return dof, sum_f

def report(name, n, joints, space, local=0):
    dof, sum_f = grubler(n, joints, space, local)
    kind = "空間" if space else "平面"
    print(f"{name}")
    print(f"  {kind}機構 | 連桿 N={n} | 關節 J={len(joints)} | Σf={sum_f}"
          f" | 局部自由度扣除={local}")
    print(f"  → 自由度 DoF = {dof}\n")

# 例題 4-1：平面四連桿
report("平面四連桿機構", 4, ["R"]*4, space=False)

# 例題 4-2(a)：六軸串聯手臂
report("六軸工業手臂", 7, ["R"]*6, space=True)

# 例題 4-2(b)：Delta 機器人 (含 6 個被動局部自由度)
report("Delta 機器人", 11, ["R"]*3 + ["S"]*12, space=True, local=6)

# 補充：Stewart 平台 (6 條 UPS 支腿)
# 連桿：基座 1 + 每腿 2 段(上下)×6 + 動平台 1 = 14
report("Stewart 平台", 14, (["U", "P", "S"]*6), space=True, local=6)

```

執行結果應為：四連桿 1、六軸手臂 6、Delta 3、Stewart 平台 6。請試著修改：把六軸手臂改成七軸 (N=8、七個 R)，看看自由度變成多少；再想想這多出來的一個自由度，在開頭故事中對應到什麼動作。

■ 模擬實作：用 URDF 描述一個機構

機器人的機構在軟體中如何描述？ROS 2 使用 URDF (Unified Robot Description Format) ——一種以 XML 描述連桿與關節的格式。以下是一個平面兩連桿手臂的最簡 URDF 片段，展示了本章概念如何落實為程式碼：

```

<?xml version="1.0"?>
<robot name="two_link_arm">

  <!-- 連桿 0：固定基座 -->
  <link name="base_link"/>

```

```

<!-- 連桿 1 -->
<link name="link1">
  <visual>
    <geometry><box size="0.3 0.05 0.05"/></geometry>
    <origin xyz="0.15 0 0"/>
  </visual>
</link>

<!-- 關節 1：旋轉關節，連接 base_link 與 link1 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <axis xyz="0 0 1"/>          <!-- 繞 Z 軸旋轉 -->
  <limit lower="-3.14" upper="3.14"
          effort="10" velocity="2.0"/>    <!-- 關節極限與能力 -->
</joint>

<!-- 連桿 2 -->
<link name="link2">
  <visual>
    <geometry><box size="0.25 0.05 0.05"/></geometry>
    <origin xyz="0.125 0 0"/>
  </visual>
</link>

<!-- 關節 2：旋轉關節，連接 link1 與 link2 -->
<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0.3 0 0"/>      <!-- 沿連桿 1 長度偏移 -->
  <axis xyz="0 0 1"/>
  <limit lower="-2.6" upper="2.6"
          effort="5" velocity="2.0"/>
</joint>

</robot>

```

請注意這份檔案如何精確對應本章的概念：link 標籤是連桿、joint 標籤是關節、type="revolute" 指定關節類型（改成 prismatic 就變成移動關節）、parent 與 child 定義了運動鏈的連接順序、limit 則是關節行程極限——正是 4.10.3 節中限縮有效工作空間的因素之一。

實作步驟：（一）將上述內容存為 two_link.urdf。（二）若已安裝 ROS 2，執行 check_urdf two_link.urdf 驗證語法並印出運動鏈結構。（三）用 urdf_to_graphiz 產生連桿關節結構圖。（四）試著加入第三個連桿與關節，觀察自由度如何變化。第 29 章會用 RViz 把它視覺化並讓它動起來。

國中小也看得懂：數數看有幾個可以轉的地方

你有沒有玩過那種可以彎來彎去的檯燈？它有幾個「可以轉的地方」？每一個可以轉的地方，就是一個「自由度」。轉的地方越多，燈頭就能擺到越多不同的位置。

現在做個小實驗：把你的手掌平放在桌上不要移開，然後試著把手肘抬高。有沒有發現——手掌完全沒動，可是手臂的樣子變了！這是因為你的手臂「可以轉的地方」比需要的還多了一個，所以它有好幾種方法可以擺出同一個姿勢。這就叫做「多出來的自由」，它很有用——你伸手到書桌下面撿橡皮擦時，就是靠這個多出來的自由，讓手肘繞過桌角。

機器人也一樣。有的機器人手臂只有四個可以轉的地方（像工廠裡鎖螺絲的機器手），因為它只需要上下左右轉一轉就夠了；有的有六個、七個，因為它要像人一樣做各種動作。下次你看到機器人，試著數數看：它身上有幾個可以動的關節？

叫獸提醒與常見錯誤

- 誤區一：自由度等於馬達數。這只在「開放鏈且每關節皆有驅動」時成立。閉合鏈、驅動不足系統與被動關節都會打破這個等式。
- 誤區二：計算 Grübler 公式時忘記把固定基座算進連桿數 N 。這會讓答案剛好差 6（空間）或 3（平面）。
- 誤區三：對閉合鏈誤用開放鏈的簡化式（ $\text{DoF} = \Sigma f$ ）。看到迴路就必須回到完整公式。
- 誤區四：忽略被動局部自由度。並聯機構（尤其 Delta、Stewart）常有空轉但不影響輸出的自由度，必須手動扣除。
- 誤區五：把型錄上的工作半徑當成可用範圍。實際的有效工作空間要再扣除自我碰撞區、邊界奇異區與關節極限死角，建議保留 10% 以上餘裕。
- 誤區六：選型只看負載重量。必須計算負載慣量矩（重量 $\times$ 重心到法蘭的距離），並考慮加速時的動態負載。
- 誤區七：以為自由度越多越好。每增加一軸就增加成本、重量與控制複雜度，並降低剛性。工程的目標是「剛剛好」。
- 誤區八：混淆重複精度與絕對精度。型錄上漂亮的數字通常是重複精度；離線編程與視覺導引任務需要的是絕對精度，兩者常差一個數量級。

課堂討論

1. 開頭故事的問題：如果人類的手臂只有六個自由度（少掉冗餘的那一個），日常生活會有哪三件事變得困難？請具體描述動作情境。
2. 為什麼 SCARA 只用四個自由度就能主宰電子組裝市場？如果幫它加上兩個自由度變成六軸，會得到什麼、失去什麼？
3. Delta 機器人的三顆馬達全部固定在基座上，這個設計決策帶來哪些連鎖優勢？請至少列出三項，並說明它們之間的因果關係。
4. 人形機器人的腿部若採用閉合鏈設計（把膝關節驅動器移到大腿根部），會帶來什麼好處與代價？這與第 28 章的動態行走有什麼關聯？
5. 假設你要為醫院設計一台送藥並能開門的機器人。它需要幾個自由度？移動底盤與手臂的自由度該如何分配？請畫出構型草圖並說明理由。

章末摘要

- 機構是機器人的骨骼，決定「能做什麼」的物理上限；分析建立在剛體假設之上，此假設在高速、長臂、重載時會失效。
- 連桿是剛體構件，關節限制相對運動；最常用的是 1 自由度的旋轉關節（R）與移動關節（P），球關節（S）為 3 自由度。
- 開放鏈工作空間大、分析簡單但剛性低、誤差累積；閉合鏈剛性高、精度好、可高速，但工作空間小、正向運動學複雜。
- 空間剛體有 6 個自由度、平面剛體有 3 個，這是所有自由度計算的基礎。
- 自由度不等於馬達數：驅動不足、冗餘驅動與被動關節都會打破此等式。
- Grübler 公式：空間 $\text{DoF} = 6(N-1-J) + \sum f_i$ ，平面 $\text{DoF} = 3(N-1-J) + \sum f_i$ ；開放鏈退化為 $\text{DoF} = \sum f_i$ 。
- 使用公式須注意三點：基座要算進 N、需扣除被動局部自由度、過度約束機構會使公式失準。
- 串聯式範圍大彈性高，並聯式剛性強速度快；SCARA（4 DoF）主攻垂直插裝、Delta（3 DoF）主攻高速取放、Stewart（6 DoF）主攻高剛性精密定位。
- 冗餘（自由度多於任務所需）可用於避障、避奇異點、避關節極限與最佳化次要目標，代價

是成本、重量、剛性與控制複雜度。

- 工作空間分三層：可達、靈巧、有效；規劃時應使用有效工作空間並保留邊界餘裕，注意基座附近的空洞區。
- 選型六問：自由度需求、空間形狀、速度節拍、負載（含慣量矩）、精度（重複 vs 絕對）、環境與安全條件。

觀念題與計算題

觀念題

1. 定義機構、連桿、關節與運動鏈，並在一台六軸手臂上指出各自對應的部位。
2. 剛體假設是什麼？在哪三種情況下這個假設會明顯失效？
3. 比較旋轉關節與移動關節在行程、位置關係、空間佔用、防護與成本上的差異。
4. 為什麼工業手臂幾乎全用旋轉關節，而 3D 印表機全用移動關節？請從工作空間效率與精度需求分析。
5. 如何判斷一個機構是開放鏈還是閉合鏈？請描述判斷方法。
6. 說明「空間剛體有 6 個自由度」的組成，並解釋為何六軸手臂是工業主流。
7. 舉出三種「自由度不等於馬達數」的情形，各給一個機器人上的實例。
8. 寫出空間與平面的 Grübler 公式，並說明開放鏈為何可簡化為 $DoF = \sum f_i$ 。
9. 什麼是被動局部自由度？為什麼 Delta 機器人的計算必須扣除 6？
10. 比較 SCARA、Delta 與 Stewart 平台的自由度、強項與典型應用。
11. 定義冗餘自由度，列舉其四大好處與三項代價。
12. 區分可達工作空間、靈巧工作空間與有效工作空間，並說明實務規劃該用哪一個。

計算題

1. 一個平面五連桿機構（含固定基座共 5 根連桿），以 5 個旋轉關節連接成一個封閉迴路。求其自由度，並說明需要幾顆馬達才能完全控制它。
2. 某並聯機器人由 1 個基座、3 條支鏈與 1 個動平台組成。每條支鏈含 2 根連桿，關節配置由基座往上依序為：旋轉關節（R）、萬向關節（U）、球關節（S）。（a）求連桿總數 N 與關節總數 J。（b）以 Grübler 公式計算自由度。（c）若已知此機構實際只有 3 個自由度，

請推論存在幾個被動局部自由度。

3. 一台 SCARA 手臂的第一臂長 400 公釐、第二臂長 300 公釐，兩軸的關節角度範圍分別為 ± 140 度與 ± 145 度，Z 軸行程 150 公釐。(a) 求水平面上可達工作空間的最大與最小半徑。(b) 估算此工作空間的環狀面積（不考慮角度限制）。(c) 若要在距離基座 750 公釐處作業，是否可行？請說明。
4. 某六軸手臂額定負載 10 公斤（於法蘭處量測）。現欲加裝一支重 2.5 公斤、長 250 公釐的夾具，抓取一個重 6 公斤、重心位於夾具前端 80 公釐處的工件。(a) 求總重量是否超過額定負載。(b) 求負載重心到法蘭的距離（假設夾具重心在其中點）。(c) 以「等效力矩 = 總重 $\times$ 重心距離」評估，若原廠規定法蘭處最大容許力矩為 30 N·m（取 $g = 9.8 \text{ m/s}^2$ ），此配置是否安全？

動手做與專題延伸

本章實作：機構自由度普查

在校園、住家與實驗室中找出八種含有機構的裝置，完成下表。重點不在數字正確，而在於學會用結構化的方式觀察與分析機構。

表 4-7 機構自由度普查表（前兩列為範例）

裝置名稱	連桿數 N	關節型別與數量	平面 / 空間	計算自由度	實際觀察	差異原因
彎臂檯燈	4	$R \times 3$	平面	3（開放鏈）	3（可上下、前後、旋轉）	一致
汽車雨刷	4	$R \times 4$	平面	1	1（一顆馬達帶動）	閉合迴路，一致
（自行填寫六列：剪刀、折疊傘、辦公椅、自行車變速器、抽屜滑軌、玩具機器手臂等）						

專題延伸

- 用紙板、圖釘與吸管製作一個平面三連桿手臂，實測它的工作空間：在紙上標出末端能到達的所有點，畫出邊界。然後把某一個關節的活動範圍限制在 ± 90 度，重新標繪一次，觀察有效工作空間縮小了多少。

- 拆解一支折疊傘或一台舊印表機的進紙機構，畫出它的機構簡圖（用線段代表連桿、圓圈代表旋轉關節、方塊代表移動關節），並用 Grübler 公式計算自由度，與實際操作結果比對。
- 進階：為附錄 J 的十二軸四足機器狗建立機構分析——（一）單腿在騰空時是開放鏈還是閉合鏈？自由度為何？（二）當四腳同時著地時，透過地面形成了什麼結構？此時整體的自由度如何變化？（三）這個變化對控制策略有什麼影響？（提示：對照第 28 章的支撐多邊形概念）

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 1、8 章：機構與結構設計)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 2 章：位形空間與自由度，含 Grübler 公式的完整討論)。
- Tsai, L.-W., Robot Analysis: The Mechanics of Serial and Parallel Manipulators, Wiley (並聯機構分析的經典著作)。
- Merlet, J.-P., Parallel Robots, 2nd ed., Springer (Stewart 平台與並聯機器人專論)。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (第 1 部分：運動學與機構)。
- Norton, R. L., Design of Machinery, McGraw-Hill (機構學經典教材，四連桿與自由度分析)。
- Clavel, R., "Delta, a Fast Robot with Parallel Geometry," Proc. 18th Int. Symp. on Industrial Robots, 1988 (Delta 機器人原始論文)。
- Makino, H. 等關於 SCARA 構型之早期文獻與日本產業機器人發展史料。
- ROS 2 官方 URDF 教學文件 (docs.ros.org)：連桿、關節與機器人模型描述。
- 各機器人廠商技術型錄與選型工具：工作空間圖、負載能力曲線與慣量矩計算工具。

第 5 章

致動器、馬達與傳動系統

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一張工作檯上的「致動器家族合照」：由左至右依序為玩具直流馬達、RC 伺服機、步進馬達、無刷外轉子馬達、以及一顆剖開的機器人關節模組（露出內部的馬達、諧波減速機、雙編碼器與驅動板）。最右側放一顆線圈焦黑的燒毀馬達，旁註一行小字：它不是被壓垮的，是被自己發的熱燒死的。

叫獸開講 「同樣是馬達，為什麼玩具車馬達不能直接拿來做機器手臂？」

上學期末，兩位學生抱著一台自製的機械手臂衝進研究室，臉色發青。手臂的第二軸垂了下來，馬達燙得幾乎不能碰，空氣中有一股燒焦的塑膠味。

「老師，這顆馬達的規格明明夠啊！」小翰把型錄推過來，「我們算過了：手臂前端要舉 500 公克，力臂 30 公分，需要的扭力是 $0.5 \times 9.8 \times 0.3$ ，大約 1.5 牛頓米。我們買的伺服馬達標示 25 公斤公分，換算過來是 2.45 牛頓米，還有 60% 的餘裕！」

叫獸沒有馬上回答，而是問：「你們的手臂舉起重物之後，是靜止不動，還是會移動？」

「當然會動啊，它要在一秒內從水平轉到垂直。」

「那就對了。」叫獸拿起白板筆，「你算的是**靜態扭力**——手臂靜止不動、只對抗重力所需的扭力。但你的手臂還要在一秒內轉九十度，這代表它必須先加速、再減速。加速需要額外的扭力，而這個扭力等於轉動慣量乘上角加速度。你們的連桿有多長？」

「30 公分，鋁合金，大概 200 公克。」

叫獸開始算：「連桿加負載的轉動慣量，粗估大約 $0.03 \text{ kg}\cdot\text{m}^2$ 。一秒內轉 90 度，若採梯形速度曲線，角加速度大約 6 rad/s^2 。動態扭力就是 $0.03 \times 6 \approx 0.18$ 牛頓米——嗯，這部分其實不大。」他停頓了一下，「所以問題不在這裡。你們的馬達，是不是常常在快到定位的時候，還微微抖動、發出嗡嗡聲？」

兩人對看一眼，用力點頭。「對！它到定位之後會一直抖，我們以為是程式問題。」

「那才是真正的兇手。」叫獸在白板上寫下一個詞：**堵轉**。「當馬達在定位時抖動，或是手臂被卡住、被使用者用手擋住時，馬達會進入堵轉狀態——轉不動，但電流會飆到額定值的五到十倍。這時馬達幾乎不做功，所有的電能全部變成熱。你們的馬達不是被扭力壓垮的，是被自己發的熱燒死的。」

他把燒毀的馬達拆開，指著焦黑的線圈：「型錄上寫的 25 公斤公分，是**瞬間堵轉扭力**——那是它拚了老命才能撐住的極限，不是它能持續輸出的力量。可以持續使用的**額定扭力**，通常只有標示值的三分之一到二分之一。這是初學者最常掉進去的坑：把極限值當成常用值。」

小翰苦笑：「所以我們該買多大的？」叫獸把型錄翻到後面的曲線圖：「不是買多大，是先

學會看這張圖。今天這堂課，我們就從一顆馬達的內在說起——為什麼它會轉、為什麼轉速快就沒力氣、為什麼需要減速機、以及為什麼有時候馬達太大反而控制不好。」

本章為什麼重要

第 4 章決定了機器人的骨骼，本章決定它的肌肉。致動器是機器人系統中最貴、最重、最容易燒毀，也最常被選錯的元件。一台機器人的成本結構中，馬達與減速機往往佔了三到五成；而學生專題失敗的原因排行榜上，「馬達選錯」也長年名列前茅。

本章要建立三個關鍵能力。第一，看懂馬達型錄——分辨額定值與極限值、讀懂扭力轉速曲線。第二，正確計算需求——不只是靜態負載，還包括加速所需的動態扭力。第三，理解匹配——馬達、減速機與負載三者是一個整體，慣量比不當時，再強的馬達也控制不好。這些能力會直接支撐第 14 章的動力學計算與第 16 章的控制器設計。

學習目標

- 能說明致動器在機器人系統中的角色，並比較電動、液壓與氣壓三類致動器。
- 能說明直流馬達的基本原理，並解釋扭力與轉速的反比關係。
- 能比較有刷直流、無刷直流、步進與伺服馬達的特性與適用場合。
- 能讀懂馬達型錄，區分額定扭力、堵轉扭力、額定轉速與空載轉速。
- 能計算馬達的輸出功率，並由扭力與轉速需求反推所需功率。
- 能說明減速機的作用，並計算減速比對扭力、轉速與慣量的影響。
- 能比較諧波、行星與擺線三種精密減速機的特性。
- 能說明螺桿、皮帶、鏈條與鋼索四種傳動方式的優缺點。
- 能計算轉動慣量與慣量比，並判斷馬達與負載是否匹配。
- 能依任務需求完成一次完整的致動器選型計算，含靜態、動態與安全餘裕。
- 能說明馬達驅動器的功能與保護電路的必要性。

先備知識

第 3 章的驅動器概念與第 4 章的機構自由度。本章需要高中物理的力矩、轉動慣量與功率概

念，以及基本的單位換算能力。若對轉動慣量不熟悉，可先閱讀附錄 D.1 的力學速成；本章所有公式都會附上物理意義的說明與數值範例。

核心名詞中英對照

表 5-1 本章核心名詞

中文	英文	說明
致動器	Actuator	將能量轉換為機械運動的元件
扭力 (轉矩)	Torque	使物體轉動的作用，單位 N·m
額定扭力	Rated Torque	可持續輸出而不過熱的扭力
堵轉扭力	Stall Torque	轉速為零時的最大瞬間扭力
空載轉速	No-load Speed	無負載時的最高轉速
反電動勢	Back EMF	馬達旋轉時產生的反向電壓
扭力常數	Torque Constant, K_t	電流與扭力的比例係數，N·m/A
占空比	Duty Cycle	PWM 訊號中高電位所佔的比例
減速機	Gearbox / Reducer	降低轉速並放大扭力的傳動元件
減速比	Gear Ratio	輸入轉速與輸出轉速之比
背隙	Backlash	齒輪嚙合間隙造成的空轉量
轉動慣量	Moment of Inertia	物體抵抗角加速度的量度， $\text{kg}\cdot\text{m}^2$
慣量比	Inertia Ratio	折算後負載慣量與馬達轉子慣量之比
諧波減速機	Harmonic Drive	利用彈性齒輪變形的高減速比零背隙減速機
伺服系統	Servo System	具位置回授的閉迴路驅動系統
驅動器	Motor Driver	將控制訊號放大為馬達功率的電路

5.1 致動器的功能與分類

5.1.1 能量轉換的終點站

致動器 (actuator) 是機器人系統中把能量轉換成機械運動的元件。它位於整條訊號鏈路的終點：感測器把物理世界轉成訊號、控制器把訊號轉成指令、驅動器把指令轉成功率、最後由致動器把功率轉成運動。第 1 章的「感知—思考—行動」循環中，致動器就是「行動」的執行者。

依能量來源，致動器分為三大類：電動 (電能)、液壓 (液體壓力)、氣壓 (壓縮空氣)。

此外還有新興的智慧材料致動器（形狀記憶合金、介電彈性體、人工肌肉），目前多在研究階段。

5.1.2 三大類的比較

電動致動器是機器人的主流。優點是控制精準（電流與扭力成正比，容易做閉迴路）、乾淨無洩漏、能量來源方便、效率高、易於整合感測器。缺點是功率密度（單位重量能輸出的功率）不如液壓，且在低速大扭力時需要減速機。

液壓致動器的功率密度極高——這是它至今無法被完全取代的原因。挖土機、大型工程機械、以及波士頓動力早期的 BigDog 與 Atlas 都採用液壓，因為它能在很小的體積內輸出巨大的力量，且能承受劇烈衝擊。缺點是需要泵、油箱與管路（笨重且噪音大）、會漏油、維護成本高、控制精度較差。近年來人形機器人的發展趨勢已明顯從液壓轉向高功率密度的電動方案。

氣壓致動器便宜、輕巧、天生柔順（空氣可壓縮，撞到人時會自然退讓）。工廠中大量用於夾爪、推料與簡單的兩點式動作。缺點是因為空氣可壓縮，位置控制困難，通常只能做「全開 / 全關」的開關式動作。這個「天生柔順」的特性，在軟性機器人與人機協作領域反而成為優點。

表 5-2 三類致動器的比較

面向	電動	液壓	氣壓
功率密度	中	極高	低
控制精度	高（可精確位置控制）	中	低（可壓縮）
響應速度	快	快	中
剛性	高（透過減速機）	極高	低（天生柔順）
維護	少	多（漏油、濾芯）	中（需乾燥空氣）
噪音	低	高（泵浦）	高（排氣）
成本	中	高	低
典型應用	工業手臂、協作機器人、AMR	重型機械、大型腿式機器人	夾爪、推料、軟性機器人

5.2 直流馬達的原理與特性

5.2.1 為什麼會轉

直流馬達的原理是安培力：載流導體在磁場中會受力。定子（外殼上的永久磁鐵或激磁線圈）提供固定磁場，轉子上的線圈通電後受力而轉動。關鍵在於，當線圈轉過半圈後，受力方向會反

過來——因此需要換向器 (commutator) 與電刷 (brush) 在適當時機切換電流方向，讓轉子持續朝同一方向轉。這就是「有刷」馬達名稱的由來。

對機器人工程師而言，最重要的是兩個線性關係。第一，扭力與電流成正比：

$$\tau = K_t \cdot I$$

τ 為扭力 ($N \cdot m$)， K_t 為扭力常數 ($N \cdot m/A$)， I 為電流 (A)。這是電流控制即力矩控制的物理基礎。

第二，反電動勢與轉速成正比。馬達轉動時本身也是發電機，會產生反向電壓 (back EMF) 阻礙電流流入：

$$V_{emf} = K_e \cdot \omega$$

V_{emf} 為反電動勢 (V)， K_e 為反電動勢常數， ω 為角速度 (rad/s)。

5.2.2 扭力—轉速曲線：一切的關鍵

把上面兩式與電路方程結合，就能推出直流馬達最重要的特性：扭力與轉速呈線性反比。轉速越高、反電動勢越大、可流入的電流越小、因此扭力越小。這條直線的兩個端點分別是：

堵轉扭力 (stall torque)：轉速為零時的最大扭力。此時沒有反電動勢，電流達到最大 (僅受線圈電阻限制)，扭力最大——但也是發熱最嚴重的狀態。空載轉速 (no-load speed)：扭力為零時的最高轉速。

圖 5-1 直流馬達的扭力—轉速—功率—效率曲線

建議配圖：橫軸為扭力 (由 0 到堵轉扭力)，左縱軸為轉速 (由空載轉速線性下降至 0)、右縱軸為功率與效率。標出四條線：轉速 (直線下降)、電流 (直線上升)、輸出功率 (拋物線，峰值在中點)、效率 (在低扭力區達到最高後下降)。並用網底標出「連續運轉區」(約在堵轉扭力的 1/3 以內) 與「短時間可用區」。

5.2.3 最大功率點與最佳效率點不同

輸出功率是扭力與轉速的乘積： $P = \tau \cdot \omega$ 。由於兩者呈線性反比，功率曲線呈拋物線，峰值恰好落在「堵轉扭力的一半、空載轉速的一半」處。但要注意：最大功率點並不是最佳效率點。在最大功率點，馬達的效率通常只有 50% 左右，另一半的能量全變成熱。最佳效率點通常落在較低的扭力區 (約堵轉扭力的 10~30%)。

這對機器人設計的啟示是：日常運轉應該落在高效率區，只有在偶爾需要爆發力時才短暫進入高扭力區。這正是「額定扭力」與「堵轉扭力」必須分開看的原因——也是開頭故事中那顆馬達的死因。

$$P = \tau \cdot \omega \quad (\omega \text{ 單位為 rad/s ; 若用 rpm 需換算 : } \omega = 2\pi N/60)$$

馬達輸出功率的基本關係式。

5.3 無刷直流馬達

5.3.1 去掉電刷之後

有刷馬達的電刷是機械接觸元件，會磨損、會產生火花與電磁干擾、且限制了轉速與壽命。無刷直流馬達（BLDC）把這個機械換向改成電子換向：永久磁鐵移到轉子上、線圈固定在定子上，再由驅動器根據轉子位置（以霍爾感測器或無感測演算法偵測）切換三相電流的通電順序。

優點極為明顯：壽命長（無磨損件）、效率高（無電刷摩擦與壓降）、功率密度大、散熱好（線圈在外殼上容易導熱）、無火花可用於易燃環境。缺點是必須配合較複雜的三相驅動器，成本較高。

5.3.2 機器人領域的兩種重要變體

外轉子馬達（outrunner）：外殼旋轉、內部固定。轉子直徑大，因此扭力大但轉速較低。無人機的螺旋槳馬達與近年四足機器人的關節馬達多屬此類。

準直驅馬達（quasi-direct drive, QDD）：使用大扭力的外轉子馬達搭配低減速比（約 6:1 到 10:1）的行星減速機。這是近年腿式機器人的關鍵技術突破——低減速比意味著背隙小、摩擦低、且可以「反向驅動」（backdrivable），使關節能感知外力並柔順地順應衝擊。傳統高減速比方案（如諧波減速機 100:1）雖然扭力大、剛性高，但幾乎無法反向驅動，撞到東西時會硬碰硬。第 28 章討論四足機器人的落地緩衝時，會再回到這個特性。

◎ 反向驅動性為何重要

把馬達關掉，用手去轉關節：如果能輕鬆轉動，就是可反向驅動；如果紋風不動，就是不可反向驅動。可反向驅動的關節能透過馬達電流「感覺」到外力，不需額外的力覺感測器，而且撞到人時會自然退讓——這是協作機器人與腿式機器人的安全基礎。減速比越高，反向驅動性越差。

5.4 步進馬達

5.4.1 用磁極數步走

步進馬達的定子上有多組線圈，依序通電時，轉子會被磁力一格一格地「吸」過去。常見的規格是每步 1.8 度（即每轉 200 步）。它的最大特色是：不需要位置回授，只要數出送了幾個脈

波，就知道轉了幾度——這就是所謂的開迴路定位。

優點是控制簡單（脈波數即位置）、低速扭力大、保持扭力強（停止時仍能牢牢抓住位置）、成本低。這使它成為 3D 印表機、雕刻機與小型 XY 平台的主流選擇。

5.4.2 失步：開迴路的致命弱點

步進馬達的最大風險是失步（step loss）。當負載超過馬達能力、或加速過快時，轉子跟不上磁場的變化，就會「跳過」幾步。此時控制器仍然以為位置正確，但實際位置已經偏移，且這個誤差會永久累積——因為沒有回授可以修正。3D 印表機列印到一半突然「錯層」，多半就是失步造成的。

此外，步進馬達的扭力隨轉速上升而急遽下降，高速時扭力所剩無幾；且在特定轉速下會產生共振而失步。因應之道包括：加大安全餘裕（通常留 50% 以上）、限制加速度、採用微步驅動（microstepping）平滑運轉，或加裝編碼器變成閉迴路步進系統。

5.5 伺服馬達

5.5.1 伺服不是一種馬達，而是一套系統

嚴格來說，「伺服」（servo）指的不是馬達的種類，而是一套具備位置回授的閉迴路控制系統：馬達 + 編碼器 + 驅動器 + 控制迴路。馬達本身可能是有刷或無刷直流馬達，關鍵在於它有回授、能自我修正。

工業伺服系統通常採三層串級控制：最內層是電流（扭力）迴路，週期約數十微秒；中層是速度迴路，週期約數百微秒；外層是位置迴路，週期約一毫秒。這正是第 3 章時間尺度金字塔的具體實例，其控制原理將在第 16 章詳述。

5.5.2 RC 伺服機：教學與社團的主力

學生最常接觸的是 RC 伺服機（俗稱舵機），如 SG90、MG996R 等。它把馬達、減速齒輪、電位器與控制電路封裝在一個小盒子裡，只需三條線（電源、地、訊號），用 PWM 訊號的脈寬即可指定角度——標準協定是 1.0 毫秒對應 0 度、1.5 毫秒對應 90 度、2.0 毫秒對應 180 度。

使用 RC 伺服機有三個實務要點。第一，型錄標示的扭力（如「25 kg·cm」）是堵轉扭力，持續可用約為其三分之一。第二，必須使用獨立電源——伺服機啟動瞬間的電流可達 1~2 安培，

若與微控制器共用電源會造成重開機（第 3 章的經典故障）。第三，塑膠齒輪的機型在受到衝擊時容易崩齒，關節類應用建議選用金屬齒輪版本。

表 5-3 四種常見馬達的比較

馬達類型	定位方式	優點	缺點	典型應用	本書實作
有刷直流馬達	需外加編碼器	便宜、驅動簡單	電刷磨損、有火花	輪型底盤、玩具	附錄 A、F
無刷直流馬達	霍爾或編碼器	效率高、壽命長、功率密度大	驅動器複雜、成本高	無人機、四足關節	附錄 I、J
步進馬達	開迴路（脈波計數）	低速扭力大、定位簡單	會失步、高速無力	3D 印表機、XY 平台	—
RC 伺服機	內建閉迴路	整合度高、只需三條線	扭力有限、精度普通	小型手臂、雙足關節	附錄 B ~ E、J

5.6 馬達的扭力、轉速與功率計算

5.6.1 靜態扭力：對抗重力

最基本的需求是舉起負載所需的靜態扭力。對一根水平伸出的連桿：

$$\tau_{\text{static}} = m \cdot g \cdot L$$

m 為負載質量 (kg)， $g = 9.8 \text{ m/s}^2$ ， L 為重心到轉軸的距離 (m)。若連桿本身有重量，需另計其重心力矩並相加。

注意兩件事。第一，力臂 L 是「重心」到轉軸的距離，不是連桿長度——若連桿均勻，其重心在中點。第二，這是最壞情況（連桿水平時力矩最大）；當連桿轉到垂直時，重力力矩為零。設計時必須以最壞情況為準。

5.6.2 動態扭力：對抗慣性

要讓手臂加速，還需要額外的扭力，由牛頓第二定律的轉動版本給出：

$$\tau_{\text{dynamic}} = J \cdot \alpha$$

J 為轉動慣量 ($kg \cdot m^2$)， α 為角加速度 (rad/s^2)。

角加速度可由運動需求推算。若採用簡單的「加速一半、減速一半」三角形速度曲線，在時間 t 內轉過角度 θ ：

$$\alpha = 40 / t^2$$

三角形速度曲線下的角加速度。若採梯形曲線（含等速段），加速度會略大，需依實際曲線計算（第 15 章）。

5.6.3 總需求與安全餘裕

馬達所需的總扭力為兩者相加，再乘上安全係數：

$$\tau_{\text{required}} = (\tau_{\text{static}} + \tau_{\text{dynamic}}) \times SF$$

SF 為安全係數，一般取 1.5~2.0，考慮摩擦、效率損失、製造誤差與未來的負載增加。

最後，把這個扭力需求與馬達的**額定扭力**（不是堵轉扭力）比較。若使用減速機，需求扭力要除以減速比與傳動效率，才能得到馬達端的需求——這是 5.8 節的內容。例題 5-1 會完整走一遍這個流程。

5.7 減速機與齒輪比

5.7.1 為什麼幾乎每個關節都需要減速機

電動馬達有一個先天特性：它們在高轉速時效率最好，但機器人的關節需要的是低轉速、大扭力。一顆典型的無刷馬達最佳工作點在每分鐘三千轉以上，而機器手臂的關節轉速通常低於每分鐘六十轉。這中間相差五十倍以上，必須靠減速機來轉換。

減速機的作用可以用一句話概括：用轉速換扭力。減速比 N 定義為輸入轉速與輸出轉速之比，其效果為：

$$\omega_{\text{out}} = \omega_{\text{in}} / N \quad \tau_{\text{out}} = \tau_{\text{in}} \times N \times \eta$$

N 為減速比， η 為傳動效率（一般 0.7~0.95）。轉速降為 $1/N$ ，扭力放大 N 倍（再乘上效率）。

這說明了一個重要的設計槓桿：與其買一顆大十倍的馬達，不如加一組減速比 10 的減速機——後者通常更輕、更便宜、也更容易散熱。這正是所有工業機器人關節都內建減速機的原因。

5.7.2 減速機對慣量的影響：平方關係

減速機還有一個常被忽略、卻極其重要的效果：它會以平方的比例縮小負載慣量在馬達端看到的大小。

$$J_{\text{reflected}} = J_{\text{load}} / N^2$$

折算到馬達端的負載慣量與減速比的平方成反比。

這個平方關係是機器人設計的關鍵武器。假設負載慣量是 $1 \text{ kg}\cdot\text{m}^2$ ，透過減速比 100 的減速

機後，馬達「感覺到」的慣量只有 $0.0001 \text{ kg}\cdot\text{m}^2$ ——縮小了一萬倍。這使得一顆慣量極小的馬達，也能穩定地控制一個沉重的手臂。5.10 節的慣量匹配就建立在這個關係上。

但天下沒有白吃的午餐：減速比越高，反向驅動性越差（見 5.3.2 節）、背隙的影響被放大到輸出端、且系統的響應速度受限。這就是為什麼腿式機器人選擇低減速比（要柔順）、而工業手臂選擇高減速比（要剛硬與精準）的根本原因。

5.8 諧波、行星與擺線減速機

5.8.1 諧波減速機：工業手臂的心臟

諧波減速機（Harmonic Drive，又稱諧波齒輪）由三個元件組成：橢圓形的波產生器（wave generator，接馬達）、可撓的柔輪（flexspline，薄壁杯狀齒輪）、以及固定的剛輪（circular spline）。波產生器旋轉時，把柔輪撐成橢圓，使柔輪的齒在長軸兩端與剛輪嚙合。由於柔輪比剛輪少兩個齒，波產生器每轉一圈，柔輪只會相對倒退兩個齒——因此可在單級中達到 50:1 到 160:1 的極高減速比。

優點是：減速比極高且體積小、零背隙（齒是被壓著嚙合的）、精度極高、同時有多個齒接觸因此扭力大。缺點是：柔輪要反覆彈性變形，有疲勞壽命限制；剛性雖高但有非線性的扭轉柔度；價格昂貴（往往比馬達本身還貴）；且幾乎完全無法反向驅動。

這是六軸工業手臂與協作機器人手腕關節的標準配備，也是台灣廠商積極投入國產化的關鍵零組件——第 1 章提到的上銀科技即在此領域布局。

5.8.2 行星減速機：均衡的通才

行星減速機由中心的太陽輪、周圍的行星輪、外圈的環齒輪與行星架組成。動力由太陽輪輸入，行星輪同時自轉與公轉，由行星架輸出。因為多個行星輪同時分擔負載，扭力密度高且結構對稱、徑向力平衡。

單級減速比通常在 3:1 到 10:1，可多級串接達到更高比值（但效率會逐級相乘而下降）。優點是效率高（單級可達 95% 以上）、可反向驅動、成本適中、可承受高輸入轉速。缺點是有背隙（精密型可做到幾個角分，但成本上升）。這是四足機器人準直驅關節、AMR 輪組與各類通用場合的主流選擇。

5.8.3 擺線減速機：重載的硬漢

擺線減速機 (cycloidal drive) 利用偏心軸帶動擺線盤，擺線盤的曲線輪廓與固定的針齒接觸並滾動，產生減速。因為同時有多個齒接觸 (可達 60% 以上的齒同時受力)，承載能力與抗衝擊能力極強。

優點是：扭力密度最高、抗過載與衝擊能力極佳、剛性高、壽命長。缺點是結構複雜、成本高、有輕微背隙、且高速時因偏心運動會產生振動。典型應用在大型工業手臂的基座關節 (第一、二軸，承受整條手臂的重量) 與重載機械。

表 5-4 三種精密減速機的比較

特性	諧波減速機	行星減速機	擺線減速機
單級減速比	50 ~ 160	3 ~ 10	10 ~ 100
背隙	接近零	小 (精密型幾角分)	小
效率	70 ~ 85%	90 ~ 97%	80 ~ 90%
扭力密度	高	中	極高
抗衝擊	較弱 (柔輪疲勞)	中	極佳
反向驅動	幾乎不可	可	困難
成本	高	中	高
典型應用	工業手臂手腕、協作機器人	四足關節、AMR 輪組	大型手臂基座、重載

◎ 背隙的實際影響

背隙是齒輪嚙合的間隙，表現為「反轉時要空轉一小段才開始出力」。在定位任務中，背隙造成的誤差無法用位置回授完全消除 (因為編碼器多裝在馬達端，看不到輸出端的空轉)；在力控制中，背隙會讓接觸瞬間產生衝擊。這是精密機器人堅持使用零背隙減速機的原因，也是為什麼工業手臂的重複精度 (同方向) 遠優於雙向精度。

5.9 螺桿、皮帶、鏈條與鋼索傳動

5.9.1 滾珠螺桿：把旋轉變成直線

滾珠螺桿 (ball screw) 將馬達的旋轉運動轉換成直線運動：螺桿旋轉，螺帽沿軸向移動。與傳統梯形螺桿相比，滾珠螺桿在螺紋槽中放入循環滾動的鋼珠，把滑動摩擦變成滾動摩擦，效率由約 30% 提升到 90% 以上。

關鍵參數是導程 (lead) ——螺桿每轉一圈螺帽移動的距離。導程小則推力大、解析度高，但速度慢。直線運動的位置換算為：

$$x = (\theta / 2\pi) \times \text{lead} \quad F = (2\pi \times \eta \times \tau) / \text{lead}$$

x 為直線位移， θ 為螺桿轉角 (rad)， lead 為導程 (m)； F 為推力 (N)， τ 為馬達扭力 (N·m)， η 為效率。

滾珠螺桿是工具機、3D 印表機 Z 軸與直角座標機器人的核心元件，也是台灣精密機械產業的強項——上銀科技即為全球主要供應商之一。

5.9.2 同步皮帶：輕量與遠距離傳動

同步皮帶 (timing belt) 帶有齒形，能無滑動地傳遞旋轉。優點是輕、安靜、可長距離傳動、有一定彈性可吸收衝擊、且能把馬達移到遠離運動部位的地方 (降低運動慣量)。缺點是有彈性變形 (高精度定位時會造成遲滯)、需要張力調整、且會隨時間鬆弛。

典型應用是 3D 印表機的 XY 軸、SCARA 的第二軸傳動、以及協作機器人把馬達往基座移動的設計。設計時要注意皮帶張力：太鬆會跳齒，太緊會增加軸承負荷並縮短壽命。

5.9.3 鏈條與鋼索：重載與遠端驅動

鏈條傳動能承受大扭力、不易打滑，但笨重、有噪音、需要潤滑，機器人上較少用，多見於大型移動平台。

鋼索傳動 (cable/tendon drive) 則是仿生設計的代表：像人的肌腱一樣，用鋼索把驅動力從遠端傳到關節。最大優點是可以把所有馬達放在基座或手掌根部，讓末端極輕巧——這是靈巧機械手 (如 DLR 手、Shadow Hand) 的標準做法，也呼應第 21 章提到的人手生物力學。缺點是鋼索會伸長、有摩擦遲滯、且只能拉不能推 (需成對配置或加彈簧回復)。

表 5-5 四種傳動方式的比較

方式	運動轉換	效率	剛性	優點	限制	典型應用
滾珠螺桿	旋轉→直線	90%+	高	推力大、精度高	行程受限、成本高	工具機、3D 印表機 Z 軸
同步皮帶	旋轉→旋轉 / 直線	95%+	中	輕、安靜、可遠傳	有彈性、需張力調整	3D 印表機 XY、SCARA
鏈條	旋轉→旋轉	90%+	高	承載大、不打滑	重、吵、需潤滑	大型移動平台
鋼索 /	旋轉→拉力	70~	低~	末端極輕、可遠	只能拉、會伸長、	靈巧手、仿生機構

方式	運動轉換	效率	剛性	優點	限制	典型應用
鏈		90%	中	端驅動	有遲滯	

5.10 馬達慣量與負載匹配

5.10.1 為什麼太大的馬達反而不好

學生常以為「馬達越大越安全」，但過大的馬達會帶來一個問題：它自己的轉子就很重。當馬達轉子的慣量遠大於負載慣量時，馬達大部分的力氣都花在加速自己身上，系統反應遲鈍、效率低落。反過來，若負載慣量遠大於馬達轉子慣量，則系統容易振盪、控制器增益難以調高——就像用一根細牙籤去攪動一鍋濃粥。

衡量這個關係的指標是慣量比 (inertia ratio)：

$$R = J_{\text{reflected}} / J_{\text{motor}} = (J_{\text{load}} / N^2) / J_{\text{motor}}$$

J_{load} 為負載慣量， N 為減速比， J_{motor} 為馬達轉子慣量。

5.10.2 建議的慣量比範圍

業界的經驗準則是：高精度、高響應的伺服系統建議慣量比在 1 到 3 之間；一般工業應用可放寬到 5 到 10；超過 10 通常會出現整定時間拉長、易振盪、增益調不上去的問題。若慣量比過大，解法有三：提高減速比（記得是平方效果，最有效）、改用轉子慣量較大的馬達（有些廠商提供「大慣量」型號）、或降低負載慣量（減輕連桿重量、把重物往轉軸移近）。

這裡有一個重要的設計洞見：轉動慣量與質量分布的距離平方成正比。把手臂末端的一顆 500 公克馬達，往轉軸方向移近一半的距離，慣量就會降為四分之一。這正是為什麼協作機器人與四足機器人拚命把驅動器往基座移，也是 5.9.3 節鋼索傳動存在的理由。

5.10.3 常見形狀的轉動慣量

實務計算中，最常用的兩個公式是：

$$\text{細長桿繞一端} : J = (1/3) m L^2 \quad \text{質點繞轉軸} : J = m r^2$$

m 為質量 (kg)， L 為桿長， r 為質點到轉軸距離 (m)。

對一根末端帶負載的機器手臂連桿，總慣量約為兩者相加：連桿本身的 $(1/3)m_{\text{link}} L^2$ 加上末端負載的 $m_{\text{load}} L^2$ 。若需要更精確的值，可由 CAD 軟體直接查詢，或用第 14 章的平行軸定

理計算。

5.11 致動器選型流程

5.11.1 七個步驟

完整的致動器選型是一個反覆迭代的過程，但可依下列七步驟進行第一輪估算。

步驟一：確定運動需求。列出負載質量、力臂長度、運動範圍、要求的運動時間與運動曲線類型。

步驟二：計算靜態扭力。以最壞姿態（通常是水平伸出）計算重力力矩，含連桿自重。

步驟三：計算轉動慣量與動態扭力。用 5.10.3 節的公式估算 J ，由運動時間推算角加速度 α ，得 $\tau_{\text{dynamic}} = J \cdot \alpha$ 。

步驟四：加總並乘安全係數。 $\tau_{\text{required}} = (\tau_{\text{static}} + \tau_{\text{dynamic}}) \times SF$ ， SF 建議 1.5 ~ 2.0。

步驟五：選定減速比。由關節所需轉速與馬達的額定轉速決定： $N \approx \omega_{\text{motor_rated}} / \omega_{\text{joint_required}}$ 。

步驟六：反算馬達端需求並選型。 $\tau_{\text{motor}} = \tau_{\text{required}} / (N \times \eta)$ 。以此值對照馬達的額定扭力（非堵轉扭力）挑選型號。

步驟七：驗算慣量比。計算 $R = (J_{\text{load}}/N^2)/J_{\text{motor}}$ ，確認落在建議範圍內。若過大，回到步驟五提高減速比，重新迭代。

圖 5-2 致動器選型流程圖

建議配圖：流程圖。由「運動需求」出發 → 靜態扭力計算 → 動態扭力計算 → 乘安全係數 → 選減速比 → 反算馬達端扭力 → 查型錄選馬達 → 驗算慣量比；慣量比不合格時以回饋箭頭返回「選減速比」重新迭代。右側標註各步驟對應的公式編號。

5.11.2 容易忽略的三件事

第一，工作週期與發熱。馬達的額定扭力是以連續運轉為前提。若你的動作是「快速動作三秒、停十秒」，可以用較高的均方根扭力（RMS torque）作為判準，允許短時間超過額定值。反之，若是連續高負載運轉，還要考慮環境溫度與散熱條件，可能需要降額使用。

第二，摩擦與效率損失。減速機效率、軸承摩擦、皮帶預張力都會吃掉扭力。粗估時可在安

全係數中一併考慮，精算時應逐項列出。

第三，電源能力。馬達的峰值電流可能是額定的數倍，若電源或驅動器供不上這個電流，馬達就無法輸出型錄上的扭力——這是第 3 章電源問題的延伸，也是實務上「明明選對馬達卻沒力」的常見原因。

5.12 馬達驅動器與保護電路

5.12.1 H 橋與 PWM

如第 3 章所述，控制器輸出的是毫安培級的訊號，馬達需要的是安培級的電流，中間必須有驅動器做功率放大。直流馬達驅動器的核心是 H 橋電路：四顆功率電晶體排成 H 形，分別導通對角的兩顆，就能控制電流方向，進而控制轉向。

轉速控制則靠 PWM（脈寬調變）：以高頻（通常 1~20 kHz）快速開關電晶體，改變導通時間的比例（占空比）。若占空比為 60%，馬達感受到的等效電壓就約為電源電壓的 60%。之所以用開關而非線性調壓，是因為電晶體在完全導通或完全截止時發熱最少，效率遠高於線性方式。

PWM 頻率的選擇有個權衡：頻率太低（低於 2 kHz）會聽到馬達發出可聞的嘯叫聲；頻率太高則電晶體的開關損耗增加、發熱上升。實務上多數機器人採用 16~20 kHz，剛好在人耳聽覺上限之外。

5.12.2 三種必備的保護

第一是過流保護。馬達堵轉時電流會飆升到額定值的五到十倍（開頭故事的死因）。驅動器應具備電流偵測與限流功能，或至少在電源路徑上加裝保險絲。進階做法是監控電流並在超過門檻時自動降低占空比或停止輸出。

第二是反電動勢與飛輪二極體。馬達是電感性負載，當電晶體突然截止時，電感會產生極高的反向電壓尖峰，足以擊穿電晶體。解法是在馬達兩端反向並聯二極體（飛輪二極體），提供電流的回流路徑。多數整合型驅動 IC 已內建此保護，但使用分立元件時務必自行加裝。

第三是散熱。功率電晶體導通時仍有壓降，會產生熱。驅動器的持續電流規格是在有散熱片、環境通風的條件下量測的；若你的驅動器裝在密閉外殼中，實際可用電流會顯著低於標示值。實

務準則是：驅動器的持續電流規格至少要是馬達額定電流的 1.5 倍以上。

◎ 驅動器選型的三個數字

一、持續電流：必須大於馬達額定電流的 1.5 倍。二、峰值電流：必須大於馬達堵轉電流（或至少大於你設定的限流值）。三、電壓範圍：電源電壓必須落在驅動器規格內，並注意馬達煞車時的回生電壓可能超過電源電壓。三個數字都要看，只看一個就是下一台燒毀的驅動器。

5.13 機器人關節模組

5.13.1 把整個關節做成一個零件

近年最重要的產業趨勢之一，是關節模組化（joint module 或 actuator module）。傳統做法是工程師自行選購馬達、減速機、編碼器、驅動器與軸承，再自行設計外殼組裝；模組化則把這些全部整合在一個標準化的圓柱體中，對外只提供機械安裝面與一條通訊線（通常是 CAN 或 EtherCAT）。

這個轉變帶來三個好處。第一，開發時間大幅縮短——不需要自己處理散熱、軸承預壓與編碼器對位這些困難的細節。第二，可靠度提升——模組出廠前已完成校正與測試。第三，系統簡潔——串接式的匯流排取代了大量的動力線與訊號線（第 3 章的分散式架構）。

代價是成本較高、且被廠商的介面與韌體綁定。但對多數研發團隊與教學單位而言，用模組換取時間仍然划算——這也是為什麼近年的協作機器人與四足機器人幾乎清一色採用模組化關節。

5.13.2 關節模組的內部構成

一個典型的關節模組由內而外包含：無刷馬達（提供動力）、減速機（諧波或行星）、雙編碼器（馬達端測轉速與換相、輸出端測實際關節角度，可消除減速機背隙的影響）、力矩感測器（部分高階型號，用於力控制與碰撞偵測）、驅動電路板（含電流迴路與通訊介面）、以及煞車器（斷電時鎖住關節，避免手臂掉落）。

值得注意的是「雙編碼器」設計：只在馬達端裝編碼器的話，減速機的背隙與扭轉變形對控制器是不可見的；加裝輸出端編碼器後，控制器才能看到關節的真實角度。這是高階協作機器人能達到高絕對精度的關鍵，也解釋了第 4 章提到的「重複精度與絕對精度差一個數量級」現象。

5.14 台灣的關鍵零組件產業

5.14.1 傳動元件的世界級聚落

如第 1 章所述，台灣在精密機械領域有完整的產業聚落，而傳動元件正是其中最具有國際競爭力的一環。上銀科技 (HIWIN) 的滾珠螺桿與線性滑軌是全球工具機與自動化設備的重要供應來源，並延伸至諧波減速機、直驅馬達與醫療機器人；台達電子 (Delta) 從電源與變頻器技術出發，發展出完整的伺服驅動器與馬達產品線，並整合為 SCARA 機器人與整廠自動化方案。此外，和大工業、利苕機械等廠商也在減速齒輪領域深耕。

5.14.2 國產化的挑戰與機會

機器人三大關鍵零組件是減速機、伺服馬達與控制器，長期以來高階市場由日本與歐洲廠商主導（諧波減速機、RV 減速機、高階伺服系統）。台灣的機會在於：一、精密加工能力與供應鏈完整，具備切入減速機製造的基礎；二、電力電子與馬達驅動技術成熟，伺服驅動器已具國際競爭力；三、協作機器人與 AMR 等新興市場尚未被完全壟斷，是切入的窗口。

對學生的意義是：這些領域正是台灣產業最需要人才的地方。理解本章的扭力計算、慣量匹配與減速機原理，不只是為了考試，而是進入這個產業的入場券。

完整計算例題

例題 5-1 機械手臂關節的完整選型

題目：設計一支水平伸出的機械手臂關節。連桿長 400 mm、質量 0.6 kg（均勻分布）；末端負載含夾具共 1.5 kg。要求在 0.8 秒內從水平轉到垂直（90 度），採三角形速度曲線。安全係數取 1.8，減速機效率 0.9。試完成選型計算。

解（一）靜態扭力：以水平姿態為最壞情況。連桿重心在中點（0.2 m），負載在末端（0.4 m）。

$$\tau_{\text{static}} = (0.6 \times 9.8 \times 0.2) + (1.5 \times 9.8 \times 0.4) = 1.176 + 5.88 = 7.06 \text{ N}\cdot\text{m}$$

連桿自重重力矩加上負載力矩。

解（二）轉動慣量：連桿繞一端 $J = (1/3)mL^2$ ，負載視為質點 $J = mr^2$ 。

$$J = (1/3)(0.6)(0.4^2) + (1.5)(0.4^2) = 0.032 + 0.240 = 0.272 \text{ kg}\cdot\text{m}^2$$

連桿與負載的轉動慣量總和。

解（三）角加速度：轉 90 度即 $\theta = \pi/2 \approx 1.571 \text{ rad}$ ， $t = 0.8 \text{ s}$ ，三角形速度曲線 $\alpha = 4\theta/t^2$ 。

$$\alpha = 4 \times 1.571 / 0.8^2 = 6.284 / 0.64 \approx 9.82 \text{ rad/s}^2$$

所需角加速度。

解（四）動態扭力與總需求：

$$\tau_{\text{dynamic}} = J \cdot \alpha = 0.272 \times 9.82 \approx 2.67 \text{ N}\cdot\text{m}$$

克服慣性所需扭力。

$$\tau_{\text{required}} = (7.06 + 2.67) \times 1.8 = 9.73 \times 1.8 \approx 17.5 \text{ N}\cdot\text{m}$$

關節輸出端所需扭力（含安全係數）。

解（五）選減速比：關節平均角速度 = $1.571/0.8 \approx 1.96 \text{ rad/s} \approx 18.8 \text{ rpm}$ ；三角形曲線的峰值角速度是平均的兩倍，約 37.6 rpm。若選用額定轉速 3000 rpm 的馬達：

$$N \approx 3000 / 37.6 \approx 80 \rightarrow \text{取標準值 } N = 80$$

減速比選定。

解（六）反算馬達端扭力：

$$\tau_{\text{motor}} = 17.5 / (80 \times 0.9) \approx 0.243 \text{ N}\cdot\text{m} \approx 243 \text{ mN}\cdot\text{m}$$

馬達所需的額定扭力。

因此應選擇額定扭力大於 243 mN·m、額定轉速約 3000 rpm 的無刷馬達，搭配減速比 80 的減速機。請注意：這裡比較的是馬達的**額定扭力**，不是堵轉扭力。

解（七）驗算慣量比：假設所選馬達的轉子慣量 $J_{\text{motor}} = 3.0 \times 10^{-5} \text{ kg}\cdot\text{m}^2$ 。

$$J_{\text{reflected}} = 0.272 / 80^2 = 0.272 / 6400 \approx 4.25 \times 10^{-5} \text{ kg}\cdot\text{m}^2$$

折算到馬達端的負載慣量。

$$R = 4.25 \times 10^{-5} / 3.0 \times 10^{-5} \approx 1.42$$

慣量比落在建議範圍 1~3 內，匹配良好。

結論：本設計的慣量比為 1.42，屬於高響應伺服系統的理想範圍，選型成立。若慣量比算出來是 15，就應回到步驟五提高減速比（例如改用 $N = 120$ ，慣量比會降為約 0.63 倍，即約 $0.63 \times 15 \dots$ 實際為 $0.272/120^2/3e-5 \approx 0.63$ ），或改選轉子慣量較大的馬達。

例題 5-2 開頭故事的診斷

題目：回到本章開頭。學生使用標示「25 kg·cm」的 RC 伺服機，手臂連桿 30 cm、負載 500 g。（a）驗算他們的靜態扭力計算是否正確。（b）分析為何馬達仍然燒毀。（c）提出兩個

改善方案。

解 (a) : $25 \text{ kg}\cdot\text{cm}$ 換算為 SI 單位 : $25 \times 0.0098 \approx 0.245 \text{ kg}\cdot\text{m}\cdot\text{g}$... 正確算法為 $25 \text{ kgf}\cdot\text{cm} = 25 \times 9.8 \text{ N} \times 0.01 \text{ m} = 2.45 \text{ N}\cdot\text{m}$ 。學生計算的靜態需求 $\tau = 0.5 \times 9.8 \times 0.3 = 1.47 \text{ N}\cdot\text{m}$ 。表面上餘裕率 $= (2.45 - 1.47)/1.47 \approx 67\%$ ，計算本身無誤。

解 (b) : 問題出在三個未被計入的因素。第一， $2.45 \text{ N}\cdot\text{m}$ 是堵轉扭力，持續可用約為其 $1/3$ ，即約 $0.82 \text{ N}\cdot\text{m}$ ——已經低於靜態需求的 $1.47 \text{ N}\cdot\text{m}$ 。第二，未計連桿自重：若連桿 200 g 、長 30 cm ，另需 $0.2 \times 9.8 \times 0.15 \approx 0.29 \text{ N}\cdot\text{m}$ 。第三，未計動態扭力與定位時的抖動堵轉發熱。實際持續需求已遠超過馬達的持續能力，長時間運轉必然過熱燒毀。

解 (c) : 方案一，改用額定 (非堵轉) 扭力大於 $(1.47+0.29)\times 1.8 \approx 3.2 \text{ N}\cdot\text{m}$ 的伺服系統，或改用無刷馬達加減速機。方案二，改變機構——縮短力臂至 15 cm ，可使力矩需求減半；或加裝配重與彈簧來平衡重力 (重力補償，第 17 章)，大幅降低持續扭力需求。方案三 (附加)：加裝電流監控，在偵測到持續大電流時自動斷電保護。

程式範例：致動器選型計算器

以下 Python 程式實作 5.11 節的七步驟選型流程，可直接輸入設計參數，自動計算扭力需求、減速比、馬達端扭力與慣量比，並判斷匹配是否合格。程式最後以例題 5-1 的數據驗證。

```
import math

G = 9.8 # 重力加速度 m/s^2

def select_actuator(link_len, link_mass, load_mass,
                    angle_deg, time_s, sf=1.8,
                    gear_eff=0.9, motor_rpm=3000,
                    motor_inertia=3.0e-5, gear_ratio=None):
    """機器人關節致動器選型計算 (七步驟)"""

    # 步驟 2：靜態扭力 (水平姿態，最壞情況)
    t_static = (link_mass * G * link_len / 2) + (load_mass * G * link_len)

    # 步驟 3：轉動慣量與動態扭力
    J = (1/3) * link_mass * link_len**2 + load_mass * link_len**2
    theta = math.radians(angle_deg)
    alpha = 4 * theta / time_s**2 # 三角形速度曲線
    t_dynamic = J * alpha
```

```

# 步驟 4：總需求含安全係數
t_required = (t_static + t_dynamic) * sf

# 步驟 5：選減速比 (依峰值轉速)
w_avg_rpm = (theta / time_s) * 60 / (2 * math.pi)
w_peak_rpm = w_avg_rpm * 2          # 三角形曲線峰值為平均兩倍
if gear_ratio is None:
    gear_ratio = round(motor_rpm / w_peak_rpm / 10) * 10

# 步驟 6：反算馬達端扭力
t_motor = t_required / (gear_ratio * gear_eff)

# 步驟 7：慣量比驗算
j_reflected = J / gear_ratio**2
ratio = j_reflected / motor_inertia

if ratio < 1:      verdict = "偏低 (馬達慣量過大，反應遲鈍)"
elif ratio <= 3:  verdict = "理想 (高響應伺服系統建議範圍)"
elif ratio <= 10: verdict = "可接受 (一般工業應用)"
else:             verdict = "過高！易振盪，建議提高減速比"

print("=" * 48)
print(f"靜態扭力  $\tau_{static}$  = {t_static:8.3f} N·m")
print(f"轉動慣量 J = {J:8.4f} kg·m2")
print(f"角加速度  $\alpha$  = {alpha:8.2f} rad/s2")
print(f"動態扭力  $\tau_{dynamic}$  = {t_dynamic:8.3f} N·m")
print(f"需求扭力(含 SF={sf}) = {t_required:8.2f} N·m (關節輸出端)")
print("-" * 48)
print(f"峰值關節轉速 = {w_peak_rpm:8.1f} rpm")
print(f"建議減速比 N = {gear_ratio:8d}")
print(f"馬達端所需額定扭力 = {t_motor*1000:8.1f} mN·m")
print("-" * 48)
print(f"折算負載慣量 = {j_reflected:.3e} kg·m2")
print(f"慣量比 R = {ratio:8.2f} → {verdict}")
print("=" * 48)
return t_motor, gear_ratio, ratio

# 驗證：例題 5-1
select_actuator(link_len=0.4, link_mass=0.6, load_mass=1.5,
                angle_deg=90, time_s=0.8)

```

請試著修改參數觀察趨勢：(一) 把連桿長度加倍，需求扭力變成幾倍？(二) 把運動時間從 0.8 秒縮短到 0.4 秒，動態扭力如何變化？(三) 把減速比從 80 改成 20，慣量比會變成多少？這三個實驗會讓你對本章的三個平方關係產生直覺。

實作：馬達特性實測

本章的實作是量測一顆真實馬達的扭力—轉速曲線，親眼驗證 5.2 節的理論。所需器材：直流馬達（含編碼器或轉速計）、可調電源供應器、電流表、滑輪與細繩、砝碼組或彈簧秤。

1. 固定馬達，在輸出軸裝上已知半徑 r 的滑輪，繞上細繩並在末端掛砝碼。負載扭力 $\tau = m \cdot g \cdot r$ 。
2. 以固定電壓驅動馬達，由空載開始逐步增加砝碼重量，記錄每一組的：砝碼質量、轉速（rpm）、電流（A）。至少取八組數據，直到馬達停轉（堵轉）。
3. 將數據換算為扭力（N·m）與角速度（rad/s），繪製扭力—轉速圖，觀察是否呈線性關係。
4. 由圖上外插求出空載轉速與堵轉扭力，與型錄標示值比較，計算誤差百分比。
5. 計算每組數據的輸出功率 $P = \tau \cdot \omega$ 與輸入功率 $P_{in} = V \cdot I$ ，繪製功率曲線與效率曲線，找出最大功率點與最佳效率點，驗證兩者確實不重合。
6. 安全注意：堵轉狀態下電流極大且發熱迅速，每次堵轉測試不得超過三秒，並立即斷電冷卻。

延伸實驗：把同一顆馬達裝上減速比 N 的減速機，重複量測，驗證輸出扭力是否約為原來的 N 倍、轉速是否降為 $1/N$ ，並由兩者的乘積變化估算減速機的傳動效率。

國中小也看得懂：馬達就像腳踏車的變速器

你有沒有騎過有變速器的腳踏車？上坡的時候要換到「輕齒輪」，踩起來很輕鬆，但騎得慢；平路換到「重齒輪」，踩起來費力，但速度快。這就是機器人身上「減速機」在做的事情——用速度換力氣。

馬達其實跟腳踏車一樣：它轉得很快，但力氣很小。如果直接把馬達裝在機器人的手臂上，手臂會轉得超快卻舉不起任何東西。所以我們幫它裝上一組齒輪，把「轉很快、沒力氣」變成「轉很慢、力氣大」。減速機讓馬達轉一百圈，手臂才轉一圈——但力氣就變成大約一百倍！

還有一件重要的事：馬達會發熱。當你用手把機器人的手臂用力抓住不讓它動，馬達會拚命想轉卻轉不動，這時候它會非常燙，甚至燒壞。所以玩機器人的時候，千萬不要抓住正在動的手臂不放喔。如果馬達摸起來燙手，就要馬上關掉電源讓它休息。

叫獸提醒與常見錯誤

- 誤區一：把堵轉扭力當成可用扭力。型錄上漂亮的數字通常是瞬間極限值，持續可用約為其三分之一到二分之一。
- 誤區二：只算靜態扭力，忘記加速所需的動態扭力。快速運動的機構，動態扭力可能超過靜態扭力。
- 誤區三：計算力臂時用連桿長度而非重心距離。均勻連桿的重心在中點，力矩只有用全長計算的一半。
- 誤區四：忘記算連桿自身的重量。長而重的連桿，其自重力矩往往不亞於負載力矩。
- 誤區五：以為馬達越大越好。過大的轉子慣量會使系統反應遲鈍；慣量比應落在 1~10 之間。
- 誤區六：忽略減速比對慣量的平方效果。提高減速比是改善慣量比最有效的手段，效果是平方級的。
- 誤區七：驅動器只看持續電流。必須同時滿足持續電流、峰值電流與電壓範圍三項規格，並確保散熱條件與規格書一致。
- 誤區八：伺服機與微控制器共用電源。伺服機啟動瞬間的電流會拉低電壓造成重開機，必須獨立供電並共地。
- 誤區九：忽略反向驅動性。高減速比的關節無法被外力推動，撞到人時是硬碰硬——協作與腿式機器人必須慎選減速比。
- 誤區十：讓馬達長時間堵轉。這是燒毀馬達的第一元凶，務必加裝電流監控或熱保護。

課堂討論

1. 開頭故事中，若你是那兩位學生，在買馬達之前應該做哪三項計算？請依 5.11 節的流程說明。
2. 為什麼波士頓動力早期的機器人使用液壓，而近年的四足機器人多改用電動準直驅？請從功率密度、控制精度、維護與成本四方面分析。
3. 諧波減速機幾乎無法反向驅動，這對協作機器人的安全設計造成什麼挑戰？工程上有哪些補救方式（提示：力矩感測器、電流監控、第 20 章的柔順控制）？
4. 同樣要移動一個 5 公斤的物體 30 公分：方案 A 用滾珠螺桿、方案 B 用同步皮帶、方案 C

用鋼索。請分別分析三者速度、精度、成本與重量上的取捨。

5. 假設你要設計一支能與人握手的機械手臂。從致動器選型的角度，你會做哪些不同於工業手臂的決定？為什麼？

章末摘要

- 致動器分電動、液壓、氣壓三類：電動控制精準為主流、液壓功率密度最高用於重載、氣壓天生柔順用於夾爪與軟性機器人。
- 直流馬達的兩個核心關係：扭力與電流成正比 ($\tau = K_t \cdot I$)、反電動勢與轉速成正比；因此扭力與轉速呈線性反比。
- 最大功率點在堵轉扭力與空載轉速的各一半處，但此處效率僅約 50%；最佳效率點落在較低扭力區。
- 額定扭力（可持續）與堵轉扭力（瞬間極限）必須分清，後者通常是前者的二到三倍；混淆兩者是燒毀馬達的首要原因。
- 無刷馬達以電子換向取代電刷，壽命長效率高；準直驅（大扭力馬達 + 低減速比）具備反向驅動性，是腿式機器人的關鍵技術。
- 步進馬達以脈波開迴路定位，簡單但會失步；RC 伺服機整合度高，但需獨立供電且標示扭力為堵轉值。
- 扭力需求 = (靜態 + 動態) × 安全係數；靜態用重心距離計算、動態為 $J \cdot \alpha$ 、安全係數建議 1.5 ~ 2.0。
- 減速機用轉速換扭力：轉速降 $1/N$ 、扭力升 N 倍、而折算慣量降 $1/N^2$ （平方效果，是慣量匹配最有力的槓桿）。
- 諧波減速機零背隙高減速比適合工業手臂；行星減速機效率高可反驅適合腿式與 AMR；擺線減速機抗衝擊適合重載基座。
- 慣量比建議落在 1 ~ 3（高響應）或 1 ~ 10（一般工業）；過高易振盪，可提高減速比、換大慣量馬達或減輕負載改善。
- 驅動器選型須同時滿足持續電流（ $\geq$ 馬達額定 1.5 倍）、峰值電流與電壓範圍，並具備過流、飛輪二極體與散熱三項保護。
- 關節模組化整合馬達、減速機、雙編碼器、驅動器與煞車，是近年協作與四足機器人的主

流；雙編碼器是達成高絕對精度的關鍵。

觀念題與計算題

觀念題

1. 比較電動、液壓與氣壓三類致動器在功率密度、控制精度、維護與成本上的差異，各舉一個適用場合。
2. 說明直流馬達扭力與轉速呈反比的物理原因（提示：反電動勢）。
3. 在扭力—轉速曲線上標出：堵轉扭力、空載轉速、最大功率點、最佳效率點，並說明為何後兩者不重合。
4. 額定扭力與堵轉扭力有何差異？為什麼混淆兩者會導致馬達燒毀？
5. 無刷馬達相較有刷馬達有哪四項優點？代價是什麼？
6. 什麼是反向驅動性？它與減速比有何關係？為何對協作機器人與腿式機器人特別重要？
7. 步進馬達的失步是什麼？列出三種預防方法。
8. 說明減速機對轉速、扭力與慣量的三種影響，並指出哪一項是平方關係。
9. 比較諧波、行星與擺線減速機的減速比、背隙、效率與抗衝擊能力。
10. 什麼是背隙？為什麼位置回授無法完全消除背隙造成的誤差？
11. 解釋慣量比的意義與建議範圍，並列出慣量比過高時的三種改善方法。
12. 馬達驅動器必須具備哪三項保護？飛輪二極體的作用為何？

計算題

1. 某機械手臂連桿長 0.5 m、質量 1.2 kg（均勻分布），末端負載 2.0 kg，需在 1.0 秒內轉動 120 度（三角形速度曲線）。安全係數 1.8、減速機效率 0.88。（a）求靜態扭力。（b）求轉動慣量與動態扭力。（c）求關節端所需總扭力。（d）若選用額定轉速 4000 rpm 的馬達，建議減速比為何？（e）求馬達端所需額定扭力。
2. 承上題，若所選馬達的轉子慣量為 $5.0 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ 。（a）計算折算慣量與慣量比。（b）判斷是否合格。（c）若不合格，欲將慣量比降到 3 以下，減速比至少需提高到多少？
3. 一顆直流馬達的規格為：空載轉速 6000 rpm、堵轉扭力 0.6 N·m、額定電壓 24 V、堵轉電流 12 A。（a）求扭力常數 K_t （提示： $\tau = K_t \cdot I$ ，堵轉時電流最大）。（b）求最大輸出功率

及其發生的轉速與扭力。(c) 在最大功率點時，輸入功率為多少？效率為何？

4. 某滾珠螺桿導程 5 mm、效率 0.9，由額定扭力 0.5 N·m 的馬達直接驅動。(a) 求可產生的最大推力。(b) 若馬達以 1500 rpm 運轉，求螺帽的移動速度 (mm/s)。(c) 若欲將推力提升一倍而保持相同馬達，可採取哪兩種方法？請各計算所需的參數變更。

動手做與專題延伸

本章實作：馬達型錄判讀練習

上網或向廠商索取三份不同類型馬達的型錄 (建議：一份 RC 伺服機、一份無刷馬達、一份含減速機的伺服馬達)，完成下表。目的是學會在真實文件中找到正確的數字。

表 5-6 馬達型錄判讀表 (第一列為範例)

項目	馬達 A (範例：RC 伺服機)	馬達 B	馬達 C
型號	MG996R		
標示扭力與種類	11 kg·cm (堵轉 · 6V)		
推估可持續扭力	約 0.36 N·m (標示值 1/3)		
額定 / 空載轉速	約 60 rpm (0.17 s/60°)		
工作電壓 / 電流	4.8 ~ 6.6 V / 堵轉約 2.5 A		
轉子慣量	型錄未提供		
是否可反向驅動	否 (塑膠蝸桿減速)		
適用場合判斷	小型手臂關節、教學用途		

專題延伸

- 為附錄 D 的四伺服雙足機器人做完整的致動器驗算：量測連桿長度與質量、估算最壞姿態的靜態扭力、與所用伺服機的持續扭力比較，判斷是否有餘裕。若不足，提出改善方案。
- 設計一個「重力補償」機構：在機械手臂的關節上加裝彈簧或配重，使手臂在任何姿態下的重力力矩接近零。計算所需的彈簧常數，並比較加裝前後馬達的持續扭力需求。(這是第 17 章重力補償控制的機構版本)
- 進階：比較兩種四足機器人關節方案的優劣——方案 A：小馬達 + 諧波減速機 (減速比 100)；方案 B：大扭力外轉子馬達 + 行星減速機 (減速比 8)。請就扭力密度、反向驅動性、抗衝擊、成本與控制難度五個面向分析，並說明為何近年學界與業界多選擇方案 B (提

示：對照第 28 章的落地衝擊）。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 8 章：機構設計與致動器) 。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (致動器與傳動系統專章) 。
- Hughes, A. and Drury, B., Electric Motors and Drives: Fundamentals, Types and Applications, 5th ed., Newnes (馬達與驅動器的經典入門書) 。
- Kenjo, T. and Sugawara, A., Stepping Motors and Their Microprocessor Controls, Oxford University Press.
- Seok, S. et al., "Design Principles for Energy-Efficient Legged Locomotion," IEEE/ASME Trans. Mechatronics, 2015 (準直驅致動器設計原理) 。
- Wensing, P. M. et al., "Proprioceptive Actuator Design in the MIT Cheetah," IEEE Trans. Robotics, 2017 (低減速比高反驅性關節設計) 。
- Harmonic Drive 公司技術手冊：諧波減速機原理與選型指南。
- 各廠商選型工具與技術文件：上銀科技 (HIWIN) 、台達電子 (Delta) 滾珠螺桿、線性滑軌與伺服系統技術資料。
- Maxon Motor 與 Faulhaber 公司的馬達技術手冊 (含扭力—轉速曲線判讀與選型方法的完整說明) 。
- IEC 60034 系列標準：旋轉電機的額定值與性能規範。

第 6 章

感測器與機器人的感官

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一台服務型機器人正面朝向一扇玻璃門的照片，畫面上疊加三層視覺化資料：光達點雲（在玻璃處出現一片空洞，彷彿門不存在）、超音波錐形波束（正確偵測到玻璃）、以及相機影像（看見門把與反光）。標題文字：三種感官，三種真相——沒有一個感測器會告訴你它壞了。

叫獸開講 「閉著眼睛也能摸到鼻子，機器人做得到嗎？」

叫獸把一位學生的眼睛蒙上，帶他站到教室中央，然後說：「請你摸到自己的鼻子。」

學生輕鬆地做到了，手指毫不猶豫地落在鼻尖上。「再來，」叫獸說，「請你走到門口。」這次學生猶豫了，伸出雙手向前試探，走了三步就停下來：「我不知道走到哪了。」

「這就是機器人感測器的兩大分類。」叫獸解開他的眼罩，「你能閉著眼睛摸到鼻子，是因為你的肌肉與關節裡有本體感覺——大腦隨時知道每一節手臂彎了幾度，不需要用看的。這叫**內部感測**。但你走不到門口，因為門在哪裡是外部世界的資訊，沒有眼睛就沒有答案。這叫**外部感測**。」

接著他請三位學生上台，分別用三種方法測量教室後牆到講台的距離：第一位用捲尺、第二位用手機的雷射測距儀、第三位用步伐數乘以自己的步長。三個答案分別是 8.42 公尺、8.4 公尺、8.7 公尺。

「哪一個是對的？」叫獸問。學生們說當然是捲尺。「錯，」叫獸說，「捲尺也有誤差，只是比較小而已。真正的重點是——**你永遠不會知道真值**。你只會有一堆帶著不同誤差的量測值。工程師的工作不是找到真值，而是知道每個量測值錯多少、以及怎麼把它們合起來得到更好的估計。」

他在白板上寫下第三位學生的方法：「步伐計數其實就是機器人的**里程計**——用輪子轉了幾圈推算走了多遠。它有一個致命弱點：如果輪子打滑了，它完全不知道。走得越遠，錯得越多，而且錯誤永遠不會自己修正。」

「那雷射測距儀呢？」有人問。「它很準，但它只在有東西可以反射時才有用。對著玻璃、對著黑色絨布、對著鏡子，它會給你完全荒謬的數字——而且它不會告訴你『我這次量錯了』，它會理直氣壯地回報一個錯誤的數字。**沒有一個感測器會告訴你它壞了**，這是初學者最大的陷阱。」

下課前，叫獸留下一個問題：「如果你手上有一個會慢慢累積誤差但很平滑的感測器（里程計），和一個不累積誤差但偶爾會抽風的感測器（雷射），你要怎麼把它們合起來？答案在第 24 章。但在那之前，你得先認識它們每一個的脾氣——這就是今天這堂課要做的事。」

本章為什麼重要

第 4 章給了機器人骨骼、第 5 章給了肌肉，本章給它感官。感測器決定了機器人「能知道什麼」，而機器人永遠不可能對它無法感知的東西做出反應——這是一條無法用演算法繞過的物理界線。再聰明的 AI，接到錯誤的感測資料，也只會做出錯誤的決定。

本章要建立三個核心觀念。第一，每種感測器都有它的物理原理，而原理決定了它的極限與失效模式——超音波為什麼怕斜面、光達為什麼怕玻璃、IMU 為什麼會漂移，都能從原理推導出來。第二，感測器的規格不只是「準不準」，還包括解析度、精度、重複性、更新率與延遲，這五個指標常常互相衝突。第三，沒有完美的感測器，所以融合是必然——這將在第 24 章展開為完整的機率理論。

學習目標

- 能區分內部感測與外部感測，並說明兩者在控制與自主性中的不同角色。
- 能說明編碼器的工作原理，並計算其解析度與四倍頻後的效果。
- 能說明 IMU 三種元件的功能，並解釋加速度計與陀螺儀為何必須互補。
- 能比較超音波、紅外線、ToF 與光達四種測距原理的優缺點與失效情境。
- 能說明相機與深度相機的成像原理，並比較三種深度取得方式。
- 能說明力覺與觸覺感測的原理及其在柔順控制中的角色。
- 能區解析度、精度、重複性、準確度與更新率五個規格指標。
- 能分析感測器的雜訊來源，並說明濾波的基本策略與代價。
- 能說明校正的必要性，並執行一次簡單的感測器校正程序。
- 能為一個指定的機器人任務，選定合適的感測器組合並說明理由。

先備知識

第 3 章的感測器訊號鏈路（換能、調理、ADC、取樣、數值處理）與第 5 章的馬達控制概念。本章涉及基本的三角函數與簡單的統計量（平均值、標準差），若不熟悉可先閱讀附錄 D。機率的完整理論留待第 24 章。

核心名詞中英對照

表 6-1 本章核心名詞

中文	英文	說明
內部感測	Proprioception	量測機器人自身狀態的感知
外部感測	Exteroception	量測外部環境的感知
編碼器	Encoder	將旋轉角度轉換為電訊號的元件
增量式編碼器	Incremental Encoder	輸出相對變化量的編碼器
絕對式編碼器	Absolute Encoder	輸出絕對角度值的編碼器
慣性測量單元	Inertial Measurement Unit, IMU	整合加速度計與陀螺儀的感測模組
漂移	Drift	誤差隨時間累積的現象
飛行時間法	Time of Flight, ToF	以光或聲波往返時間測距的原理
光達	LiDAR	以雷射掃描量測環境距離的感測器
點雲	Point Cloud	由大量三維點構成的環境表示
解析度	Resolution	感測器能分辨的最小變化量
精度	Precision	重複量測結果的一致程度
準確度	Accuracy	量測值接近真值的程度
訊噪比	Signal-to-Noise Ratio, SNR	訊號強度與雜訊強度之比
校正	Calibration	以已知標準修正感測器輸出的程序
感測器融合	Sensor Fusion	整合多感測器資訊以獲得更佳估計

6.1 感測器的角色：內部感測與外部感測

6.1.1 兩種感知，兩種任務

如開頭故事所示，感測器分為兩大類，而這個分類不只是學術上的區分，它直接對應到機器人的兩種能力。

內部感測 (proprioception) 量測機器人自己的狀態：關節轉了幾度、輪子轉了幾圈、身體傾斜多少、馬達流過多少電流、電池剩多少電。它是**控制的基礎**——沒有編碼器，連 PID 都做不了，因為你不知道現在在哪、離目標多遠。

外部感測 (exteroception) 量測外部環境：前方多遠有障礙、目標物在哪、地面是什麼材質、有沒有人靠近。它是**自主的基礎**——沒有外部感測，機器人只能重播固定動作，無法應對環

境變化。

這解釋了一個常見的現象：傳統工業手臂可以只有編碼器（因為環境是固定的、工件永遠在同一個位置），而移動機器人必須有豐富的外部感測（因為環境未知且會變）。當工業手臂要進行「亂料夾取」時，它就必須加上相機——瞬間從內部感測的世界跨入外部感測的世界。

6.1.2 主動式與被動式

外部感測還有另一種分類方式。被動式感測器只接收環境中既有的能量：相機接收環境光、麥克風接收聲音。優點是不干擾環境、功耗低、不會與其他機器人互相干擾；缺點是完全依賴環境條件（相機在黑暗中失效）。

主動式感測器則主動發射能量再接收回波：超音波發射聲波、光達發射雷射、結構光相機投射紅外線圖案。優點是不受環境光影響、可在黑暗中工作、量測距離直接；缺點是耗電、可能互相干擾（兩台超音波機器人靠近時會收到對方的回波）、且會被特定材質欺騙（黑色吸光、玻璃透光、鏡面反射）。

圖 6-1 機器人感測器全景圖

建議配圖：以一台移動操作機器人為中心的分解圖。左側標註內部感測（關節編碼器、IMU、馬達電流、電池電壓、力矩感測器），右側標註外部感測（RGB 相機、深度相機、2D / 3D 光達、超音波、紅外線、麥克風、GNSS）。各感測器以引線指向安裝位置，並標註其量測的物理量與典型更新率。

6.2 編碼器與角度感測

6.2.1 增量式編碼器

編碼器是機器人最重要的感測器，沒有之一——每一個伺服關節都需要它。最常見的光學增量式編碼器原理很簡單：一片刻有等間隔透光縫隙的圓盤隨軸旋轉，一側是光源、另一側是光電接收器，圓盤轉動時光線被反覆遮斷，產生方波脈波。數脈波就知道轉了多少角度。

關鍵設計是兩相正交輸出（A 相與 B 相），兩者相位差 90 度。這帶來兩個好處：第一，可以判斷方向（若 A 領先 B 則正轉，反之則反轉）；第二，可以做四倍頻——A、B 兩相的上升緣與下降緣共有四個邊緣，因此解析度可提升四倍。

$$\text{角度解析度} = 360^\circ / (\text{PPR} \times 4)$$

PPR 為每轉脈波數 (Pulses Per Revolution)，×4 為正交四倍頻後的效果。

例如一顆 1000 PPR 的編碼器，四倍頻後每轉可分辨 4000 個位置，解析度為 0.09 度。若後

方再接減速比 50 的減速機，關節端的解析度更可達 0.0018 度——這正是工業手臂能達到高重複精度的原因。

6.2.2 增量式的弱點與絕對式編碼器

增量式編碼器只知道「變化了多少」，不知道「現在在哪」。因此每次開機都必須執行歸原點（homing）程序：讓關節慢慢移動直到觸發原點開關，才建立起絕對位置的基準。這在工業現場是每天開機的例行程序，但對人形機器人或協作機器人來說很不方便——想像一台機器人每次開機都要先揮舞手臂找原點。

絕對式編碼器則在圓盤上刻有格雷碼（Gray code）或以磁性、電容方式編碼，任一角度都對應唯一的數值，斷電後再開機立刻知道位置。單圈絕對式只知道一圈內的位置；多圈絕對式再加上機械或電子計圈器，可記錄轉了幾圈。代價是成本較高、體積較大。

如第 5 章 5.13.2 節所述，高階關節模組採用雙編碼器：馬達端用增量式（高解析度、供換相與速度控制），輸出端用絕對式（知道關節真實角度、消除減速機背隙的影響）。

表 6-2 常見角度感測方式的比較

方式	原理	解析度	斷電記憶	成本	典型應用
光學增量式編碼器	光遮斷計數	極高（可達數萬 PPR）	無（需歸原點）	中	伺服馬達、工業手臂
光學絕對式編碼器	格雷碼盤讀取	高	有	高	協作機器人關節輸出端
磁性編碼器	霍爾或磁阻感測磁場方向	中（12~14 位元）	有（單圈）	低~中	無刷馬達換相、關節模組
電位器	電阻分壓	低（受雜訊限制）	有	極低	RC 伺服機內部、教學
旋轉變壓器	電磁耦合	高（抗惡劣環境）	有	高	汽車、航太、高溫環境

◎ 編碼器的解析度不等於精度

一顆 4000 線的編碼器可以分辨 0.09 度，但這不代表它的量測誤差只有 0.09 度。安裝偏心、圓盤刻線誤差、軸承間隙都會造成系統性誤差。解析度是「能分辨多細」，精度是「量得多準」——前者容易靠數位技術提升，後者取決於機械製造品質。第 6.8 節會完整區分這組容易混淆的概念。

6.3 IMU：加速度計、陀螺儀與地磁計

6.3.1 三種元件，三種資訊

慣性測量單元 (IMU) 通常整合三種感測元件，各自量測不同的物理量。

加速度計 (accelerometer) 量測比力 (specific force)，也就是「加速度減去重力」。靜止時它讀到的其實是重力的反作用——這個特性很有用：靜止或等速時，加速度計指向的方向就是「下」，因此可以推算傾斜角。缺點是只要機器人在加速，重力與運動加速度就混在一起，無法分離；且對振動極為敏感（馬達的震動會讓讀值劇烈跳動）。

陀螺儀 (gyroscope) 量測角速度。它的優點是不受重力與線性加速度影響、反應極快、短時間內非常準確。但它的輸出是角速度，要得到角度必須積分——而任何微小的零點偏差積分之後都會變成隨時間線性增長的角度誤差，這就是**漂移**。一顆消費級 MEMS 陀螺儀靜置不動，數分鐘後累積的角度誤差可能就達到數度。

地磁計 (magnetometer) 量測地球磁場方向，提供絕對的方位角（航向）。它不會漂移，但極易受干擾——馬達、鐵製結構、電流導線、甚至是建築物的鋼筋，都會使讀值嚴重偏移。在室內環境中，地磁計往往不可靠。

6.3.2 互補：為什麼必須融合

這三者的特性恰好互補，這是感測器融合最經典的教科書案例。

加速度計：長期正確（重力方向永遠不變），短期雜訊大（受運動與振動干擾）。陀螺儀：短期精確（反應快、平滑），長期漂移（積分累積誤差）。因此最簡單的融合方式是**互補濾波**：用高通濾波取陀螺儀的短期變化、用低通濾波取加速度計的長期趨勢，兩者加權相加：

$$\theta = \alpha \cdot (\theta_{\text{prev}} + \omega_{\text{gyro}} \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

互補濾波。 α 通常取 $0.95 \sim 0.99$ ， Δt 為取樣週期。 α 越大越信任陀螺儀（平滑但可能漂移），越小越信任加速度計（不漂移但抖動）。

這條式子只有一行，卻是自平衡車、四軸飛行器與雙足機器人姿態估測的基礎。第 24 章的 Kalman 濾波器可視為它的最佳化版本——用機率理論自動決定最佳的權重 α ，而非人工調參。

值得注意的是：加速度計與陀螺儀能提供俯仰角 (pitch) 與滾轉角 (roll)，因為重力提供了絕對參考；但**偏航角 (yaw)** 沒有任何絕對參考——重力在水平旋轉時不變。因此偏航角只能

靠陀螺儀積分（會漂移），或加上地磁計、GNSS、視覺特徵來校正。這是所有六軸 IMU 的先天限制，也是為什麼室內機器人的方向估測特別困難。

6.4 力、力矩與觸覺感測

6.4.1 應變片與力矩感測器

力覺感測的主流原理是應變規（strain gauge）：金屬箔在受力變形時電阻會改變，把四個應變片接成惠斯登電橋，就能把微小的電阻變化轉成可量測的電壓。這個訊號極其微弱（毫伏等級），必須經過高增益放大與良好的雜訊屏蔽——這是第 3 章「訊號調理」環節最典型的應用場景。

六軸力 / 力矩感測器安裝在機器手臂的末端法蘭與夾具之間，可同時量測三個方向的力與三個方向的力矩。它是精密裝配（插銷入孔）、研磨拋光、以及碰撞偵測的關鍵元件。缺點是價格昂貴（往往數萬到數十萬元）、且會降低系統剛性（因為它本身必須有一點變形才能量測）。

6.4.2 不用感測器也能感覺力

有一個省錢又優雅的替代方案：用馬達電流估算力矩。回顧第 5 章的關係式 $\tau = K_t \cdot I$ ——電流與扭力成正比，因此只要量測馬達電流，就能推估關節的輸出力矩。若已知機器人的動力學模型（第 14 章），把重力與慣性所需的力矩扣掉，剩下的就是外力造成的力矩。

這個方法的優點是零額外成本（電流本來就要量測以做控制）、且不會降低剛性。缺點是精度較差，因為摩擦力（尤其是減速機的摩擦）會混在其中，且高減速比會讓外力的訊號被稀釋——這就回到第 5 章的反向驅動性：減速比越高，越感覺不到外力。這正是協作機器人採用中低減速比並搭配關節力矩感測器的原因。

6.4.3 觸覺感測

觸覺感測器提供接觸的分布資訊，而非單點的力。常見形式包括：電阻式壓力薄膜（便宜、可做成陣列）、電容式（靈敏度高、可偵測接近）、以及近年的視覺觸覺感測器——在透明彈性體內裝一顆相機，透過觀察表面變形的影像來推算接觸力的分布與物體表面紋理。這類感測器能提供極豐富的資訊（包括滑動偵測），是第 21 章靈巧抓取研究的熱門方向。

最簡單也最實用的觸覺感測，其實是微動開關與碰撞板——掃地機器人前方的保險桿就是如此。它只回答「有沒有撞到」，卻足以支撐整套避障行為，成本不到十元。這提醒我們：感測

器的選擇取決於任務需求，不是越精密越好。

6.5 超音波與紅外線測距

6.5.1 超音波：回音定位

超音波感測器發射一段約 40 kHz 的聲波脈衝，測量回波返回的時間，再由聲速換算距離：

$$d = (v_{\text{sound}} \times t) / 2$$

d 為距離， t 為往返時間， $v_{\text{sound}} \approx 343 \text{ m/s}$ (20°C)。除以 2 是因為聲波走了來回兩趟。

優點是便宜（數十元）、不受光線影響、對透明物體（玻璃）有效——這是它相對於光學感測器的獨特優勢。缺點有四個，都源自聲波的物理特性。

第一，波束角寬（約 $15 \sim 30$ 度）：它回報的是「這個錐形範圍內最近的東西」，無法知道確切方位，也無法分辨兩個相近的物體。第二，怕斜面：當目標表面與波束夾角過大時，聲波會被反射到別的方向而收不到回波，機器人會以為前方無障礙——這是最危險的失效模式。第三，怕吸音材質：窗簾、海綿、毛毯會吸收聲波。第四，更新率低：聲速慢，量測三公尺需要約 17.5 毫秒，且必須等回音散去才能發下一次，因此更新率通常低於 40 Hz。

此外，溫度會影響聲速（約每升高 1°C 增加 0.6 m/s ），高精度應用需做溫度補償。

6.5.2 紅外線測距：三角測距與 ToF

紅外線測距有兩種截然不同的原理，常被混淆。

三角測距式（如 Sharp GP2Y 系列）：發射器與接收器相隔一段距離，物體越近，反射光落在感光陣列上的位置越偏。它的輸出是非線性的類比電壓，需要查表或曲線擬合來換算距離。特性是反應快、成本低，但量測範圍有限（通常 $10 \sim 150$ 公分）、且受物體顏色與材質影響大——黑色物體反射弱，會被誤判為較遠。

飛行時間式（ToF，如 VL53L0X）：直接量測紅外光往返的時間（皮秒等級，靠特殊電路實現）。優點是量測結果與物體反射率關係較小、精度高（毫米級）、波束窄（可精確指向）、更新率高。缺點是強烈的環境光（尤其是陽光）會造成干擾、對透明與鏡面物體無效。近年 ToF 模組價格大幅下降，已逐漸取代三角測距式成為小型機器人的主流。

紅外線的共同弱點是：對黑色物體與玻璃無效。這正是掃地機器人常常撞到黑色沙發腳的原

因——實務上必須搭配碰撞板作為最後防線，這就是感測器互補的思想。

表 6-3 四種測距感測器的比較

感測器	原理	範圍	更新率	強項	失效情境	成本
超音波	聲波 ToF	2 cm ~ 4 m	低 (<40 Hz)	便宜、對玻璃有效	斜面、吸音材質、波束寬	極低
紅外線三角測距	光學三角	10 ~ 150 cm	高 (>100 Hz)	便宜、反應快	受顏色影響大、非線性	低
紅外線 ToF	光飛行時間	5 cm ~ 4 m	高 (>50 Hz)	精度高、波束窄	強光、透明與鏡面物體	低 ~ 中
2D 光達	雷射掃描 ToF	10 cm ~ 30 m	中 (5 ~ 20 Hz)	全周掃描、精度高、範圍大	玻璃、鏡面、強光、雨霧	中 ~ 高

6.6 光達與雷達

6.6.1 2D 光達：移動機器人的標配

光達 (LiDAR, Light Detection and Ranging) 以雷射測距搭配旋轉機構，在一個平面上掃描出數百到數千個距離點。一台典型的 2D 光達每秒旋轉 5 ~ 20 圈，每圈輸出 360 個以上的距離讀值，形成一圈環繞機器人的「距離輪廓」。

這正是掃地機器人與 AMR 能建圖與定位的關鍵。因為它一次提供整個平面的幾何資訊，可以做**掃描匹配**——把這一幀的輪廓與地圖比對，推算出機器人現在的位置。這是第 25 章 SLAM 的核心資料來源。

2D 光達的量測原理有兩種。飛行時間式 (ToF) 直接量測雷射脈衝往返時間，範圍大 (可達 30 公尺以上)、精度穩定，但成本較高。三角測距式 (如常見的低價家用型號) 則以相機觀測雷射光點的偏移，成本極低 (千元等級)，但範圍較短 (約 6 ~ 12 公尺) 且遠距精度下降明顯。

6.6.2 3D 光達與失效情境

3D 光達在垂直方向增加多線 (16、32、64、128 線) 或使用旋轉鏡面，可掃出完整的三維點雲。它是自動駕駛車與戶外機器人的主力感測器，但成本高、資料量大 (每秒數百萬點)，對運算能力要求高。近年的固態光達 (solid-state LiDAR) 以無機械旋轉件的設計降低成本並提升可靠度，正快速普及。

光達的失效情境必須牢記，因為它們在實驗室裡不常出現，卻在真實環境中致命。第一，**玻璃與鏡面**：雷射直接穿透玻璃或被鏡面反射到別處，光達會以為那裡是空的——這是服務型機器人撞上玻璃門的主因。第二，**強烈陽光**：戶外環境的紅外線背景會淹沒回波訊號。第三，**雨霧粉塵**：懸浮微粒會產生假回波，在點雲中形成不存在的障礙物。第四，**吸光材質**：黑色霧面物體反射率極低，可能量測不到。

實務上的因應方式是加裝互補感測器：用超音波偵測玻璃（聲波會被玻璃反射）、用碰撞板作為最後防線、用相機辨識材質。這再次呼應本章的核心思想——沒有任何單一感測器是完備的。

6.6.3 毫米波雷達

毫米波雷達發射無線電波（常見 24 GHz 或 77 GHz），量測回波的時間與頻率偏移。相較光達，它的最大優勢是**全天候**：雨、霧、雪、粉塵幾乎不影響它，且波長長，能穿透部分非金屬遮蔽。此外，它能直接利用都卜勒效應量測目標的**徑向速度**——這是光達做不到的，對偵測移動中的車輛與行人極為有用。

缺點是角解析度差（無法精細描繪物體輪廓）、對靜止的小物體不敏感、且金屬物體會產生強烈的多重反射（例如路邊的鐵欄杆可能被誤判為移動物體）。汽車產業的主流做法是雷達與相機融合：雷達提供可靠的距離與速度，相機提供分類與輪廓。

6.7 相機、深度相機與事件相機

6.7.1 相機：資訊最豐富的感測器

相機是所有感測器中資訊密度最高的：一張 640×480 的影像包含三十萬個像素，每個像素還有 RGB 三個通道。它能提供顏色、紋理、形狀、文字等其他感測器完全無法取得的資訊——只有相機能告訴你「那是一個紅色的可樂罐」而不只是「前方 50 公分有東西」。

代價是這些資訊**需要大量運算才能解讀**。光達給你的直接就是距離（可以立即使用），相機給你的是一堆數字，必須經過影像處理或深度學習才能變成有意義的資訊。這是第 22 章的主題。

相機的關鍵規格包括：解析度、幀率（fps）、視野角（FOV）、快門類型與動態範圍。其中兩個容易被初學者忽略但極為重要：

第一，**快門類型**。捲簾快門（rolling shutter）逐行曝光，成本低但在快速運動時會產生果

凍效應（直線變成斜線）；全域快門（global shutter）所有像素同時曝光，適合運動中的機器人與高速視覺，但價格較高。裝在移動機器人上的相機，強烈建議使用全域快門。

第二，**動態範圍**。機器人從室內走到窗邊時，畫面會瞬間過曝或過暗。人眼的動態範圍遠優於一般相機，這是視覺機器人在真實環境中最常遇到的挫折之一。

6.7.2 三種取得深度的方式

單一相機只能得到二維投影，深度資訊在成像過程中遺失了。取得深度有三種主流方法。

立體視覺（Stereo）：用兩顆已知間距（基線 B ）的相機，比對同一物體在兩張影像中的位置差（視差 d ），由三角關係算出深度：

$$Z = (f \times B) / d$$

Z 為深度， f 為焦距（像素）， B 為基線長度， d 為視差（像素）。深度誤差與距離平方成正比，故遠距離精度急遽下降。

優點是被動式（不發光、不干擾、不耗電、可在戶外用）。缺點是在**無紋理表面**（白牆、單色地板）上無法找到對應點，深度會出現大片空洞。

結構光（Structured Light）：主動投射已知的紅外線圖案（點陣或條紋），由圖案的變形推算深度。早期的 Kinect 即用此法。優點是在無紋理表面上也有效、近距離精度極高。缺點是強烈陽光會蓋過投射圖案，因此**僅適合室內**。

ToF 深度相機：對整個畫面同時量測光的飛行時間。優點是速度快、不受紋理影響、運算量低。缺點是解析度通常較低、多重反射會造成誤差（牆角處常見）、且多台設備間會互相干擾。

表 6-4 三種深度相機技術的比較

技術	原理	室內 / 室外	無紋理表面	精度特性	代表產品
立體視覺	雙目視差	皆可（被動式）	失效（無對應點）	誤差隨距離平方增加	RealSense D 系列、ZED
結構光	投射紅外圖案	僅室內	有效	近距離極準（毫米級）	Kinect v1、Astra
ToF	光飛行時間	室內為主	有效	全域一致，牆角有多重反射誤差	Kinect v2、Azure Kinect

6.7.3 事件相機：仿生的新選擇

事件相機 (event camera) 是一種仿生感測器，其原理與傳統相機截然不同。它不輸出完整的畫面幀，而是每個像素獨立監測亮度變化，只在變化超過門檻時輸出一個「事件」(包含像素座標、時間戳與變化極性)。

這帶來三個驚人的特性：極高的時間解析度 (微秒等級，遠超過每秒 30 幀的傳統相機)、極高的動態範圍 (可同時看清明亮的窗外與陰暗的室內)、以及極低的資料量與功耗 (靜止畫面幾乎不產生資料)。它非常適合高速運動的機器人 (如高速無人機避障) 與極端光照環境。缺點是需要全新的演算法 (傳統的影像處理方法完全不能用)、且無法直接取得靜態場景的外觀。目前仍以研究應用為主。

6.8 GNSS、地磁與無線電定位

6.8.1 GNSS 的原理與限制

全球衛星導航系統 (GNSS，包含美國 GPS、歐洲 Galileo、中國北斗、俄羅斯 GLONASS) 以三角定位原理運作：接收器量測來自多顆衛星的訊號傳播時間，由此推算距離，再解出自己的三維座標。理論上三顆衛星即可定位，但因接收器時鐘不夠精確，實務上至少需要四顆來同時解出時間誤差。

一般消費級 GNSS 的水平精度約 2~5 公尺，這對汽車導航足夠，但對機器人 (需要公分級精度來避開路緣) 遠遠不夠。提升精度的方法有兩種：

RTK (Real-Time Kinematic)：在已知精確位置的基準站與移動站之間比對載波相位，可將精度提升到公分級 (1~3 公分)。這是農業無人機自動導航與測量機器人的標準配備，也是第 27 章農用無人機案例的關鍵技術。缺點是需要基準站或訂閱網路 RTK 服務。

差分 GNSS (DGPS)：以基準站廣播誤差修正量，精度約 0.5~2 公尺，介於兩者之間。

GNSS 的根本限制是：**室內完全失效**、都市峽谷 (高樓之間) 因多重路徑反射而嚴重失準、樹冠下訊號衰減、且更新率低 (通常 1~10 Hz)。因此戶外機器人必定要把 GNSS 與 IMU、輪速計融合 (第 24 章)，才能在訊號短暫遺失時維持定位。

6.8.2 室內定位技術

室內無法使用 GNSS，替代方案包括：UWB (超寬頻) 標籤定位，以飛行時間測距，精度

可達 10~30 公分，但需佈建錨點；WiFi 或藍牙訊號強度指紋比對，精度約 2~5 公尺，成本低但不穩定；視覺標記（如 AprilTag、ArUco），在環境中貼上已知圖案，相機看到即可精確定位，成本極低且精度高，是實驗室與倉儲最實用的方案；以及最通用的——SLAM，用光達或相機自行建立地圖並定位，不需任何預先佈建（第 25 章）。

6.9 感測器的規格指標

6.9.1 五個必須分清的指標

讀感測器規格書時，有五個指標常被混淆，但它們的工程意義完全不同。用射箭來比喻最容易理解：

解析度 (Resolution)：能分辨的最小變化量。例如 12 位元 ADC 在 0~5 V 範圍下的解析度是 $5/4096 \approx 1.22 \text{ mV}$ 。解析度高不代表準——它只決定「刻度多細」。

精度 / 重複性 (Precision / Repeatability)：重複量測同一目標時，結果的一致程度。射箭時箭都集中在同一小區域，就是精度高（即使那個區域偏離靶心）。工業機器人型錄上漂亮的「重複精度 $\pm 0.02 \text{ mm}$ 」指的就是這個。

準確度 (Accuracy)：量測值接近真值的程度。箭落在靶心附近就是準確度高。準確度差但精度好的系統，可以透過**校正**來修正；但精度差（隨機分散）的系統無法靠校正拯救。

線性度 (Linearity)：輸出與輸入是否成正比。非線性的感測器（如紅外線三角測距）需要查表或多項式擬合來換算。

更新率與延遲 (Update Rate / Latency)：兩者不同！一顆相機可能每秒輸出 30 幀（更新率 30 Hz），但從曝光到資料抵達控制器可能耗時 100 毫秒（延遲）。在控制迴路中，延遲比更新率更致命——延遲會直接侵蝕系統的穩定裕度，這是第 16 章與第 19 章會嚴肅處理的問題。

圖 6-2 精度與準確度的靶心圖

建議配圖：四個靶心圖並排。(a) 高精度高準確度：箭群密集且在靶心；(b) 高精度低準確度：箭群密集但偏離靶心（可靠校正修正）；(c) 低精度高準確度：箭散布但平均在靶心；(d) 低精度低準確度：箭散布且偏離。下方註明：偏差可校正，隨機雜訊只能靠濾波與多次平均。

6.9.2 誤差的兩種類型

感測器誤差分為兩類，處理方式完全不同。**系統誤差 (bias)** 是固定或緩慢變化的偏移，例

如加速度計的零點偏移、編碼器的安裝偏心。它可以透過校正消除或補償——這是好消息。

隨機誤差 (noise) 則是每次量測都不同的隨機擾動，來自熱雜訊、電磁干擾、量化誤差。它無法被校正消除，只能透過濾波或多次平均來降低。統計上，取 N 次獨立量測的平均，可將隨機誤差的標準差降低為原來的 $1/\sqrt{N}$ ：

$$\sigma_{\text{mean}} = \sigma / \sqrt{N}$$

取 N 次平均可降低隨機誤差，但代價是延遲增加 N 倍。這是精度與即時性的根本權衡。

注意這個公式的殘酷之處：要把雜訊降到十分之一，需要一百次量測。對更新率 100 Hz 的感測器而言，這意味著一秒的延遲——在控制迴路中完全不可接受。這正是為什麼工程上需要更聰明的方法 (Kalman 濾波器)，它能在不犧牲即時性的前提下降低雜訊。

6.10 雜訊與濾波

6.10.1 雜訊從哪裡來

感測器讀值的雜訊有四個主要來源。**物理雜訊**：熱雜訊 (電子的隨機運動，溫度越高越嚴重)、散粒雜訊 (光子或電子到達的隨機性)，這是物理定律決定的下限，無法消除。**電子雜訊**：放大器本身的雜訊、電源的漣波、ADC 的量化誤差。**電磁干擾**：馬達的換相尖峰、PWM 的開關雜訊、無線通訊，這是機器人上最主要的雜訊來源，也是第 3 章強調「訊號線與功率線分開走」的原因。**機械振動**：馬達與減速機的振動會直接汙染 IMU 與力覺感測器的讀值。

重要的工程順序是：**先降低雜訊來源，再考慮濾波**。加裝隔離、改善接地、把訊號線遠離馬達線、在感測器旁加去耦電容，這些硬體措施的效果往往勝過任何軟體濾波，而且不會引入延遲。用軟體濾波去補救糟糕的佈線，是本末倒置。

6.10.2 三種基本濾波器

移動平均濾波 (Moving Average)：取最近 N 筆的平均。最簡單、雜訊抑制效果好，但延遲明顯 (約 $N/2$ 個取樣週期)，且對脈衝雜訊 (單一極端值) 抵抗力弱。

低通濾波 (一階 IIR)：以指數加權的方式融合新舊值：

$$y[k] = \alpha \cdot y[k-1] + (1 - \alpha) \cdot x[k]$$

α 為濾波係數 ($0 \sim 1$)。 α 越大越平滑但延遲越大。只需儲存一個變數，極省記憶體，是微控制器上最常用的濾波器。

中值濾波 (Median Filter)：取最近 N 筆的中位數。它的獨門優勢是能完全消除脈衝雜訊——當超音波偶爾回報一個荒謬的 0 或 400 公分時，中值濾波可以直接把它濾掉，而移動平均會被它拉偏。對測距感測器特別有效。

所有濾波器都遵守同一條鐵律：**平滑度與即時性不可兼得**。濾得越乾淨，延遲就越大；而延遲會侵蝕控制系統的穩定性。工程上的做法是找到剛好足夠的濾波強度，而不是追求最平滑的曲線。

表 6-5 三種基本濾波器的比較

濾波器	原理	記憶體需求	延遲	對脈衝雜訊	適用場合
移動平均	最近 N 筆平均	N 筆	約 $N/2$ 週期	抵抗力弱	一般類比訊號、電池電壓
一階低通 (IIR)	指數加權遞迴	1 筆	由 α 決定	抵抗力弱	微控制器、IMU、電流
中值濾波	最近 N 筆中位數	N 筆	約 $N/2$ 週期	完全消除	超音波、紅外線測距

6.11 校正

6.11.1 為什麼一定要校正

每一顆感測器出廠時都有個體差異：同一型號的兩顆加速度計，靜置時的讀值可能差好幾個百分點。校正就是用已知的參考標準，找出感測器輸出與真值之間的關係，並在程式中補償。

最基本的校正模型是線性的兩參數模型：

$$\text{真值} = \text{尺度係數} \times (\text{原始讀值} - \text{零點偏移})$$

尺度係數 (scale factor) 修正靈敏度誤差，零點偏移 (bias/offset) 修正零點漂移。

6.11.2 三種常見的校正程序

零點校正 (最簡單)：在已知為零的條件下量測，記錄下讀值作為偏移量。例如：把機器人靜置水平，記錄陀螺儀的三軸讀值（應為零，實際不為零），此值即為零點偏移，之後每次讀值都減掉它。這是 IMU 開機時必做的程序，而且每次開機都要重做，因為零點會隨溫度變化。

兩點校正：在兩個已知值下量測，解出尺度與偏移。例如力覺感測器：先空載 (0 kg) 記錄

一次，再掛上已知的 1 kg 砝碼記錄一次，兩點即可決定一條直線。

多點與非線性校正：對非線性感測器（如紅外線三角測距），需要在多個已知距離下量測，建立查表或擬合曲線（常用多項式或指數函數）。

更複雜的校正還包括：相機的內外參數校正（用棋盤格標定板，第 22 章）、IMU 的六面校正（將 IMU 依六個方向靜置，同時解出三軸的尺度與偏移）、以及機器人的手眼校正（求相機座標與機器人座標的關係，第 22 章）。

◎ 校正的黃金原則

一、校正必須在實際使用條件下進行（溫度、電壓、安裝方式都會影響）。二、校正參數要存起來並記錄日期——感測器會隨時間老化。三、要有驗證步驟：用第三個已知點檢查校正結果是否正確，不要只用校正用的那兩點自我驗證。四、若校正後誤差仍大，問題可能在安裝（歪斜、鬆動）而非感測器本身。

6.12 感測器配置策略

6.12.1 互補而非重複

為機器人選配感測器時，核心原則是**互補**：讓每個感測器的強項覆蓋其他感測器的弱項，而不是買一堆相同特性的感測器。

經典的互補組合有：光達（精確但怕玻璃）+ 超音波（不準但能偵測玻璃）+ 碰撞板（最後防線）；輪速里程計（平滑但會漂移）+ IMU（快速但漂移更快）+ 光達掃描匹配（不漂移但慢）；相機（能辨識類別但不知距離）+ 光達或雷達（知距離但不知類別）。

這正是自動駕駛車不惜成本裝上相機、光達與雷達三種感測器的原因——不是為了保險，而是因為它們的失效情境彼此不重疊：雨霧中光達失效但雷達有效、辨識號誌時雷達無用但相機有效。

6.12.2 由任務反推感測器

選配感測器的正確順序是由任務需求反推，而非由可用的零件出發。步驟為：一、列出機器人必須「知道」的事情（我在哪？前方有什麼？抓到東西了嗎？）；二、對每一項，列出需要的量測範圍、精度與更新率；三、找出能滿足的感測器候選；四、檢查各候選的失效情境，補上互補的感測器；五、確認運算資源與介面頻寬足夠（一台 3D 光達每秒數百萬點，微控制器無法處

理)。

表 6-6 常見任務的感測器配置建議

任務	必須知道	主要感測器	互補 / 備援	本書章節
關節位置控制	每個關節的角度	編碼器	限位開關 (歸原點)	第 16 章
移動機器人定位	自己在地圖上的位置	2D 光達 + 輪速計	IMU、視覺標記	第 25 章
室內避障	前方是否有障礙	光達或深度相機	超音波 (玻璃)、碰撞板	第 26 章
物件辨識與夾取	物體類別與 6D 位姿	RGB-D 相機	力覺感測 (接觸確認)	第 21 ~ 22 章
雙足 / 四足平衡	身體姿態與足端接觸	IMU + 關節編碼器	足底壓力感測	第 28 章
戶外導航	全域位置與航向	RTK-GNSS + IMU	輪速計、視覺里程計	第 27 章
人機協作安全	是否碰到人	關節力矩感測或電流	安全光柵、電容式接近感測	第 20、35 章

完整計算例題

例題 6-1 編碼器解析度與速度量測

題目：某差速輪機器人使用 500 PPR 增量式編碼器 (正交輸出)，直接裝在馬達軸上，馬達經減速比 30 的減速機驅動直徑 65 mm 的輪子。(a) 求輪子每轉一圈對應的編碼器計數。(b) 求機器人的位移解析度。(c) 若控制週期為 10 ms，機器人以 0.3 m/s 行進，每週期可計得幾個脈波？速度量測的量化誤差為何？

解 (a)：正交四倍頻後，馬達每轉為 $500 \times 4 = 2000$ 計數。經減速比 30，輪子轉一圈馬達需轉 30 圈：

$$\text{每輪轉計數} = 2000 \times 30 = 60000 \text{ counts/rev}$$

輪端的等效解析度。

解 (b)：輪子周長 = $\pi \times 0.065 \approx 0.2042 \text{ m}$ 。

$$\text{位移解析度} = 0.2042 / 60000 \approx 3.40 \times 10^{-6} \text{ m} \approx 3.4 \mu\text{m}$$

每一個計數對應約 3.4 微米的位移，解析度極高。

解 (c)：10 ms 內位移 = $0.3 \times 0.01 = 0.003 \text{ m} = 3 \text{ mm}$ 。計數 = $0.003 / 3.4 \times 10^{-6} \approx 882$

個脈波。量化誤差為 ± 1 個計數，對應速度誤差：

$$\Delta v = 3.4 \times 10^{-6} / 0.01 \approx 3.4 \times 10^{-4} \text{ m/s} = 0.34 \text{ mm/s}$$

相對誤差僅 $0.34/300 \approx 0.11\%$ ，速度量測品質良好。

工程對照：回顧第 3 章例題 3-2，該例的編碼器直接裝在輪軸上且線數低，量化誤差高達 31%。本例因為編碼器裝在減速機之前的馬達軸上，等效解析度被減速比放大了 30 倍——這是實務上編碼器一律安裝在馬達端的關鍵原因之一。

例題 6-2 超音波的失效分析

題目：某機器人以超音波感測器（波束角 30 度、量測範圍 2~400 cm）進行避障。（a）在距離 100 cm 處，波束覆蓋的寬度為何？（b）若前方有一面與波束軸夾角 40 度的斜牆，是否能偵測到？（c）機器人以 0.5 m/s 前進，感測器更新率 20 Hz，求兩次量測之間的移動距離，並評估避障是否安全。

解（a）：波束為圓錐，半角 15 度。在距離 $d = 100 \text{ cm}$ 處的覆蓋半徑：

$$r = d \times \tan(15^\circ) = 100 \times 0.268 \approx 26.8 \text{ cm} \rightarrow \text{覆蓋寬度約 } 53.6 \text{ cm}$$

這表示感測器無法分辨物體在此 53.6 cm 寬範圍內的確切方位。

解（b）：聲波遵守反射定律，入射角等於反射角。當牆面法線與波束軸夾角為 40 度時，反射波會偏離 80 度射出，遠超過感測器的接收範圍（約 ± 15 度）。因此無法收到回波，感測器會回報「無障礙」或超時值。一般而言，超音波在入射角超過約 20~30 度時即失效。這是超音波最危險的失效模式：它不會報錯，而是安靜地告訴你前方是空的。

解（c）：更新週期 $= 1/20 = 0.05 \text{ s}$ ，兩次量測間移動 $0.5 \times 0.05 = 0.025 \text{ m} = 2.5 \text{ cm}$ 。加上煞車距離與處理延遲，若系統總延遲為 150 ms，則反應期間已前進 7.5 cm。結論：對於低速機器人尚可接受，但必須設定足夠的安全距離（建議至少 30 cm 以上），且必須加裝碰撞板作為最後防線，以因應（b）的斜面失效情境。

例題 6-3 IMU 漂移與互補濾波

題目：某 MEMS 陀螺儀的零點偏移為 $0.05^\circ/\text{s}$ （已扣除靜態校正後的殘餘）。（a）若僅靠陀螺儀積分求角度，10 秒與 60 秒後的角度誤差各為何？（b）若加速度計的角度讀值雜訊標準差為 2 度，採互補濾波 $\alpha = 0.98$ 、取樣週期 10 ms，求濾波器的等效時間常數。（c）說明此設

計的物理意義。

解 (a)：積分後誤差隨時間線性增長：

$$\text{誤差} = 0.05^\circ/\text{s} \times t \rightarrow 10 \text{ s}: 0.5^\circ; 60 \text{ s}: 3.0^\circ$$

陀螺儀單獨使用時，誤差無界增長，長時間必然失效。

解 (b)：一階互補濾波的等效時間常數為：

$$\tau = (\alpha \times \Delta t) / (1 - \alpha) = (0.98 \times 0.01) / 0.02 = 0.49 \text{ s}$$

約 0.5 秒。

解 (c)：這代表濾波器在約 0.5 秒的時間尺度上做切換——短於 0.5 秒的快速變化交給陀螺儀（平滑、無雜訊），長於 0.5 秒的緩慢趨勢由加速度計主導（提供絕對重力參考、防止漂移）。因此陀螺儀的漂移只會累積約 0.5 秒（誤差僅 $0.05 \times 0.5 = 0.025$ 度），而加速度計的 2 度雜訊則被平均掉大部分。兩者的缺點都被有效抑制——這正是互補濾波的精髓，也是第 24 章 Kalman 濾波器的直覺基礎。

程式範例：濾波器比較與 IMU 姿態估測

以下 Python 程式模擬帶雜訊與脈衝干擾的測距訊號，比較三種濾波器的效果；接著實作互補濾波，示範如何由加速度計與陀螺儀融合出穩定的傾角。

```
import random
import math

# ----- 第一部分：三種濾波器比較 -----
class MovingAverage:
    def __init__(self, n=5):
        self.n, self.buf = n, []
    def update(self, x):
        self.buf.append(x)
        if len(self.buf) > self.n: self.buf.pop(0)
        return sum(self.buf) / len(self.buf)

class LowPass:
    def __init__(self, alpha=0.8):
        self.a, self.y = alpha, None
    def update(self, x):
        self.y = x if self.y is None else self.a*self.y + (1-self.a)*x
        return self.y

class Median:
    def __init__(self, n=5):
        self.n, self.buf = n, []
    def update(self, x):
```



```

        self.buf.append(x)
        if len(self.buf) > self.n: self.buf.pop(0)
        return sorted(self.buf)[len(self.buf)//2]

true_dist = 100.0                # 真實距離 100 cm
ma, lp, md = MovingAverage(5), LowPass(0.8), Median(5)

print("步  原始讀值  移動平均  低通  中值")
for k in range(12):
    noise = random.gauss(0, 2.0)          # 一般雜訊
    spike = 300.0 if k == 6 else 0.0      # 第 6 步注入脈衝干擾
    raw = true_dist + noise + spike
    print(f"{k:2d}  {raw:8.1f}  {ma.update(raw):8.1f}"
          f"  {lp.update(raw):7.1f}  {md.update(raw):6.1f}")
print("> 注意第 6 步的脈衝：中值濾波完全免疫，另兩者被拉偏\n")

# ----- 第二部分：互補濾波姿態估測 -----
def complementary_filter(gyro_rate, accel_angle, dt=0.01, alpha=0.98):
    """回傳一個可持續呼叫的濾波器"""
    state = {"angle": accel_angle[0]}
    out = []
    for w, a in zip(gyro_rate, accel_angle):
        # 陀螺儀積分 (短期精確) + 加速度計絕對參考 (長期不漂移)
        state["angle"] = alpha * (state["angle"] + w * dt) + (1-alpha) * a
        out.append(state["angle"])
    return out

# 模擬：真實傾角固定 10 度；陀螺儀有 0.05 deg/s 偏移；加速度計雜訊大
N, dt = 500, 0.01
true_angle = 10.0
gyro = [0.05 for _ in range(N)]          # 只有偏移，無真實旋轉
accel = [true_angle + random.gauss(0, 2.0) for _ in range(N)]
gyro_only = []
ang = true_angle
for w in gyro:
    ang += w * dt
    gyro_only.append(ang)
fused = complementary_filter(gyro, accel, dt, 0.98)

print(f"{'時間':>6} {'僅陀螺儀':>10} {'僅加速度計':>12} {'互補濾波':>10}")
for i in [0, 100, 250, 499]:
    print(f"{i*dt:5.1f}s {gyro_only[i]:10.2f} {accel[i]:12.2f} {fused[i]:10.2f}")
print(f"\n 真實角度 = {true_angle}°")
print("> 陀螺儀持續漂移、加速度計劇烈跳動、融合結果穩定且不漂移")

```

請試著修改：(一) 把 α 從 0.98 改成 0.5，觀察融合結果是否變得抖動；改成 0.999 呢？

(二) 把陀螺儀偏移加大到 0.5 °/s，看互補濾波是否仍能壓制漂移。這兩個實驗會讓你直觀理解 α 的物理意義。

實作：感測器特性實測與校正

本章的實作是實測一顆測距感測器的真實特性，並完成校正。所需器材：超音波模組（HC-SR04）或紅外線 ToF 模組、微控制器（Arduino 或 ESP32）、捲尺、以及不同材質的目標板（白紙板、黑色絨布、玻璃、金屬板、海綿）。

1. **線性度測試**：以捲尺量出 10、20、40、60、80、100、150、200 cm 的位置，在每個位置以白紙板為目標各量測 20 次，記錄平均值與標準差。繪製「實測值對真實距離」圖，觀察是否為直線。
2. **校正**：以線性回歸求出尺度係數與零點偏移，寫入程式進行補償。再取三個未用於校正的距離（如 35、75、130 cm）驗證校正效果。
3. **材質測試**：固定在 50 cm 處，分別以五種材質為目標各量測 20 次。記錄每種材質的平均值、標準差與失效次數（回報超時或荒謬值）。
4. **角度測試**：以白紙板置於 50 cm 處，將板面由垂直逐步旋轉 0°、10°、20°、30°、40°、50°，記錄每個角度的成功率。找出感測器失效的臨界角度，並與例題 6-2 的理論分析比較。
5. **雜訊與濾波**：在固定距離下連續取樣 200 筆，計算標準差；分別套用移動平均、低通與中值濾波，比較三者的標準差降低程度與響應延遲（可用突然移開目標板的方式量測延遲）。
6. **綜合報告**：整理出這顆感測器的「使用說明書」——可靠工作範圍、對哪些材質失效、臨界角度、建議的濾波設定與安全裕度。

這份報告的價值在於：你將擁有對這顆感測器的第一手認識，而不是型錄上的行銷數字。往後任何專題用到它，你都知道它會在哪裡背叛你。

國中小也看得懂：機器人的眼睛會騙人

閉上眼睛，用手摸摸自己的鼻子——很簡單對不對？可是閉著眼睛要走到教室門口，就很難了。這是因為你身上有兩種「感覺」：一種是知道自己身體的樣子（手彎了多少、腳站得穩不穩），另一種是知道外面有什麼（門在哪裡、前面有沒有桌子）。機器人也一樣，它需要這兩種感覺器官。

機器人用什麼當眼睛呢？有好幾種喔。**超音波**像蝙蝠一樣，發出人聽不到的聲音，等聲音撞到東西彈回來，算一算就知道多遠。但如果牆壁是斜的，聲音會彈到旁邊去，機器人就以為前面

沒東西——這就是為什麼掃地機器人有時候會撞到牆角。**雷射（光達）**用光來量，比較準，可是遇到玻璃就慘了，光直接穿過去，機器人以為那裡是空的，就一頭撞上玻璃門。**相機**看得到顏色和形狀，可以認出「那是一顆球」，但要用很多計算才看得懂。

最重要的一件事：**每個感測器都會有出錯的時候，而且它不會告訴你它錯了**。所以聰明的做法是裝好幾種不一樣的感測器，讓它們互相幫忙。就像掃地機器人，前面除了雷射，還會裝一片會被撞到的板子——萬一雷射沒看到，撞到板子還是會停下來。你在附錄 G 做的超音波避障車，也可以試試看加一個碰撞開關喔！

叫獸提醒與常見錯誤

- 誤區一：以為感測器會告訴你它出錯了。感測器失效時通常安靜地回報一個看似合理的錯誤數字，必須靠合理性檢查（範圍、變化率）主動偵測。
- 誤區二：混淆解析度與精度。解析度是刻度多細，精度是量得多準；4000 線編碼器不代表誤差只有 0.09 度。
- 誤區三：忽略延遲。更新率 30 Hz 不代表延遲只有 33 ms；在控制迴路中，延遲比更新率更致命。
- 誤區四：用軟體濾波補救糟糕的佈線。應先降低雜訊來源（隔離、接地、分離走線），濾波是最後手段且會引入延遲。
- 誤區五：對測距感測器使用移動平均。單一脈衝干擾就會拉偏平均值；測距類應優先使用中值濾波。
- 誤區六：IMU 開機不做零點校正。陀螺儀零點會隨溫度變化，每次開機都應在靜止狀態下重新量測偏移。
- 誤區七：期待六軸 IMU 提供穩定的偏航角。重力無法提供水平旋轉的參考，偏航角必然漂移，需靠地磁計、GNSS 或視覺校正。
- 誤區八：在室內信任地磁計。鋼筋、馬達與電流會嚴重扭曲磁場，室內的地磁讀值往往不可用。
- 誤區九：以為光達萬能。玻璃、鏡面、強光、雨霧粉塵都會使其失效，必須搭配互補感測器與碰撞防線。
- 誤區十：校正一次用到永遠。感測器會老化、零點會隨溫度漂移，應定期重新校正並記錄日

期。

課堂討論

1. 開頭故事中三位學生量出三個不同的距離。若要你設計一個程序來得到「最好的估計」，你會怎麼做？需要知道每個方法的什麼資訊？
2. 掃地機器人為什麼常常撞到黑色的沙發腳與玻璃門？請分別從紅外線與光達的物理原理解釋，並提出三種改善方案。
3. 自動駕駛車同時裝設相機、光達與毫米波雷達，成本高昂。若必須刪掉其中一種以降低成本，你會刪哪一種？在什麼情境下這個決定會出問題？
4. 事件相機的資料形式與傳統相機完全不同（只有變化事件、沒有完整畫面）。這對演算法設計帶來什麼挑戰？你認為它最適合用在哪些機器人任務上？
5. 有人主張「感測器越多越好」。請從成本、運算負擔、故障率、以及資料衝突（兩個感測器給出矛盾答案時怎麼辦）四個角度反駁或支持這個主張。

章末摘要

- 感測器分內部感測（自身狀態，是控制的基礎）與外部感測（環境資訊，是自主的基礎）；外部感測又分主動式與被動式。
- 編碼器是最重要的機器人感測器；正交輸出可判方向並四倍頻，解析度 = $360^\circ / (\text{PPR} \times 4)$ ；增量式需歸原點，絕對式有斷電記憶。
- 編碼器安裝在減速機之前的馬達端，可使等效解析度被減速比放大，是實務標準做法。
- IMU 三元件互補：加速度計長期正確短期雜訊大、陀螺儀短期精確長期漂移、地磁計不漂移但易受干擾。
- 互補濾波以一行公式融合加速度計與陀螺儀，是自平衡與姿態估測的基礎；六軸 IMU 的偏航角必然漂移，無絕對參考。
- 力覺可用應變片直接量測，或用馬達電流間接估算；高減速比會稀釋外力訊號，影響反向驅動與碰撞偵測能力。
- 超音波便宜且對玻璃有效，但波束寬、怕斜面（入射角超過約 $20 \sim 30$ 度即失效）、更新率低。

- 光達精確且提供全周幾何資訊，是 SLAM 的核心；但對玻璃、鏡面、強光與雨霧失效，必須有互補感測與碰撞防線。
- 深度取得有三法：立體視覺（被動、無紋理失效）、結構光（室內、無紋理有效）、ToF（快速、牆角有多重反射誤差）。
- 五個規格指標必須分清：解析度（刻度細）、精度（一致性）、準確度（接近真值）、線性度、更新率與延遲。
- 誤差分系統誤差（可校正）與隨機誤差（只能濾波或平均， σ 隨 $\sqrt{N}$ 下降，代價是延遲）。
- 應先降低雜訊來源再濾波；移動平均與低通易受脈衝干擾，中值濾波對測距感測器最有效。
- 感測器配置的核心原則是互補而非重複，並應由任務需求反推所需的感測能力。

觀念題與計算題

觀念題

1. 區分內部感測與外部感測，說明為何工業手臂可只有前者、移動機器人必須有後者。
2. 說明增量式編碼器的正交輸出如何達成方向判斷與四倍頻。
3. 比較增量式與絕對式編碼器，並說明高階關節模組為何採用雙編碼器。
4. 說明加速度計、陀螺儀與地磁計各自量測什麼、各自的優缺點為何。
5. 為什麼六軸 IMU 無法提供穩定的偏航角？有哪三種補救方式？
6. 說明用馬達電流估算外力的原理、優點與兩項限制。
7. 超音波為什麼怕斜面？請以反射定律說明，並估算失效的臨界角度。
8. 列舉光達的四種失效情境，並各提出一種互補的感測方案。
9. 比較立體視覺、結構光與 ToF 三種深度相機的原理與適用環境。
10. 說明捲簾快門與全域快門的差異，為何移動機器人建議使用後者？
11. 用射箭比喻說明解析度、精度與準確度的差異，並指出哪一種可用校正改善。
12. 為什麼「先降低雜訊來源、再濾波」是正確的工程順序？濾波的代價是什麼？
13. 說明中值濾波相對於移動平均的獨特優勢，以及它最適合的感測器類型。
14. 說明感測器配置的「互補原則」，並以自動駕駛車的三種感測器為例說明。

計算題

- 某機器手臂關節使用 2048 PPR 正交編碼器，裝於馬達端，經減速比 100 的諧波減速機。
(a) 求關節端的角度解析度 (度)。(b) 若手臂長 500 mm，求末端的位置解析度 (mm)。(c) 若改將編碼器裝在關節輸出端 (減速機之後)，解析度變為多少？由此說明馬達端安裝的優勢。
- 某 2D 光達每秒轉 10 圈，每圈輸出 360 個點，量測範圍 12 m。(a) 求角解析度 (度) 與資料率 (點 / 秒)。(b) 在 5 m 距離處，相鄰兩點的間距為多少公分？(c) 若要偵測寬 10 cm 的桌腳，在多遠的距離之外會因角解析度不足而可能漏檢 (至少需兩點打到目標)？
- 某立體視覺相機基線 $B = 6 \text{ cm}$ 、焦距 $f = 600$ 像素、視差量測精度 ± 0.5 像素。(a) 求在 1 m、3 m、5 m 處的深度值對應的視差。(b) 求三個距離處的深度誤差 (提示：對 $Z = fb/d$ 微分， $\Delta Z \approx Z^2 \cdot \Delta d / (fB)$)。(c) 由結果說明立體視覺為何不適合遠距離量測。
- 某陀螺儀零點偏移 $0.02^\circ/\text{s}$ ，加速度計角度雜訊標準差 1.5° 。(a) 僅用陀螺儀積分，多久後角度誤差達到 1 度？(b) 採互補濾波 $\alpha = 0.99$ 、 $\Delta t = 5 \text{ ms}$ ，求等效時間常數。(c) 在此時間常數內，陀螺儀累積的漂移為多少度？此設計是否合理？

動手做與專題延伸

本章實作：感測器規格調查表

選擇三種不同類型的感測器 (建議各選一種內部感測與兩種外部感測)，查閱其規格書並完成下表。目的是學會在真實文件中辨識關鍵指標，並看出型錄沒有明說的限制。

表 6-7 感測器規格調查表 (第一列為範例)

項目	感測器 A (範例：HC-SR04)	感測器 B	感測器 C
類型與原理	超音波測距 (聲波 ToF)		
量測範圍	2 ~ 400 cm		
解析度	約 0.3 cm		
精度 (規格書標示)	$\pm 3 \text{ mm}$ (理想條件)		
更新率 / 延遲	約 20 ~ 40 Hz / 約 25 ms		
工作電壓與電流	5 V / 約 15 mA		
通訊介面	觸發 / 回波脈波 (GPIO)		
已知失效情境	斜面 $> 25^\circ$ 、吸音材質、多機干擾		

項目	感測器 A (範例：HC-SR04)	感測器 B	感測器 C
建議互補感測器	碰撞板、紅外線 ToF		

專題延伸

- 為附錄 G 的超音波避障機器人加裝一顆紅外線 ToF 模組與一片碰撞板，設計一套「三層防線」的避障邏輯：光學遠距預警、超音波中距確認、碰撞板最後停止。實測三種材質（白牆、黑布、玻璃）下的避障成功率，比較加裝前後的差異。
- 使用手機的感測器記錄 App（多數手機可輸出 IMU 原始資料），把手機靜置十分鐘並記錄陀螺儀讀值。計算其零點偏移與雜訊標準差，並模擬積分後的漂移曲線，驗證本章 6.3.2 節的理論。
- 進階：為一台「圖書館還書機器人」設計完整的感測器配置。需求為：能在館內自主導航（有玻璃隔間與落地窗）、能偵測並避開行人、能辨識書架編號、能確認書本已被取走。請依 6.12.2 節的五步驟完成配置設計，列出感測器清單、預估成本、標出每個感測器的失效情境與互補方案，並繪製第 3 章格式的系統方塊圖。

參考文獻與延伸閱讀

- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer（感測與感知專章）。
- Everett, H. R., Sensors for Mobile Robots: Theory and Application, A K Peters（移動機器人感測器的經典專著）。
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press（感測器模型與不確定性的機率處理）。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer（視覺感測與影像處理）。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer（立體視覺與深度估測）。
- Groves, P. D., Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House（GNSS 與慣性導航融合）。
- Gallego, G. et al., "Event-based Vision: A Survey," IEEE TPAMI, 2022（事件相機的完整回

顧)。

- Yuan, W., Dong, S. and Adelson, E. H., "GelSight: High-Resolution Robot Tactile Sensors," Sensors, 2017 (視覺觸覺感測器) 。
- 各感測器廠商技術文件：Bosch Sensortec (IMU) 、SICK 與 Hokuyo (光達) 、Intel RealSense 與 Stereolabs (深度相機) 。
- ROS 2 感測器訊息標準 (sensor_msgs) 與驅動套件文件 (docs.ros.org) 。

第 7 章

機器人電子系統與嵌入式控制

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】比賽現場的雙格對照照片：左格是一台停在賽道上、LED 正在閃爍重開機的循跡車，旁邊放著萬用電表，螢幕顯示 3.8 V ；右格是同一台車經過改善後的內部特寫——馬達線與訊號線分開走並各自雙絞、電源入口加了一顆 $470\text{ }\mu\text{F}$ 電容、電表穩定顯示 5.02 V 。標題文字：一半的「程式問題」，其實是電在說謊。

叫獸開講 「機器人明明接好了電線，為什麼一動就重新開機？」

比賽前一天晚上十一點，實驗室的燈還亮著。小婕的循跡車在測試賽道上跑得漂亮極了——一直線加速、彎道修正、一圈只要二十八秒。她連跑五圈都完美無缺，決定收工回家。

隔天上午的正式比賽，第一圈剛過第二個彎道，車子突然停住不動，然後 LED 燈閃了一下，像是重新開機。裁判給了第二次機會，同樣的位置、同樣的症狀。第三次，車子在起跑線上就當掉了。

回到實驗室，小婕把程式從頭到尾檢查了三遍，一行問題也沒有。她甚至把程式燒回昨天測試成功的版本——還是會當。「老師，昨天明明可以跑，今天完全一樣的程式就是不行，這根本不合理！」

叫獸接過那台車，沒有看程式，而是先把萬用電表的探棒接到微控制器的電源腳位上，然後說：「你把車子舉起來，讓輪子空轉，看讀數。」電表顯示穩定的 5.02 伏特。「現在把車子放到地上，用手稍微壓住輪子，讓馬達出力。」

電表的數字瞬間跳到 3.8 伏特，然後彈回 4.6，又掉到 4.1——像心電圖一樣劇烈震盪。小婕愣住了。「這顆微控制器的最低工作電壓是 4.5 伏特，」叫獸說，「每次馬達用力，電壓就掉到門檻以下，它就重新開機。你的程式一行錯都沒有，是**電源**在說謊。」

「可是昨天為什麼可以跑？」「因為昨天你的電池是滿的。今天比賽前你只充了一半，內阻上升，同樣的電流造成的壓降就更大。而且昨天賽道是新的，摩擦力小；今天的賽道鋪了一層防滑漆，馬達要出更多力。」叫獸把電池、電線與馬達畫成一個迴路，「這三件事單獨看都不致命，加在一起就跨過了臨界點。」

他接著做了三件事：把馬達電源與控制器電源分開、在微控制器的電源腳旁加了一顆 470 微法的電容、把訊號線從馬達線旁邊移開重新綁紮。整個過程不到二十分鐘。車子再放上賽道，怎麼壓輪子都不再重開機。

「記住，」叫獸把那顆小小的電容放在小婕手心，「在機器人的世界裡，超過一半的『程式問題』其實是電的問題。而它們最狡猾的地方在於——症狀看起來永遠像是別的東西壞了。今天這堂課，我們就來學怎麼讓電乖乖聽話。」

本章為什麼重要

本章是第一篇的收尾。第 4 章給了骨骼、第 5 章給了肌肉、第 6 章給了感官，本章要處理的是把它們全部連起來的神經與血管——電子系統。這是機器人工程中最不浪漫、卻最常決定成敗的一環。

在教學現場，機器人故障原因的統計結果始終一致：電源與佈線問題佔了將近一半，遠超過演算法錯誤。更麻煩的是，這類問題的症狀極具欺騙性——程式當機、感測器讀值亂跳、通訊斷線、馬達失控，看起來全都像軟體問題。學會用電的角度診斷，能省下你未來無數個週末。

本章同時是第一篇通往第二篇的橋樑：讀完本章，你已經理解了一台機器人的完整實體構成；接下來的第 8 章開始，我們將進入數學的世界，學習如何精確描述它的運動。

學習目標

- 能比較微控制器與單板電腦的架構差異，並依任務需求選型。
- 能說明 GPIO、ADC、PWM 與計時器四種周邊的功能與典型用途。
- 能計算 ADC 的解析度與量測誤差，並說明取樣時的注意事項。
- 能說明 PWM 的原理，並計算頻率、占空比與等效電壓。
- 能比較 UART、I²C、SPI 與 CAN 四種介面的特性並正確選用。
- 能計算機器人的電流需求，並設計符合需求的電源架構。
- 能說明接地、共地與電磁干擾的成因，並提出至少五項抑制措施。
- 能說明中斷的機制與使用時機，並辨識中斷服務常式中的禁忌。
- 能設計符合安全規範的緊急停止電路。
- 能依系統性方法診斷機器人的電子故障。

先備知識

第 3 章的系統架構與上下位機分工、第 5 章的驅動器與 PWM 概念、第 6 章的感測器訊號鏈路。本章需要基本電學（歐姆定律、功率計算、串並聯），若不熟悉可先閱讀附錄 D.1。本章不需要微積分。

核心名詞中英對照

表 7-1 本章核心名詞

中文	英文	說明
微控制器	Microcontroller Unit, MCU	整合處理器與周邊介面的單晶片
通用輸入輸出	General Purpose I/O, GPIO	可程式設定為輸入或輸出的接腳
類比數位轉換器	Analog-to-Digital Converter, ADC	將電壓轉換為數值的元件
脈寬調變	Pulse Width Modulation, PWM	以脈波寬度比例控制等效功率的技術
占空比	Duty Cycle	PWM 週期中高電位所佔比例
中斷	Interrupt	由事件觸發、暫停主程式的即時處理機制
中斷服務常式	Interrupt Service Routine, ISR	中斷發生時執行的函式
上拉電阻	Pull-up Resistor	使接腳在無訊號時維持高電位的電阻
去耦電容	Decoupling Capacitor	抑制電源瞬間壓降的旁路電容
共地	Common Ground	各子系統的參考電位連接於同一點
接地迴路	Ground Loop	多條接地路徑形成迴路而引入雜訊
電磁干擾	Electromagnetic Interference, EMI	電磁能量對電路造成的干擾
光耦合器	Optocoupler	以光傳遞訊號並電氣隔離的元件
降壓轉換器	Buck Converter	將高電壓轉為較低電壓的開關式電源
緊急停止	Emergency Stop, E-Stop	以硬體迴路強制切斷動力的安全裝置
看門狗計時器	Watchdog Timer	程式當機時自動重啟系統的計時器

7.1 微控制器與單板電腦

7.1.1 兩種架構的根本差異

第 3 章已介紹過運算單元的三個層級，本節深入其硬體架構的差異。微控制器 (MCU) 與單板電腦 (SBC) 的根本不同，不在於運算速度，而在於**確定性**。

MCU 是「裸機」執行：程式從第一行跑到最後一行，沒有作業系統在背後排程。當你寫下「每 1 毫秒讀一次編碼器」，它就真的每 1 毫秒讀一次，誤差在微秒等級。它的記憶體是靜態配置的、周邊是直接映射到暫存器的、中斷延遲是可計算的。這種可預測性正是即時控制所需要的。

SBC 則跑著完整的作業系統（通常是 Linux），有虛擬記憶體、程序排程、快取機制與網路堆疊。它的運算能力可能是 MCU 的百倍，但你無法保證某一行程式一定在什麼時候執行——作

業系統隨時可能為了處理網路封包或磁碟 I/O 而打斷你的程式。這就是第 3 章所說「軟即時」的來源。

7.1.2 常見平台的比較

Arduino 系列 (AVR 或 STM32 核心)：生態系極為成熟、函式庫豐富、社群龐大，是教學與快速原型的首選。缺點是運算能力有限、且其抽象層 (如 `digitalWrite`) 效率較低，不適合高速控制。

STM32 系列：工業界最主流的 MCU 之一，型號齊全 (從幾十元的 F0 到內建 FPU 與 DSP 指令的 F4、H7)，周邊功能強大 (多組計時器、CAN、乙太網路)，是自製伺服驅動器與飛控的標準選擇。學習曲線較陡。

ESP32：內建 WiFi 與藍牙的雙核心 MCU，價格低廉且效能不錯，一顆核心可跑通訊、另一顆跑控制。是物聯網與教學型機器人的性價比之王，本書附錄多個單元採用此平台。

Raspberry Pi 5：完整的 Linux 電腦，可跑 Python、OpenCV 與 ROS 2，適合當上位機。注意它的 GPIO 沒有 ADC，且不適合做毫秒級控制迴路。

NVIDIA Jetson 系列：具備 GPU 的邊緣 AI 平台，能即時執行深度學習推論，是視覺機器人的主力，但功耗與成本較高。

表 7-2 常見開發平台的比較

平台	核心	時脈	ADC	即時性	適用角色	本書應用
Arduino Uno	AVR 8-bit	16 MHz	10-bit × 6	極佳	入門下位機	附錄 B ~ E
ESP32	Xtensa 雙核 32-bit	240 MHz	12-bit × 18	佳	下位機 / 輕量上位機	附錄 F ~ I
STM32F4	ARM Cortex-M4	168 MHz	12-bit × 16	極佳	伺服驅動、飛控	第 16 章
Raspberry Pi 5	ARM Cortex-A76 ×4	2.4 GHz	無 (需外接)	普通 (Linux)	上位機、ROS 2	第 29 章
Jetson Orin Nano	ARM ×6 + GPU	1.5 GHz	無 (需外接)	普通	視覺 AI 上位機	第 22、31 章

7.2 數位輸入輸出與 GPIO

7.2.1 數位訊號的電壓門檻

數位接腳只認識兩種狀態：高電位（邏輯 1）與低電位（邏輯 0）。但實體世界的電壓是連續的，因此晶片規格書會定義四個門檻： V_{IH} （判定為高的最低輸入電壓）、 V_{IL} （判定為低的最髙輸入電壓）、 V_{OH} （輸出高時的最低電壓）、 V_{OL} （輸出低時的最髙電壓）。介於 V_{IL} 與 V_{IH} 之間的電壓是**未定義區**，讀值可能隨機跳動。

這帶出機器人上一個極常見的問題：**電壓準位不相容**。5 V 系統（Arduino Uno、多數感測器模組）與 3.3 V 系統（ESP32、STM32、Raspberry Pi）混用時，若把 5 V 的訊號直接接到 3.3 V 的接腳上，輕則讀值錯誤、重則燒毀晶片。解法是使用位準轉換器（level shifter）、分壓電阻，或選用支援 5 V 容忍（5V tolerant）的接腳。

7.2.2 上拉、下拉與浮空

一個未連接任何東西的輸入接腳稱為**浮空**（floating），它的電位會隨環境電磁場隨機變化，讀值毫無意義。因此所有輸入接腳都必須有確定的預設電位。

上拉電阻（pull-up）把接腳經電阻接到電源，使其預設為高電位；按鈕按下時接地，變為低電位。這是最常見的接法，因為多數 MCU 內建可程式啟用的上拉電阻，不需外加零件。下拉電阻（pull-down）則相反，預設為低電位。

電阻值的選擇是一個權衡：太大（如 1 M Ω ）抗雜訊能力差、容易受干擾；太小（如 100 Ω ）則耗電且增加接腳負擔。實務上常用 4.7 k Ω 到 10 k Ω 。

7.2.3 開關彈跳與消除

機械開關在切換瞬間，金屬觸點會反覆彈跳數次，在數毫秒內產生多個假訊號。若程式直接計數，按一次可能被讀成三次。消除方式有兩種：硬體消抖（RC 濾波電路或施密特觸發器）與軟體消抖（偵測到變化後延遲 10~50 毫秒再確認，或要求連續數次讀值一致）。第 6 章提到的碰撞板、以及所有按鈕與限位開關，都必須做消抖處理。

◎ 限位開關的接線常識

安全相關的限位開關應採用**常閉接點**（NC）而非常開（NO）。原因是：若電線斷裂或接觸不良，常閉接點會呈現「觸發」狀態使機器停止（安全側失效），而常開接點會呈現「未觸發」，機器繼

續運轉直到撞毀。這個「失效導向安全」(fail-safe) 的設計原則，貫穿整個工業安全領域。

7.3 類比訊號與 ADC

7.3.1 解析度與量測誤差

類比數位轉換器把連續電壓轉成整數。一個 n 位元的 ADC 把參考電壓範圍分成 2^n 個階層，其解析度（每一階代表的電壓）為：

$$\text{LSB} = V_{\text{ref}} / 2^n$$

LSB (Least Significant Bit) 即最小可分辨的電壓變化。 V_{ref} 為參考電壓。

例如 Arduino Uno 的 10 位元 ADC 搭配 5 V 參考電壓： $\text{LSB} = 5/1024 \approx 4.88 \text{ mV}$ 。ESP32 的 12 位元 ADC 搭配 3.3 V： $\text{LSB} = 3.3/4096 \approx 0.81 \text{ mV}$ 。

此外還有**量化誤差**，其大小為 $\pm 0.5 \text{ LSB}$ 。這是數位化的本質限制，無法消除，只能靠提高位元數或縮小量測範圍來降低。

7.3.2 提升有效解析度的三個方法

第一，**縮小參考電壓**。若要量測的訊號只有 0~1 V，使用 5 V 參考等於浪費了 80% 的量測範圍。改用 1.1 V 或 2.5 V 的內部參考電壓，可大幅提升解析度。

第二，**訊號放大**。用運算放大器把微弱訊號放大到接近滿量程再進 ADC。這是力覺感測器（毫伏等級訊號）的必要處理。

第三，**過取樣與平均**。以高於所需的頻率取樣再平均，可提升有效位元數。理論上每過取樣 4 倍可增加 1 位元的有效解析度——但如第 6 章所述，這會增加延遲，且僅在訊號本身有適度雜訊時才有效（需要雜訊來「攪動」量化階層）。

7.3.3 取樣的三個陷阱

第一，**輸入阻抗**。ADC 內部有取樣電容需要充電，若訊號源的輸出阻抗太高（如高阻值的分壓電路），電容充不飽就會讀值偏低。解法是降低分壓電阻值，或加一級電壓隨耦器（voltage follower）。

第二，**混疊 (aliasing)**。如第 3 章所述，取樣頻率必須至少是訊號最高頻率的兩倍。若訊號中含有高於此的成分（例如馬達 PWM 的 20 kHz 雜訊），它會被折疊成低頻的假訊號，且無

法在數位端分辨。因此高精度量測必須在 ADC 之前加**類比抗混疊低通濾波器**——這是軟體濾波無法補救的。

第三，**共用參考電壓的污染**。多數 MCU 以電源電壓作為 ADC 參考。若電源因馬達啟動而下降 5%，所有的 ADC 讀值也會同步偏移 5%——這正是開頭故事中「感測器讀值亂跳」的另一種表現。精密量測應使用獨立的參考電壓 IC。

7.4 PWM 與馬達控制

7.4.1 原理與參數計算

PWM 以固定頻率的方波，透過改變高電位所佔的比例（占空比）來控制等效功率。對馬達這類具有電感（因而對電流變化有慣性）的負載，快速的開關會被自然平滑成穩定的等效電壓：

$$V_{\text{eff}} = V_{\text{supply}} \times D$$

D 為占空比 (0~1)。例如 12 V 電源、占空比 60%，馬達感受到約 7.2 V。

PWM 的解析度取決於計時器的位元數。以 8 位元計時器為例，占空比只有 256 階；若用於伺服馬達的角度控制（脈寬 1~2 ms 佔 20 ms 週期的 5~10%），實際可用的階層只剩約 13 階，角度解析度會粗到約 14 度——完全不堪用。因此伺服控制應使用 16 位元計時器，或使用專用的伺服驅動 IC。

7.4.2 頻率的選擇

PWM 頻率的選擇有三個考量。太低（低於約 2 kHz）會產生人耳可聽見的嘯叫，且馬達電流漣波大、發熱增加。太高則功率電晶體的開關損耗上升，驅動器發熱。此外還要考慮馬達的電氣時間常數——頻率必須高到讓電流在一個週期內不會有太大變化。

實務建議值：直流馬達 16~25 kHz（超出人耳聽覺上限）；無刷馬達 16~40 kHz；LED 調光 1 kHz 以上即可；RC 伺服機則固定為 50 Hz（這是協定規定，不可更改）。

7.4.3 從占空比到速度：一個常見誤解

初學者常以為「占空比 50% 就是最高速的一半」，這並不正確。馬達的轉速取決於等效電壓減去負載造成的壓降，而且存在**啟動死區**——占空比太低時，扭力不足以克服靜摩擦，馬達根本不轉。實測上，多數直流馬達要到 15~25% 的占空比才會開始轉動，而且左右兩顆馬達的死區還可能不同（這是循跡車直線走不直的常見原因）。

正確的做法有二：一是實測每顆馬達的死區並在程式中補償（把輸出映射到死區以上的範圍）；二是加裝編碼器做閉迴路速度控制，讓 PID 自動補償這些非線性——這正是第 16 章的主題。

7.5 通訊介面：UART、I²C、SPI 與 CAN

7.5.1 四種介面的原理

UART（非同步序列埠）：最簡單的點對點通訊，只需 TX、RX 兩條線。雙方必須事先約定相同的鮑率（如 115200 bps），因為沒有時脈線同步。優點是簡單、支援長距離（搭配 RS-485 可達千公尺）。缺點是只能一對一、且鮑率不匹配時會產生亂碼。

I²C：兩線（SDA 資料、SCL 時脈）匯流排，可掛接多達 127 個裝置，每個裝置有唯一位址。優點是接線極省，適合連接多顆感測器。缺點是速度較慢（標準 100 kHz、快速模式 400 kHz）、傳輸距離短（數十公分）、且必須外加上拉電阻（通常 4.7 k Ω ）——這是初學者最常忘記的一步，會導致完全無法通訊。

SPI：四線（MOSI、MISO、SCLK、CS）全雙工介面，速度可達數十 Mbps。適合高速裝置如 IMU、SD 卡、顯示器。缺點是每增加一個裝置就需要多一條 CS（片選）線，接腳消耗大。

CAN Bus：兩線差動匯流排，專為惡劣環境設計。它的三大特色是：差動訊號（抗雜訊能力極強）、多主架構（任何節點皆可主動發送）、以及基於訊息 ID 的優先權仲裁（多個節點同時發送時，ID 較小者自動勝出且不損失資料）。這使它成為汽車與工業機器人的標準，也是多馬達驅動器連接的首選。

表 7-3 四種通訊介面的比較

介面	線數	速度	距離	節點數	同步	典型用途
UART	2 (TX/RX)	9.6k ~ 1 Mbps	數公尺 (RS-485 可達 1 km)	2 (點對點)	非同步	上下位機通訊、除錯輸出
I ² C	2 (SDA/SCL)	100k ~ 1 Mbps	< 1 m	多 (位址定址)	同步	感測器、EEPROM、擴充板
SPI	4+ (含 CS)	1 ~ 50 Mbps	< 30 cm	多 (片選)	同步	IMU、SD 卡、顯示器、ADC
CAN	2 (CANH/CANL)	125k ~ 1 Mbps	40 m ~ 1 km (依速率)	多 (廣播 + 仲裁)	非同步差	馬達驅動器、車輛、工業

介面	線數	速度	距離	節點數	同步	典型用途
					動	

7.5.2 選擇的判準與實務要點

選擇的順序是：先問距離（超過一公尺就排除 I²C 與 SPI）、再問節點數（多節點選 I²C 或 CAN）、再問速度（高速選 SPI 或 CAN）、最後問環境雜訊（馬達旁邊選 CAN 的差動訊號）。

三個實務要點。第一，I²C 忘記接上拉電阻是最常見的除錯陷阱，且部分模組已內建上拉，多個模組並聯時上拉電阻會過小而導致通訊失敗。第二，CAN 匯流排兩端必須各接一顆 120 Ω 終端電阻，否則訊號反射會造成通訊錯誤。第三，長距離的 UART 應改用 RS-485（差動訊號），而非直接拉長 TTL 電平的線。

7.6 電源系統設計

7.6.1 為什麼馬達會讓微控制器重開機

回到開頭故事。要理解那個現象，必須認識一個常被忽略的事實：**電線與電池都有內阻**。當馬達啟動時瞬間抽取大電流 I ，這個電流流過電池內阻 R_{int} 與導線電阻 R_{wire} ，就會產生壓降：

$$V_{drop} = I \times (R_{int} + R_{wire})$$

電流越大、內阻越高，壓降越嚴重。這個壓降發生在電源本身，因此所有共用此電源的元件都會受害。

舉例：一顆老化的鋰電池內阻 0.15 Ω，加上細導線與接頭的 0.1 Ω，總計 0.25 Ω。當兩顆馬達同時啟動抽取 6 A 時，壓降達 $6 \times 0.25 = 1.5$ V。原本 6 V 的電源瞬間只剩 4.5 V——正好跨過微控制器的工作門檻。這就是開頭故事的完整解釋，也說明了為什麼「昨天可以、今天不行」：電池充電狀態與地面摩擦力的變化，剛好把系統推過了臨界點。

7.6.2 電源架構的四個原則

原則一：功率與訊號分離。馬達、加熱器等大電流負載應由獨立的線路供電，不與控制器和感測器共用。若電池只有一顆，則在電池端就分成兩條路徑（各自有自己的開關與保護），而不是共用一條線再分岔。

原則二：單點共地。所有子系統的地線必須連接（否則訊號沒有共同參考電位，通訊會失敗），但應該匯集於一個點（通常是電池負極或電源分配板）。若各處隨意互連，會形成接地迴

路 (ground loop) ——地線本身的電阻使不同位置的「地」電位不同，馬達電流流過地線時會在其上產生電壓，這個電壓會直接疊加到感測器訊號上。

原則三：就近去耦。在每顆晶片的電源腳與地之間，跨接一顆 $0.1\ \mu\text{F}$ 的陶瓷電容 (高頻旁路)，並在電源入口處加一顆 $100 \sim 1000\ \mu\text{F}$ 的電解電容 (提供瞬間電流的儲能池)。這是抑制電源雜訊最便宜、最有效的手段——開頭故事中叫獸加的那顆電容就是後者。

原則四：穩壓與保護。電池電壓會隨放電下降 (鋰電從 $4.2\ \text{V}$ 掉到 $3.0\ \text{V}$)，需經降壓 (Buck) 或升壓 (Boost) 模組穩壓後供給邏輯電路。同時應具備保險絲 (過流)、逆接保護二極體、以及低電壓切斷 (過放保護)。

圖 7-1 機器人電源分配架構圖

建議配圖：由電池出發，經主開關與保險絲後分為兩路。上路 (訊號電源，細線)：經降壓模組至 $5\ \text{V}/3.3\ \text{V}$ ，供給 MCU、感測器與通訊模組，各晶片旁標註 $0.1\ \mu\text{F}$ 去耦電容。下路 (功率電源，粗線)：經急停開關與大電容至馬達驅動器與馬達。兩路的地線最終匯於電池負極的單一星形接地點。圖中並標出量測壓降的建議測點位置。

7.6.3 電流預算

設計電源前應先做電流預算表：列出每個元件的**靜態電流** (正常工作) 與**峰值電流** (啟動或堵轉)，分別加總。電源供應能力必須滿足峰值總和，導線線徑則依持續電流選擇。

一個常見的疏漏是：只加總靜態電流。實務上馬達堵轉電流可達額定的五到十倍 (第 5 章)，若電源按靜態值設計，第一次撞牆就會全系統斷電。建議做法是：電源容量取「所有元件靜態電流總和 + 最大單一負載的峰值電流」，並保留 30% 餘裕。

7.7 接地、雜訊與電磁干擾

7.7.1 干擾的三條路徑

電磁干擾 (EMI) 從干擾源傳到受害電路，有三條路徑，各有不同的抑制方法。

傳導耦合：透過共用的電源線或地線傳遞。這是機器人上最主要的路徑——馬達的電流突波經電源線傳遍全機。抑制方法：電源分離、去耦電容、LC 濾波器、以及必要時使用獨立電池。

電容耦合：兩條靠近的導線之間存在寄生電容，高頻訊號會透過它「跳」過去。這是訊號線與馬達線網在一起時的元兇。抑制方法：拉開距離 (有效距離與間距成反比)、垂直交叉而非平行行走線、加接地隔離層。

電感耦合：變化的電流產生磁場，在鄰近的迴路中感應出電壓。迴路面積越大，感應越強。

抑制方法：**雙絞線**（把去線與回線絞在一起，使兩者的感應電壓互相抵消）、減小迴路面積、使用差動訊號（如 CAN、RS-485）。

7.7.2 七項實務抑制措施

一、訊號線與功率線分開走，若必須交叉則採垂直交叉。二、所有訊號採雙絞或使用屏蔽線，屏蔽層僅在一端接地（兩端接地會形成接地迴路）。三、在馬達端子上並聯 $0.1\ \mu\text{F}$ 陶瓷電容以吸收換相火花。四、縮短所有訊號線長度，尤其是類比訊號與高速數位訊號。五、以光耦合器或數位隔離器隔離不同電壓域（特別是編碼器與驅動器之間）。六、大電流迴路的去線與回線平行緊靠，減小迴路面積。七、在感測器端而非 MCU 端做訊號放大，使長線上傳遞的是強訊號。

◎ 一個免費的雜訊抑制技巧

把馬達的兩條電源線互相絞緊（雙絞），是完全免費卻極其有效的做法。原理是：去線與回線的電流方向相反，緊靠絞合時它們產生的磁場互相抵消，對外輻射大幅降低。同理，把感測器的訊號線與其地線絞在一起，可顯著提升抗干擾能力。這個技巧在自製機器人上的效果，往往比昂貴的屏蔽線更明顯——因為多數學生的屏蔽線根本沒有正確接地。

7.7.3 隔離：最徹底的解法

當干擾嚴重到無法用上述方法解決時，最終手段是**電氣隔離**：讓兩個電路之間沒有任何導體連接，訊號透過光（光耦合器）、磁（隔離變壓器）或電容耦合傳遞。這樣一來，干擾就無法從一側傳到另一側。

工業機器人上，編碼器訊號、急停迴路、以及與 PLC 的介面通常都是隔離的。代價是成本增加、速度受限（一般光耦合器的頻寬有限，高速訊號需用高速型號）、且隔離側需要獨立的電源。

7.8 中斷與即時程式設計

7.8.1 輪詢與中斷

MCU 要偵測事件（按鈕按下、編碼器脈波、通訊資料到達）有兩種方式。**輪詢（polling）**是在主迴圈中反覆檢查狀態，優點是簡單、時序可預測；缺點是浪費運算資源，且若主迴圈中有耗時的運算，就可能錯過短暫的事件。

中斷 (interrupt) 則由硬體在事件發生時主動打斷主程式，跳去執行中斷服務常式 (ISR)，處理完再返回原處。優點是反應極快 (微秒等級)、不會漏掉事件、且主程式可以專心做其他事。這是編碼器計數、精確計時與高速通訊的唯一正解——一顆 1000 線編碼器在高速旋轉時每秒可產生數萬個脈波，用輪詢必定漏計。

7.8.2 中斷服務常式的三大禁忌

ISR 執行期間，其他中斷通常被暫時阻擋，因此它必須**極短**。三個絕對禁忌是：

禁忌一：在 ISR 中使用延遲函式 (如 delay)。這會癱瘓整個系統，因為計時本身也可能依賴中斷。

禁忌二：在 ISR 中做序列輸出 (如 Serial.print)。序列傳輸極慢 (傳送一個字元在 115200 bps 下需約 87 微秒)，且其本身依賴中斷，容易造成死結。除錯時應在 ISR 中設定旗標，由主迴圈負責輸出。

禁忌三：在 ISR 中做複雜運算 (浮點數學、迴圈、動態記憶體配置)。ISR 應只做最必要的事：讀取數值、更新計數器、設定旗標，其餘交給主迴圈。

此外還有一個常見錯誤：ISR 與主程式共用的變數必須宣告為 **volatile**，否則編譯器可能做最佳化而快取舊值，導致主程式讀不到 ISR 的更新。對於多位元組的變數 (如 32 位元的計數器)，主程式讀取時還應暫時關閉中斷，避免讀到「讀了一半就被更新」的破碎數值。

7.8.3 計時器與看門狗

硬體計時器是 MCU 中最有用的周邊之一，能在完全不佔用 CPU 的情況下產生精確的時間基準。控制迴路應由計時器中斷觸發 (例如每 1 毫秒觸發一次)，而非在主迴圈中用延遲函式湊時間——後者的週期會隨主迴圈中其他程式的執行時間而變動，破壞控制系統的假設。

看門狗計時器 (watchdog) 是安全設計的重要一環：它是一個持續倒數的計時器，程式必須定期「餵狗」(重置計數)。若程式當機或陷入無窮迴圈而沒有餵狗，計時器歸零後會自動重啟系統。使用要點是：餵狗的動作應放在確認系統健康的位置 (例如控制迴路完成一輪之後)，而不是隨便放在主迴圈開頭——否則即使控制邏輯已經卡死，看門狗仍會被餵飽而失去意義。

7.9 安全電路與緊急停止

7.9.1 為什麼急停必須是硬體

第 3 章已強調：急停必須是硬體迴路，直接切斷動力，不經過任何軟體或通訊協定。理由很簡單——急停存在的目的，正是為了應對「軟體已經失控」的情況。若急停按鈕只是接到 MCU 的一個 GPIO，然後由程式決定要不要停止馬達，那麼當程式當機時，急停按鈕就完全失效了。

正確的做法是：急停開關（採常閉接點）串聯在功率電源的路徑上，或用來切斷驅動器的致能訊號與繼電器線圈。按下時，物理上切斷馬達的動力來源。同時可另接一路訊號給 MCU，讓軟體知道急停被觸發並做出對應處置（如記錄狀態、通知上位機），但這只是輔助，不是主要機制。

圖 7-2 緊急停止電路示意圖

建議配圖：電池 → 保險絲 → 急停開關（常閉接點，紅色蘑菇頭符號）→ 主接觸器 / 繼電器 → 馬達驅動器 → 馬達。另由急停開關並接一條細線（經上拉電阻）至 MCU 的 GPIO 作狀態偵測。圖中標註：粗線為功率路徑（急停直接切斷）、細線為訊號路徑（僅供軟體知情）。並註明常閉接點的失效導向安全特性。

7.9.2 安全設計的四個層次

第一層：本質安全。從設計上就限制危險——降低馬達功率、限制運動速度、加裝機械限位擋塊、使用圓角避免夾傷。這是最可靠的一層，因為它不依賴任何主動元件。

第二層：硬體保護。急停開關、限位開關（常閉）、保險絲、過流保護、安全繼電器。這些元件不依賴軟體運作。

第三層：軟體監控。程式檢查電流、溫度、位置是否超限，並在異常時停止。同時包含看門狗與通訊超時保護（第 3 章：上位機斷線時下位機必須自動停止）。

第四層：操作規範。測試時墊高機器人使輪子懸空、手不進入運動範圍、通電前先確認佈線、實驗室備有滅火器。這一層常被學生忽略，卻是教學現場最重要的一層。

7.9.3 學生實驗室的安全清單

在自製機器人上，最低限度應具備：一、易於按下的急停開關，位置明顯且不需繞到機器背後。二、電源總開關與保險絲。三、鋰電池使用專用充電器與平衡充，充電時不離人、置於防火袋中。四、測試新程式時，機器人必須架高使輪子離地。五、馬達或電池若發熱異常（無法用手持續接觸），立即斷電。六、機器手臂測試時，所有人退出其工作半徑之外。

7.10 配線、接頭與機構整合

7.10.1 線束設計的六個要點

配線是機器人可靠度的隱形基礎——回顧第 3 章開頭那台因電線磨破而報廢的送藥機器人，這一節就是那個教訓的正面教材。

一、**線徑依電流選擇**。導線過細會發熱、壓降增大。實務參考：0.5 mm² (約 20 AWG) 可承載約 3 A，1.0 mm² (18 AWG) 約 6 A，2.0 mm² (14 AWG) 約 12 A。訊號線可用細線，但馬達線務必足夠粗——這也是降低 7.6.1 節壓降的直接手段。

二、**保留活動餘量**。經過關節的線束必須有足夠長度，並以「U 形迴繞」或螺旋方式配置，讓關節轉動時線束是彎曲而非被拉扯。切勿拉直後剛好夠長。

三、**固定與防磨**。線束每隔約 10 公分應以束帶或線夾固定；經過金屬邊角處必須加套管 (如蛇管、編織網管) 或在孔洞裝上護線環。這是最便宜的可靠度投資。

四、**預留維修長度**。接頭處保留一小段餘線，讓未來重新壓接時不必更換整條線。

五、**標示**。每條線兩端貼上標籤 (來源—去處—功能)。三個月後的你會非常感謝現在的自己。

六、**應力消除**。線材與接頭連接處是最脆弱的點，應以熱縮套管或束帶做應力消除，避免反覆彎折造成斷芯——這種斷裂常發生在絕緣皮內部，外觀完全正常卻時通時斷，是最難診斷的故障。

7.10.2 接頭的選擇

杜邦接頭：便宜、方便，但接觸電阻大、易鬆脫，僅適合訊號與原型測試，**絕不可用於馬達電流**。XT60 / XT30 接頭：可承載大電流，是電池與馬達的標準選擇。JST-XH：常用於平衡充電與訊號，有防呆設計。螺絲端子台：可靠、可承載大電流，但體積大且需工具。航空接頭：防水防塵、耐震，用於戶外與工業場合。

一個共通原則是**防呆**：不同功能的接頭應使用不同的形式或顏色，讓錯誤的插法在物理上不可能發生。這比在說明書上寫「請注意勿插錯」有效一百倍。

7.11 電子系統除錯方法

7.11.1 系統性的五步驟診斷

面對「機器人不動了」，最沒有效率的做法是立刻開始改程式。正確的順序是由物理層往上、由簡單往複雜檢查。

步驟一：確認有電。用電表量測電池電壓、各降壓模組的輸出、以及 MCU 電源腳位的實際電壓。注意要在**負載狀態**下量測（如開頭故事所示，空載時一切正常）。

步驟二：確認接線。目視檢查所有接頭是否鬆脫、有無短路、極性是否正確。用電表的通斷模式測試可疑的線材（尤其是經過關節反覆彎折的）。

步驟三：確認程式有在跑。加入 LED 閃爍或序列輸出，確認程式執行到哪一行、有沒有卡在某個迴圈或不斷重開機（若序列埠不斷輸出開機訊息，就是重開機）。

步驟四：確認輸入正確。把所有感測器讀值印出來，檢查是否合理。許多「演算法錯誤」其實是輸入資料就已經錯了。

步驟五：才懷疑演算法。前四步都確認無誤後，才進入邏輯除錯。

這個順序的價值在於：前四步都很快（各幾分鐘），而它們能解決約八成的問題。學生最常犯的錯，是直接跳到步驟五，花三小時檢查演算法，最後發現是電池沒電。

7.11.2 三種必備工具

萬用電表：最基本也最重要。用來量電壓（診斷電源）、電阻與通斷（診斷斷線）、以及電流（診斷耗電異常）。學生應學會使用通斷模式快速檢查線路。

邏輯分析儀：可同時觀察多條數位訊號的時序，是診斷 I²C、SPI、UART 通訊問題的利器。低價的 USB 型號僅需數百元，卻能省下無數猜測的時間——它能直接告訴你「主機有沒有發出訊號」「從機有沒有回應」。

示波器：能觀察類比波形，用於診斷電源漣波、雜訊、PWM 波形與訊號完整性。開頭故事中的電壓瞬間下降，用電表只能看到跳動的數字，用示波器則能清楚看到那個下陷的波形。

表 7-4 常見故障症狀與診斷方向

症狀	最可能原因	檢查方法	常見解法
馬達一動就重開機	電源壓降（7.6.1 節）	負載狀態下量 MCU 電源電壓	分離電源、加大電容、換粗線

症狀	最可能原因	檢查方法	常見解法
感測器讀值亂跳	電磁干擾或共地不良	馬達停止時是否正常	訊號線遠離馬達、雙絞、加濾波
I ² C 完全無回應	缺上拉電阻或位址錯誤	邏輯分析儀觀察 SDA/SCL	加 4.7 k Ω 上拉、掃描位址
通訊時通時斷	接頭鬆脫或線材內部斷芯	彎折線材時觀察是否中斷	重新壓接、加應力消除
編碼器計數遺漏	使用輪詢而非中斷	檢查程式架構	改用硬體中斷或編碼器介面
直線走不直	左右馬達死區不同	分別實測兩馬達啟動占空比	死區補償或閉迴路控制
伺服機抖動	電源不足或訊號受干擾	量測伺服電源電壓	獨立供電、加大電容
程式偶爾當機	ISR 過長或 volatile 缺漏	檢查中斷服務常式	縮短 ISR、加 volatile、啟用看門狗

7.12 由電子系統邁向運動學

7.12.1 第一篇的總結

走到這裡，第一篇「機器人的身體與系統」已經完整。回顧這五章：第 3 章建立了系統觀，把機器人拆解為七大子系統並理解它們的關係；第 4 章分析了機構與自由度，決定機器人能做什麼的物理上限；第 5 章選定了致動器，賦予它力量；第 6 章配置了感測器，賦予它感知；第 7 章則把這一切用電子系統連接起來，讓訊號與能量正確流動。

此刻你應該能夠：拿到一份機器人規格需求，畫出系統方塊圖、計算自由度、選定馬達與減速機、配置感測器組合、設計電源架構與通訊介面，並列出完整的零件清單與成本。這已經是一位機器人系統工程師的基本功。

7.12.2 接下來：從「能動」到「精確地動」

但是，能組出一台會動的機器人，與能讓它精確地走到指定位置，中間還隔著一整片數學的海洋。第 8 章開始的第二篇，我們將回答一個看似簡單卻極為深刻的問題：**當每個關節轉了某個角度時，末端執行器究竟在空間中的哪個位置？**

這個問題的答案需要一套精確描述空間的語言——座標系、旋轉矩陣、齊次變換。它將把

你在本篇中建立的物理直覺，轉化為可以計算、可以最佳化、可以交給電腦執行的數學模型。第 4 章數過的自由度，將在第 9 章變成矩陣連乘的維度；第 5 章算過的扭力，將在第 13 章成為動力學方程的一項；第 6 章讀到的編碼器數值，將成為運動學計算的輸入。

換句話說：第一篇教你把機器人組起來，第二篇開始教你把它算清楚。

完整計算例題

例題 7-1 電源壓降診斷

題目：某循跡車使用 7.4 V 鋰電池（內阻 $0.12\ \Omega$ ），經 5 V 降壓模組供給 ESP32（工作電壓下限 3.0 V，模組本身需輸入至少 6.0 V 才能穩壓）。馬達與 ESP32 共用同一條電源線，導線與接頭總電阻 $0.08\ \Omega$ 。兩顆馬達正常行進時各抽 0.8 A，堵轉時各抽 3.5 A。（a）求正常行進與堵轉時的電池端電壓。（b）判斷是否會造成 ESP32 重開機。（c）提出兩種改善方案並計算改善後的結果。

解（a）：正常行進總電流 $= 0.8 \times 2 + 0.1\ (\text{ESP32}) = 1.7\ \text{A}$ 。

$$V_{\text{drop}} = 1.7 \times (0.12 + 0.08) = 1.7 \times 0.20 = 0.34\ \text{V} \rightarrow \text{電池端} = 7.4 - 0.34 = 7.06\ \text{V}$$

正常行進時的壓降。

堵轉時總電流 $= 3.5 \times 2 + 0.1 = 7.1\ \text{A}$ 。

$$V_{\text{drop}} = 7.1 \times 0.20 = 1.42\ \text{V} \rightarrow \text{電池端} = 7.4 - 1.42 = 5.98\ \text{V}$$

堵轉時的壓降。

解（b）：降壓模組需要至少 6.0 V 輸入才能維持 5 V 輸出。堵轉時電池端只剩 5.98 V，已低於門檻，模組輸出將開始下降，ESP32 電源不穩而重開機。**會發生故障**，且恰好落在臨界點附近——這解釋了為何有時可以、有時不行（電池電量與摩擦條件的微小變化即可跨越門檻）。

解（c）：方案一，降低線路電阻。改用較粗的導線與 XT60 接頭，將 $0.08\ \Omega$ 降至 $0.03\ \Omega$ 。堵轉壓降 $= 7.1 \times 0.15 = 1.07\ \text{V}$ ，電池端 $= 6.33\ \text{V} > 6.0\ \text{V}$ ，通過但餘裕不足。

方案二，電源分離。將 ESP32 改用獨立電池或在其電源支路加裝大電容與二極體隔離。此時馬達電流不再影響 ESP32 的供電路徑，即使馬達堵轉，ESP32 仍維持穩定——這是最徹底的解法。

方案三，更換電池。改用內阻 $0.05\ \Omega$ 的新電池，堵轉壓降 $= 7.1 \times 0.13 = 0.92\ \text{V}$ ，電池端 $= 6.48\ \text{V}$ ，改善明顯。建議同時採用方案一與方案二。

例題 7-2 ADC 量測設計

題目：欲以 ESP32 (12 位元 ADC、參考電壓 $3.3\ \text{V}$) 監測 $12\ \text{V}$ 鋰電池的電壓，需以分壓電阻降壓。(a) 設計分壓比並選擇電阻值。(b) 求電壓量測的解析度。(c) 若分壓電阻選用 $100\ \text{k}\Omega$ 與 $33\ \text{k}\Omega$ ，可能產生什麼問題？(d) 此電路的持續耗電為多少？

解 (a)：電池滿電約 $12.6\ \text{V}$ ，需降至 $3.3\ \text{V}$ 以下，取安全裕度設計為滿量程對應 $13.2\ \text{V}$ ，分壓比 $= 3.3/13.2 = 1/4$ 。取 $R1 = 30\ \text{k}\Omega$ (上)、 $R2 = 10\ \text{k}\Omega$ (下)：

$$V_{\text{out}} = V_{\text{in}} \times R2/(R1+R2) = 13.2 \times 10/40 = 3.3\ \text{V}$$

分壓比 1:4，滿足量程需求。

解 (b)：ADC 解析度 $\text{LSB} = 3.3/4096 \approx 0.806\ \text{mV}$ ，換算回電池端需乘上分壓比的倒數 4：

$$\text{電壓解析度} = 0.806\ \text{mV} \times 4 \approx 3.22\ \text{mV}$$

可分辨約 3.2 毫伏的電池電壓變化，對電量監測綽綽有餘。

解 (c)： $100\ \text{k}\Omega$ 與 $33\ \text{k}\Omega$ 的組合分壓比雖然接近 ($33/133 \approx 1/4$)，但輸出阻抗過高 (約 $25\ \text{k}\Omega$)。ESP32 的 ADC 建議源阻抗低於 $10\ \text{k}\Omega$ ，否則取樣電容充電不足，會導致讀值偏低且不穩定。這正是 7.3.3 節第一個陷阱。解法是降低電阻值 (如 $30\ \text{k}\Omega/10\ \text{k}\Omega$ ，輸出阻抗 $7.5\ \text{k}\Omega$) 或加一級電壓隨耦器。

解 (d)：以 $30\ \text{k}\Omega + 10\ \text{k}\Omega$ 計算，總電阻 $40\ \text{k}\Omega$ ：

$$I = 12.6 / 40000 \approx 0.315\ \text{mA}$$

持續耗電約 0.32 毫安。若電池為 $2000\ \text{mAh}$ ，此電路單獨可消耗約 6300 小時，影響甚微。但若追求極低待機功耗，可加入 MOSFET 在不量測時切斷分壓電路。

例題 7-3 PWM 解析度與死區補償

題目：某直流馬達由 8 位元 PWM (0~255) 驅動，實測左馬達在占空比 48 以下不轉、右馬達在 38 以下不轉，兩者在 255 時轉速相同。(a) 說明此差異對直線行走的影響。(b) 設計死區補償的映射公式。(c) 補償後，控制指令 0~100% 對應的實際 PWM 值為何 (以左馬達為例，計算 0%、50%、100%)。

解 (a)：若程式對兩馬達下達相同的 PWM 值 (例如 45)，左馬達完全不轉、右馬達已開

始轉動，機器人會原地打轉。即使 PWM 值高於兩者的死區（例如 100），因起始點不同，兩馬達的實際轉速仍有差異，導致機器人持續偏向一側。

解（b）：把控制指令 u （0~100%）線性映射到「死區以上」的可用範圍：

$$\text{PWM} = \text{deadzone} + (255 - \text{deadzone}) \times (u / 100) \quad (u > 0 \text{ 時})$$

$u = 0$ 時直接輸出 0（停止），避免馬達在死區內發熱卻不轉。

解（c）：左馬達 $\text{deadzone} = 48$ 。 $u = 0\%$ 時 $\text{PWM} = 0$ （停止）； $u = 50\%$ 時 $\text{PWM} = 48 + 207 \times 0.5 = 48 + 103.5 \approx 152$ ； $u = 100\%$ 時 $\text{PWM} = 48 + 207 = 255$ 。右馬達則以 38 代入， $u = 50\%$ 時 $\text{PWM} = 38 + 217 \times 0.5 \approx 147$ 。

補償後，同樣的控制指令會讓兩馬達產生接近的轉速，直線行走顯著改善。但這只是開迴路補償，仍無法對抗負載變化與電池電壓下降——真正的解法是加裝編碼器做閉迴路控制（第 16 章）。

程式範例：中斷式編碼器與安全監控

以下 Arduino/ESP32 風格的 C++ 程式示範本章三個核心概念：中斷式編碼器計數（含 volatile 與臨界區保護）、死區補償的 PWM 輸出、以及包含看門狗與通訊超時的安全監控。

```
#include <Arduino.h>

// ----- 接腳定義 -----
const int ENC_A = 2, ENC_B = 3;          // 編碼器（需為中斷腳）
const int PWM_PIN = 5, DIR_PIN = 4;      // 馬達驅動
const int ESTOP_PIN = 7;                 // 急停狀態偵測（常閉，低電位=正常）

// ----- 中斷共用變數必須加 volatile -----
volatile long encoder_count = 0;
volatile bool estop_triggered = false;

// ----- 中斷服務常式：極短，只做必要的事 -----
void IRAM_ATTR encoderISR() {
    // 正交解碼：由 B 相狀態判斷方向
    if (digitalRead(ENC_B) == HIGH) encoder_count++;
    else encoder_count--;
    // 禁忌：此處不可用 delay()、Serial.print()、浮點運算
}

void IRAM_ATTR estopISR() {
```



```

    estop_triggered = true;          // 只設旗標，處置交給主迴圈
}

// ----- 安全讀取多位元組變數 (避免讀到破碎值) -----
long readEncoderSafely() {
    noInterrupts();                 // 進入臨界區
    long value = encoder_count;
    interrupts();                   // 離開臨界區
    return value;
}

// ----- 死區補償的馬達輸出 (例題 7-3) -----
const int DEADZONE = 48;           // 實測值，每顆馬達不同
void setMotor(float percent) {     // percent: -100 ~ +100
    if (fabs(percent) < 1.0) {     // 指令過小則直接停止
        analogWrite(PWM_PIN, 0);
        return;
    }
    digitalWrite(DIR_PIN, percent > 0 ? HIGH : LOW);
    int pwm = DEADZONE + (255 - DEADZONE) * (fabs(percent) / 100.0);
    analogWrite(PWM_PIN, constrain(pwm, 0, 255));
}

// ----- 主程式 -----
unsigned long last_cmd_time = 0;    // 上位機最後通訊時間
const unsigned long COMM_TIMEOUT = 500; // 通訊超時 500 ms

void setup() {
    Serial.begin(115200);
    pinMode(ENC_A, INPUT_PULLUP);   // 內建上拉，避免浮空
    pinMode(ENC_B, INPUT_PULLUP);
    pinMode(ESTOP_PIN, INPUT_PULLUP); // 常閉接點：斷線時變高 = 觸發
    pinMode(PWM_PIN, OUTPUT);
    pinMode(DIR_PIN, OUTPUT);

    attachInterrupt(digitalPinToInterrupt(ENC_A), encoderISR, RISING);
    attachInterrupt(digitalPinToInterrupt(ESTOP_PIN), estopISR, CHANGE);
    last_cmd_time = millis();
}

void loop() {
    // 安全檢查一：急停
    if (estop_triggered || digitalRead(ESTOP_PIN) == HIGH) {
        setMotor(0);
        Serial.println("[緊急停止] 動力已切斷，請排除後重置");
        delay(200);
        return;                      // 不執行任何控制邏輯
    }

    // 安全檢查二：通訊超時 (上位機當機時自動停止，見第 3 章)

```

```

if (millis() - last_cmd_time > COMM_TIMEOUT) {
    setMotor(0);
    Serial.println("[通訊逾時] 上位機失聯，馬達已停止");
    delay(200);
    return;
}

// 正常控制流程
long count = readEncoderSafely();
float target_percent = 40.0;          // 實際應由上位機指令或 PID 決定
setMotor(target_percent);

static unsigned long t_print = 0;    // 序列輸出放主迴圈，絕不放 ISR
if (millis() - t_print > 200) {
    Serial.printf("編碼器: %ld 輸出: %.1f%%\n", count, target_percent);
    t_print = millis();
}
}

```

請注意程式中體現的四個本章原則：一、ISR 極短且只設旗標；二、共用變數宣告 `volatile` 並以臨界區保護多位元組讀取；三、輸入腳啟用上拉避免浮空；四、急停採常閉接點（斷線即觸發，失效導向安全）並在軟體端加上通訊超時保護。

實作：電源壓降實測與改善

本章實作是重現並解決開頭故事的問題，親眼看見「電的謊言」。所需器材：附錄 F 的循跡車或任何雙馬達平台、萬用電表（有示波器更佳）、不同粗細的導線、電解電容（470 ~ 1000 μF ）、以及一顆額外的電池或行動電源。

- 基準量測：**將電表接在 MCU 的電源腳與地之間。輪子懸空、馬達空轉，記錄電壓。
- 負載測試：**把車放在地面並用手輕壓車體增加負載（或讓輪子頂住牆），記錄電壓的最低值。若電表有最小值保持功能請啟用；若有示波器，記錄電壓下陷的波形與持續時間。
- 臨界點測試：**逐步增加負載直到 MCU 重開機（觀察 LED 或序列輸出）。記錄此時的電壓，與 MCU 規格書的工作電壓下限比較。
- 改善一：加電容。**在 MCU 電源入口並聯一顆 470 μF 電解電容（注意極性），重複步驟 2 ~ 3，記錄改善幅度。
- 改善二：換粗線。**將馬達電源線換成更粗的導線並縮短長度，重複量測。
- 改善三：電源分離。**以獨立電池供給 MCU（務必與馬達電源共地），重複量測。

7. **綜合比較**：製作表格比較四種情況（原始、加電容、換粗線、電源分離）下的最低電壓與是否重開機，並依 7.6.1 節的公式反推各情況下的等效線路電阻。
8. **延伸觀察**：在馬達運轉時，用電表量測感測器的讀值波動幅度，比較改善前後的差異——你會發現電源改善同時也降低了感測雜訊。

國中小也看得懂：為什麼機器人會突然昏倒

你有沒有遇過這種情形：家裡開冷氣的時候，電燈會忽然暗一下？機器人身上也會發生一樣的事情！

馬達是一個非常「吃電」的傢伙。當它開始轉動、或是被卡住轉不動的時候，會一口氣把電池的電吸走很多。這時候，電池能給其他零件的電就不夠了——機器人的小電腦（控制板）吃不飽，就會突然「昏倒」，然後重新開機。這就是為什麼有時候機器人明明程式沒問題，卻走一走就停住重來。

怎麼辦呢？有三個好方法。第一，**用粗一點的電線**——就像喝珍珠奶茶要用粗吸管一樣，電線太細，電就跑不過去。第二，**裝一顆電容**——它像一個小水塔，平常存一點電，馬達突然大口喝電的時候，它就趕快補上去。第三，**分開供電**——給小電腦一顆自己的電池，這樣馬達再怎麼吃電也搶不到它的。

還有一件最重要的事：**急停按鈕一定要是真的切斷電**，不能只是叫程式停下來。因為萬一程式當掉了，它就不會理你按按鈕了。真正的急停按鈕是直接把電切斷，就算程式壞掉，機器人也一定會停。做機器人的時候，記得先想「怎麼讓它安全地停下來」，再想「怎麼讓它動起來」喔！

叫獸提醒與常見錯誤

- 誤區一：一遇到問題就改程式。超過半數的「程式問題」其實是電源或佈線問題，應依 7.11.1 節的五步驟由物理層往上診斷。
- 誤區二：只在空載時量測電源電壓。壓降只在馬達出力時才出現，必須在負載狀態下量測。
- 誤區三：5 V 與 3.3 V 系統直接互接。必須使用位準轉換器或確認接腳為 5 V 容忍，否則可能燒毀晶片。
- 誤區四：輸入接腳浮空。未使用上拉或下拉的輸入腳讀值隨機，所有按鈕與開關都應啟用上拉。

- 誤區五：忘記 I²C 的上拉電阻。這是 I²C 完全無回應的第一名原因；反之多個模組並聯時上拉過小也會失敗。
- 誤區六：在 ISR 中使用 delay 或 Serial.print。ISR 必須極短，只做讀值、計數與設旗標。
- 誤區七：中斷共用變數未宣告 volatile。編譯器最佳化會導致主程式讀到過期的值；多位元組變數還需臨界區保護。
- 誤區八：用主迴圈的延遲函式決定控制週期。應改用計時器中斷，否則週期會隨程式負載飄移，破壞控制器設計的前提。
- 誤區九：把急停做成軟體功能。急停必須是硬體迴路直接切斷動力，並採常閉接點以達成失效導向安全。
- 誤區十：訊號線與馬達線網在一起走。電容與電感耦合會嚴重汙染訊號，應分開走線並使用雙絞。
- 誤區十一：用杜邦線傳輸馬達電流。杜邦接頭接觸電阻大且易鬆脫，大電流應使用 XT60 或端子台。
- 誤區十二：線束拉直後剛好夠長。經過關節的線束必須保留活動餘量並做防磨保護，否則反覆彎折後斷裂或磨破短路。

課堂討論

1. 開頭故事中，若小婕在比賽前一天做了哪一項簡單的測試，就能提前發現問題？請設計一個五分鐘可完成的「賽前健檢」程序。
2. 為什麼安全用的限位開關要用常閉接點而非常開接點？請以「電線斷裂」與「接頭鬆脫」兩種失效情境分別說明後果。
3. 有人主張「反正有看門狗，程式當機會自動重啟，不需要急停」。請反駁這個說法，並舉出至少兩種看門狗無法處理的危險情境。
4. 一台機器人的 I²C 感測器在馬達不轉時讀值正常、馬達一轉就通訊失敗。請列出三種可能原因與對應的驗證方法。
5. 比較兩種除錯策略：（甲）先看程式碼找邏輯錯誤；（乙）先用電表量測物理量。以本章的故障統計為依據，說明為何後者更有效率，並討論什麼情況下前者才是對的。

章末摘要

- MCU 與 SBC 的根本差異在確定性而非速度：MCU 裸機執行、時序可預測，適合即時控制；SBC 有作業系統排程延遲，適合規劃與感知。
- 數位輸入需注意電壓準位相容（5 V 與 3.3 V 不可直接互接）、避免浮空（啟用上拉或下拉）、以及機械開關的彈跳消除。
- 安全相關的開關應採常閉接點，以達成失效導向安全——斷線時系統停止而非繼續運轉。
- ADC 解析度 $LSB = V_{ref}/2^n$ ，量化誤差為 $\pm 0.5 LSB$ ；提升有效解析度可縮小參考電壓、放大訊號或過取樣平均。
- ADC 三大陷阱：輸入阻抗過高、混疊（需類比抗混疊濾波器）、以及參考電壓受電源波動污染。
- PWM 等效電壓 = 電源電壓 $\times$ 占空比；頻率建議 16 ~ 25 kHz（超出人耳）；馬達存在啟動死區，需補償或以閉迴路控制。
- 四種通訊介面：UART 點對點、I²C 省線但短距且需上拉、SPI 高速但耗接腳、CAN 差動抗雜訊且有優先權仲裁。
- 電源壓降 $V_{drop} = I \times (R_{int} + R_{wire})$ 是「馬達一動就重開機」的根本原因；電池老化與負載增加會共同推過臨界點。
- 電源架構四原則：功率與訊號分離、單點共地、就近去耦（0.1 μF 陶瓷 + 大電解）、穩壓與保護。
- EMI 有傳導、電容、電感三條耦合路徑；雙絞線是免費且極有效的抑制手段，最徹底的解法是電氣隔離。
- 中斷是編碼器計數與精確計時的唯一正解；ISR 三禁忌為延遲函式、序列輸出與複雜運算，共用變數須加 volatile。
- 控制週期應由計時器中斷觸發而非主迴圈延遲；看門狗應在確認系統健康處餵食才有意義。
- 安全設計四層次：本質安全、硬體保護、軟體監控、操作規範；急停必須是硬體迴路且不經軟體。
- 除錯五步驟：有電 $\rightarrow$ 接線 $\rightarrow$ 程式在跑 $\rightarrow$ 輸入正確 $\rightarrow$ 才懷疑演算法；前四步能解決約八成的問題。

觀念題與計算題

觀念題

1. 說明 MCU 與 SBC 在確定性上的差異，並各舉兩個適合的機器人任務。
2. 什麼是浮空輸入？為什麼會造成讀值不穩？如何解決？
3. 為什麼安全用限位開關應採常閉接點？請以電線斷裂的情境說明。
4. 說明開關彈跳現象及其兩種消除方法。
5. 計算 12 位元 ADC 搭配 3.3 V 參考的解析度，並說明降低參考電壓為何能提升有效解析度。
6. 什麼是混疊？為什麼它必須在 ADC 之前用類比濾波器處理，而不能靠軟體補救？
7. 說明 PWM 的原理與等效電壓公式，並解釋為何頻率建議設在 16 kHz 以上。
8. 什麼是馬達的啟動死區？它如何造成循跡車直線走不直？請提出開迴路與閉迴路兩種解法。
9. 比較 UART、I²C、SPI 與 CAN 的線數、速度、距離與節點數，並各舉一個典型用途。
10. 寫出電源壓降的公式，並解釋為何「昨天可以跑、今天不行」是合理的現象。
11. 說明電源架構的四個原則，並解釋單點共地與接地迴路的差異。
12. 說明 EMI 的三條耦合路徑及其各自的抑制方法，並解釋雙絞線為何有效。
13. 列出中斷服務常式的三大禁忌，並說明 volatile 關鍵字的作用。
14. 為什麼急停必須是硬體迴路？請說明軟體急停在什麼情況下會失效。
15. 說明電子除錯的五個步驟，並解釋為何「先改程式」是最沒效率的做法。

計算題

1. 某機器人使用 11.1 V 鋰電池（內阻 $0.10\ \Omega$ ），導線與接頭電阻 $0.06\ \Omega$ 。四顆馬達正常運轉各抽 1.2 A，堵轉各抽 5.0 A，控制系統另耗 0.5 A。（a）求正常與全部堵轉時的電池端電壓。（b）若降壓模組需至少 9 V 輸入，堵轉時是否會失效？（c）若要在全部堵轉時仍維持 9 V 以上，總線路電阻（含電池內阻）最多可為多少？
2. 某系統以 10 位元 ADC（參考 5 V）量測力覺感測器，感測器輸出為 0~20 mV。（a）不放大大時，滿量程對應幾個 ADC 階層？（b）解析度對應多少毫伏？（c）若要讓滿量程佔滿整個 ADC 範圍，放大器增益應為多少？（d）放大後的解析度為何？改善了幾倍？
3. 某馬達以 20 kHz PWM 驅動，電源 12 V，占空比 65%。（a）求 PWM 週期（微秒）與高

電位持續時間。(b) 求等效電壓。(c) 若使用 8 位元 PWM 暫存器，65% 對應的設定值為何？(d) 若改用此 8 位元 PWM 控制 RC 伺服機 (50 Hz、脈寬 1~2 ms)，可用的角度階層有幾階？解析度為幾度？此結果說明什麼？

4. 某 CAN 匯流排以 500 kbps 運作，連接 8 個馬達驅動器節點，每個節點每 5 ms 回報一次狀態（每則訊息含協定開銷共 130 位元），上位機每 10 ms 對每個節點下達一次指令（同樣 130 位元）。(a) 求匯流排的總資料率與使用率。(b) 若建議使用率上限為 40%，此設計是否合格？(c) 最多可再增加幾個相同的節點？

動手做與專題延伸

本章實作：機器人電氣健檢表

為你手上的機器人（或附錄中的任一台）完成下列電氣健檢。這份表格應成為每次比賽或展示前的例行程序。

表 7-5 機器人電氣健檢表（第一列為範例）

檢查項目	檢查方法	合格標準	實測結果	處置
MCU 負載電壓	馬達堵轉時量測電源腳	高於規格下限 10% 以上	4.72 V (下限 4.5 V)	合格，餘裕偏低，建議加電容
電池電量與內阻	充飽後量開路電壓與負載電壓	壓降低於標稱值 10%		
急停功能	運轉中按下急停	馬達立即停止（硬體切斷）		
線束固定與防磨	目視檢查並活動關節	無鬆脫、無磨損、有活動餘量		
接頭牢固度	輕拉每個接頭	無鬆脫		
訊號與功率分離	目視檢查佈線	未網綁在一起、無平行長距離		
感測器雜訊	馬達運轉時觀察讀值波動	波動小於量程的 2%		
通訊穩定度	連續運轉五分鐘觀察	無斷線、無亂碼		

專題延伸

- 為附錄 F 的循跡車實測左右兩顆馬達的啟動死區，實作例題 7-3 的補償公式，並以「直線行

走 3 公尺的橫向偏移量」量化補償前後的改善效果。

- 設計並實作一套完整的急停系統：包含常閉急停按鈕、繼電器或接觸器、狀態偵測回授到 MCU、以及軟體端的處置邏輯。繪製電路圖並實測「按下到馬達完全停止」的時間。
- 進階：用邏輯分析儀或示波器診斷一個真實的通訊問題。刻意製造三種故障（拔除 I²C 上拉電阻、將訊號線與馬達線網在一起、鮑率設定不一致），分別觀察並記錄波形特徵，整理成一份「通訊故障波形辨識手冊」——這將是你未來除錯時最有價值的參考資料。

參考文獻與延伸閱讀

- Horowitz, P. and Hill, W., The Art of Electronics, 3rd ed., Cambridge University Press (電子電路的經典聖經)。
- Ott, H. W., Electromagnetic Compatibility Engineering, Wiley (EMI 與接地設計的權威著作)。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (機器人硬體與系統整合)。
- Yiu, J., The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes (MCU 架構與中斷機制)。
- Valvano, J. W., Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers (嵌入式即時系統設計)。
- Barr, M. and Massa, A., Programming Embedded Systems, 2nd ed., O'Reilly (嵌入式 C 程式設計與除錯)。
- IEC 60204-1：機械電氣設備安全通則；ISO 13850：緊急停止裝置設計原則。
- ISO 13849-1：控制系統安全相關部件的設計（安全等級 PL 與風險評估）。
- CAN in Automation (CiA) 技術文件與 Bosch CAN Specification 2.0。
- 各晶片原廠技術文件：STM32 參考手冊、ESP32 技術參考手冊、Arduino 官方文件與應用筆記。